Artificial Intelligence

Explore the Intelligence, Master the Future

Nguyễn Văn Diêu

HO CHI MINH CITY UNIVERSITY OF TRANSPORT

2027

Kiến thức - Kỹ năng - Sáng tạo - Hội nhập

Sứ mệnh - Tầm nhìn

Triết lý Giáo dục - Giá trị cốt lõi

Outline I

1. Artificial Intelligence?
   - 1.1 What is Artificial Intelligence?
   - 1.2 Definition
2. Intelligence Agents
   - 2.1 Introduction to Agents
   - 2.2 Specifying the Task Environment (PEAS Framework)
   - 2.3 Environment Characteristics
   - 2.4 Types of Agent Structures
     1. Simple reflex Agents
     2. Model-based reflex Agents
     3. Goal-based Agents
     4. Unity-based Agents
     5. Learning Agents
   - 2.5 From Traditional Agents to Agentic AI
   - 2.6 Case Study: The Automated Taxi Agent

Outline II

2.7 Intelligent Agents

3 Solving Problems by Searching
3.1 Problem-Solving Agents
3.2 Romania Problem
3.3 Search problem Components
3.4 Search Algorithms
3.5 Best First Search
3.6 Uninformed Search
- Breadth First Search
- Uniform Cost Search (UCS)
- Depth First Search (DFS)
- Depth Limited Search

3.7 Informed Search Strategies
- Greedy Best First Search
- A* Search

Outline III

3.8 Complex Environments
- Hill Climbing Search
- Random Restart Hill Climbing
- Simulated Annealing
- Genetic Algorithms
- Local Search in Continuous Spaces
- Search with Nondeterministic Actions
- Searching with no observation
- Searching in Partially Observable Environments
- Solving Partially Observable Problems
- Agent for Partially Observable Environments
- Online Search Agents and Unknown Environments

4 CSP

4.1 Definitions

4.2 Map Coloring

4.3 Variations on the CSP

Outline IV

4.4 Real-World CSP
4.5 Constraint Graphs
4.6 Constraint Propagation
- Local Consistency
- Types of Local Consistency

4.7 Constraint Propagation Algorithm
- Node Consistency
- Arc Consistency
- Path Consistency

4.8 Global Constraints
- Sudoku

4.9 Backtracking Search
- Example without heuristic

4.10 Local Search for CSP

5 Knowledge

Outline V

5.1 Knowledge-Base Agents
5.2 The Wumpus World
5.3 Fundamentals of Logic
5.4 Propositional Logic
- Syntax
- Semantics
- The Simple Inference Procedure
- KB for Wumpus World

5.5 Theorem Proving
- Standard logical equivalences
- Inference Rules and Proofs
- Inference for Wumpus World
- Proof by Resolution
- CNF
- Resolution Alg.
- Horn clauses and Definite clauses

Outline VI

FW and BW Chaining
Forward chaining
Backward chaining
DPLL Algorithm

6 Machine Learning
6.1 Learning from Examples
6.2 Forms of Learning
6.3 Main Types of Learning
6.4 The Theory of Learning
6.5 Data Preprocessing
6.6 The Machine Learning Pipeline
6.7 Supervised Learning
6.8 Linear Regression
6.9 Gradient descent algorithm

Outline VII

6.10 Polynomial Regression

6.11 Logistic Regression

6.12 Classification

- Binary Classification
- Naïve Bayes Classifier
- Review Probability Distribution
- Gaussian Naïve Bayes
- Multinomial Naïve Bayes
- Bernoulli Naïve Bayes

6.13 Decision Tree

- Decision Tree Overview
- Decision Tree Model
- Regression Decision Tree algo.
- Classification Decision Tree algo.

6.14 Nonparametric Models

- Nearest-neighbor models

Outline VIII

Support Vector Machine

6.15 Ensemble Learning

Wisdom of the Crowd

Majority Voting

Random Forest

6.16 Unsupervised Learning

K-Means Algorithm

Principal Component Analysis

6.17 Neural Networks

Biological Neuron

Rosenblatt's Perceptron

Perceptron Training Rule

Perceptron learning AND gate

XOR Problem

Multilayer Networks

Backward Propagation of Errors

Outline IX

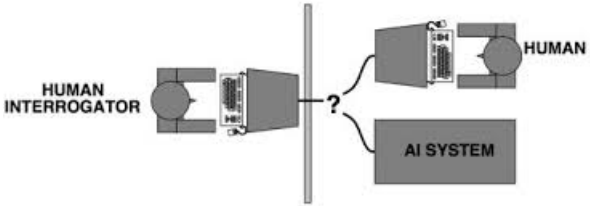
7 References

What is Artificial Intelligence?

- **Four Approaches (The AIMA Framework):**
  - **Thinking Humanly:** Modeling the cognitive processes of the human mind.
  - **Acting Humanly:** Passing the Turing Test (imitating human behavior).
  - **Thinking Rationally:** Using logic and "laws of thought" to solve problems.
  - **Acting Rationally:** The **Rational Agent** approach = acting to achieve the best outcome.
- **Key TakeAway:** Modern AI focuses on Acting Rationally - designing agents that perceive and act within an environment.

The Turing Test & Its Modern Context

- **The Turing Test:**



  - **Interrogator** posing written questions to **Human** and **Computer**.
  - **Human** and **Computer** response.
  - **Computer** pass the **Test** if **Interrogator** cannot know responses come from a **Human** or from a **Computer**.
  - **Film about Alan Turing:** **The Imitation Game (2014)**

The Turing Test & Its Modern Context

- **Components of the Test:**
  - Natural Language Processing (NLP).
  - Knowledge Representation.
  - Automated Reasoning.
  - Machine Learning.
- **Discussion:** Have Large Language Models (LLMs) like GPT-4 finally "passed" the Turing Test? Or is the test becoming obsolete?

Artificial Intelligence: Deffinition

- "The study and design of intelligent agents, where an intelligent agent is a system that perceives its environment and takes actions that maximize its chances of success."

  Russel and Norvig AI book.
- "The capability of computer system or algorithms to imitate intelligent human behavior"

  First known use in 1955, Webster's Dictionary.
- "The study of how to make computers do intelligent things that people can do, such as think and make decisions"

  Longman's Dictionary.
- "The theory and development of computer systems able to perform tasks normally requiring human intelligence, such as visual perception, speech recognition, decision-making, and transtation between languages"

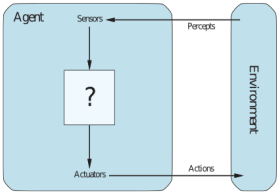
  Oxford's Dictionary.

Introduction to Agents

- **What is an Agent?** Anything that can be viewed as perceiving its **environment** through **sensors** and acting upon that environment through **actuators**.
- **The Interaction Loop:**
  - **Percepts:** Sensory input at any given instant.
  - **Actions:** What the agent does to change the environment.

Introduction to Agents

- **Agent:** is anything, can be viewed as:
  - **perceiving** its **environment** through **sensors**
  - **acting** upon that **environment** through **actuators**



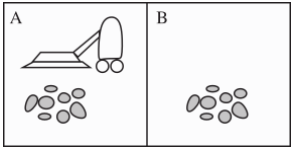
- Agent runs in cycles: (1) **Perceive**, (2) **Think**, (3) **Act**
- **Agent** = **Architecture** + **Program**

Introduction to Agents

e.g.

- **Human Agent:**
  - Sensors: eyes, ears, ...
  - Actuators: hands, legs, mouth, ...
- **Robotic agent:**
  - Sensors: Cameras and infrared range finders.
  - Actuators: Various motors.
- **Agents everywhere!**
  - Cell phone
  - Vacuum cleaner
  - Robot
  - Self-driving car
  - Human
  - etc.

e.g. Vacuum Cleaner



- Percepts: location and contents e.g., [A, Dirty].
- Actions: Left, Right, Suck, NoOp.
- Agent function: mapping from percepts to actions.

Percept	Action
[A, clean]	Right
[A, dirty]	Suck
[B, clean]	Left
[B, dirty]	Suck

Specifying the Task Environment (PEAS Framework)

Rational agent, we must using the **PEAS** framework:

- **P – Performance:** The criteria that determine how successful the agent is (e.g., safety, speed, profit).
- **E – Environment:** The "place" where the agent operates (e.g., a city road, a virtual warehouse).
- **A – Actuators:** The tools the agent uses to act (e.g., steering wheel, software API, robot arm).
- **S – Sensors:** The inputs the agent uses to see/hear (e.g., cameras, GPS, keyboard input).

Environment Characteristics

The complexity of an agent depends heavily on the nature of its environment:

1. **Fully vs. Partially Observable:** Can the agent see everything at once? (Chess vs. Poker).
2. **Deterministic vs. Stochastic:** Is the next state predictable? (Math problems vs. Traffic).
3. **Static vs. Dynamic:** Does the world change while the agent is "thinking"?
4. **Discrete vs. Continuous:** Are there a limited number of actions/states?

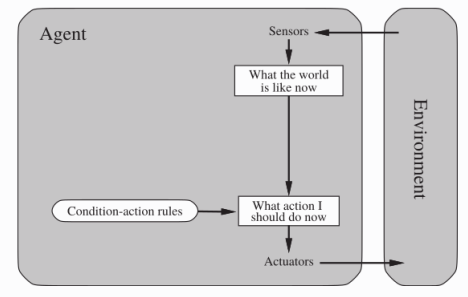
Types of Agent Structures

The complexity of an agent depends heavily on the nature of its environment:

1. **Simple Reflex Agents:** Act only on current percepts (If-Then rules).
2. **Model-based Reflex Agents:** Maintain an "internal state" to track parts of the world they cannot see now.
3. **Goal-based Agents:** Act to achieve specific goals (requires planning).
4. **Utility-based Agents:** Act to maximize "happiness" or a "utility function" (choosing the best path among many).
5. **Learning Agents:** Unlike basic agents that follow fixed rules, a Learning Agent improves its performance over time through experience.

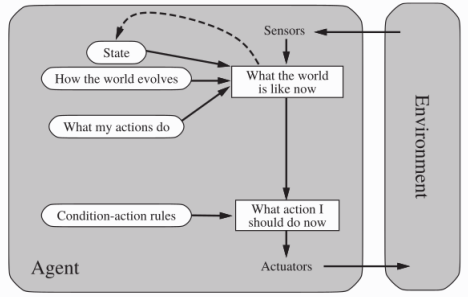
1. Simple reflex Agents

- Simple reflex agents select an action **based on the current state only** ignoring the percept history.
- Simple but limited.
- Can only work if the environment is **fully observable**, that is the correct action is based on the **current** percept only.



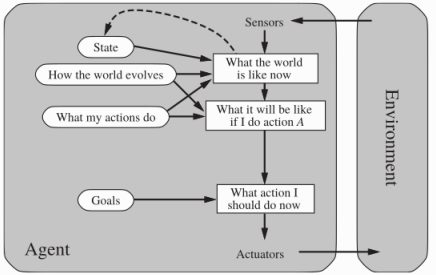
2. Model-based reflex agents

- **Handle partial observability** by keeping track of the part of the world it can't see now.
- Internal state depending on the percept history (**best guess**).
- Model of the world based on (1) how the world evolves independently from the agent, and (2) how the agent actions affects the world.



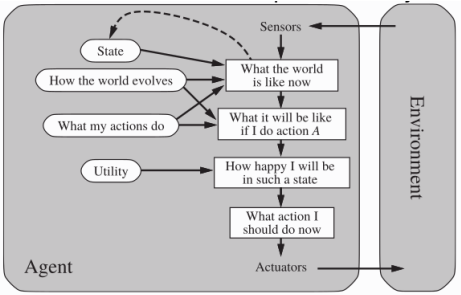
3. Goal-based Agents

- Knowing the current state of the environment is not enough. The agent needs some **goal information**.
- Agent combines the goal information with the environment model to choose the actions that achieve that goal.
- Consider the future with "What will happen if I do A?"
- Flexible as knowledge supporting the decisions is explicitly represented and can be modified.



4. Unity-based Agents

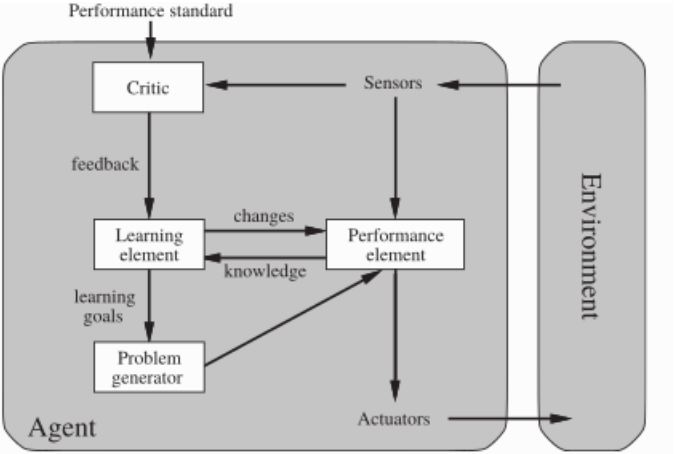
- Sometimes achieving the desired goal is not enough. We may look for quicker, safer, cheaper trip to reach a destination.
- Agent happiness should be taken into consideration. We call it **utility**.
- A utility function is the agent's performance measure.
- Because of the uncertainty in the world, a utility agent choses the action that maximizes the expected utility.



5. Learning Agents

- Programming agents by hand can be very tedious. "Some more expeditious method seem desirable" Alan Turing, 1950.
- Four conceptual components:
  - Learning element: responsible for making improvements.
  - Performance element: responsible for selecting external actions. It is what we considered as agent so far.
  - Critic: How well is the agent is doing w.r.t. a fixed performance standard.
  - Problem generator: allows the agent to explore.

5. Learning agents



From Traditional Agents to Agentic AI

The complexity of an agent depends heavily on the nature of its environment:

- **Traditional Agents:** Often rely on hard-coded rules or limited search algorithms.
- **Agentic AI (Modern):**
  - Reasoning: Uses LLMs to "think" through complex problems.
  - Memory: Uses databases (RAG) to remember past interactions.
  - Tool Use: Can decide to use a calculator, search the web, or write code.
- **Transition:** This is where Reinforcement Learning comes in - teaching the agent to improve its own actions through experience.

Case Study: The Automated Taxi Agent

Applying Agent theory to a real-world complex system.



- **Agent Type:** A high-level Utility-based Agent.
- **Goal:** Transporting passengers safely and efficiently to their destination.

PEAS Analysis for Automated Taxi

Specific Specification:

- **P - Performance:** Safety, reach destination, stay within laws, passenger comfort, maximize profits.
- **E - Environment:** City streets, highways, pedestrians, weather conditions, other unpredictable drivers.
- **A - Actuators:** Steering wheel, accelerator, brakes, signal lights, horn, display screen for passengers.
- **S - Sensors:** Cameras, LiDAR, Radar, GPS, speedometer, engine sensors, microphone (for sirens).

Environment Characteristics (Taxi Context)

Is the road "easy" for an AI?

- **Partially Observable:** You can't see what's around the next corner or behind the truck in front.
- **Stochastic:** You cannot predict exactly when a pedestrian will jump into the street.
- **Dynamic:** The world changes rapidly while the car is calculating its next move.
- **Continuous:** Speed and steering angle are not just "on/off" but have infinite values.
- **Multi-agent:** The car must compete/cooperate with other drivers and pedestrians.

The Evolution of Taxi Intelligence

How would different agent structures handle driving?

1. **Simple Reflex:** If brake lights ahead are red, then hit brakes. (Too dangerous for complex driving).
2. **Model-based:** Keeping track of a car that just entered your blind spot. (Knowing it's still there even if you can't see it).
3. **Goal-based:** Planning a route from Point A to Point B.
4. **Utility-based:** Choosing the best route (fastest vs. safest vs. most fuel-efficient).

The "Agentic AI" Upgrade (Modern Perspective)

How do we move to the next level?

- **Traditional Logic:** Hard-coded rules for every traffic situation (Impossible to cover all "edge cases").
- **Agentic/Modern Approach:**
  - **Reasoning:** "A ball just rolled into the street, a child might follow it soon." (LLM/Reasoning capability).
  - **Learning (RL):** Improving driving smoothness based on thousands of hours of simulation.
  - **Communication:** Explaining to the passenger why the car chose a certain detour.

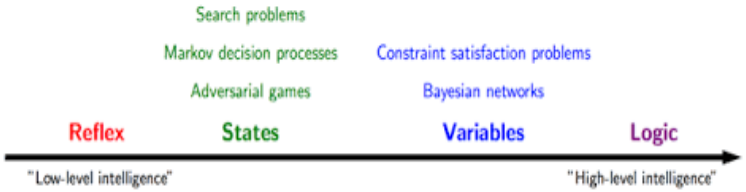
Intelligent Agents

- The concept of intelligent agent is central in AI.
- AI aims to design intelligent agents that are useful, reactive, autonomous and even social and pro-active.
- An agent perceives its environment through percept and acts through actuators.
- A performance measure evaluates the behavior of the agent.
- An agent that acts to maximize its expected performance mea- sure is called a rational agent.
- PEAS: A task environment specification that includes Performance measure, Environment, Actuators and Sensors.

$$\textbf{Agent} = \textbf{Architecture} + \textbf{Program}$$

Intelligent Agents

- Four types of agents: Reflex agents, model-based agents, goal-based agents, and utility-based agents.
- Agents can improve their performance through **learning**.
- This is a high-level present of agent programs.
- States representations: atomic, factored, structured. Increasing expressiveness power.



Credit: Courtesy Percy Liang

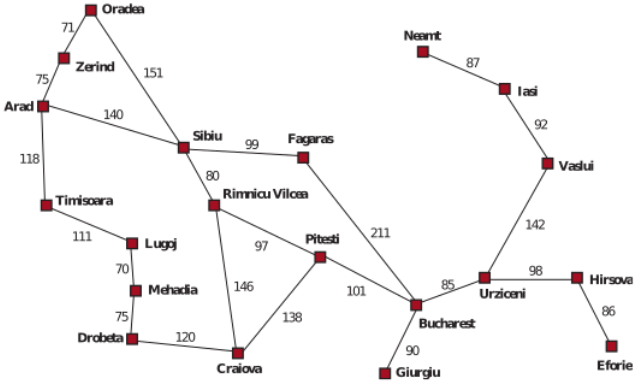
Problem-Solving Agents

A type of **Goal-based Agent** that decides what to do by finding sequences of actions that lead to desirable states.

- **The Process:**
  1. **Goal Formulation:** Defining what we want to achieve.
  2. **Problem Formulation:** Deciding what actions and states to consider.
  3. **Search:** Looking for a sequence of actions (a Path) to the goal.
  4. **Execution:** Performing the actions.
- **Assumptions:** The environment is observable, deterministic, and discrete.

Romania Problem

Agent in **Arad** city and go to **Bucharest** city by road.



A simplified road map of part of Romania.

Search problem Components

A search problem can be defined formally by 5 components:

1. **State (State Space):** All states reachable from the initial state by any sequence of actions (e.g. **All City**).
2. **Initial State:** Where the agent starts (e.g. **Arad**).
3. **Actions:** The set of possible moves. Given a state s, **Action**(s) returns a finite set of actions that can be executed in s.

   e.g. **Action**$(Arad) = \{ToSibiu, ToTimisoara, ToZerind\}$.
4. **Transition Model:** A description of what each action does.

   Result(s, a) returns the state that results from doing action a in state s.

   e.g. **Result**$(Arad, ToZerind) = Zerind$

Search problem Components

A search problem can be defined formally by 5 components:

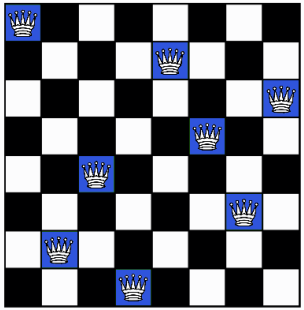
5. **Goal Test:** Determines whether a given state is the goal.

   e.g. *Bucharest*

6. **Path Cost:** Function that assigns a numeric cost to a path w.r.t. performance measure (e.g., distance, time).

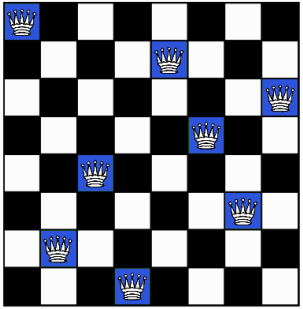
   - **Action cost function**: **A-Cost**$(s, a, s^{'})$ gives numeric cost of applying action a in state s to reach state $s^{'}$

8-Queen Problem



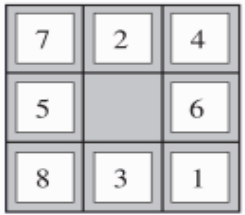
- Place 8 queens so that no queen is attacking any other horizontally, vertically or diagonally.
- Number of possible sequences to investigate:
  $64 * 63 * 62 * ... * 57 = 1.8 * 10^{14}$

8-Queens Problem



1. **Initial state:** Any arrangement of 0 to 8 queens on the board is a state.
2. **States**: No queen on the board.
3. **Actions**: Add a queen to any empty square.
4. **Transition model:** Returns the board with a queen added to the specified square.
5. **Goal test:** 8 queens on the board with none attacked.

8-Puzzle Problem



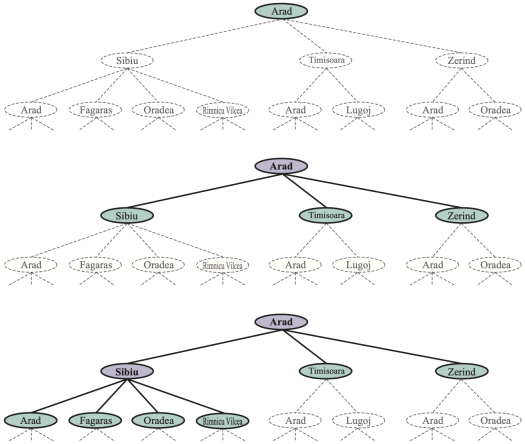
1. **States:** Location of each of the 8 tiles in the 3x3 grid.
2. **Initial state:** Any state.
3. **Actions**: Move Left, Right, Up or Down.
4. **Transition model:** Given a state and an action, returns resulting state.
5. **Goal test:** State matches the goal configuration.
6. **Path cost:** Each step costs 1. Path cost is the number of steps in the path.

Search Algorithms

A search algorithm takes a problem as input and returns a solution in the form of an action sequence.

- **Search Tree:**
  - The root of the tree is the **Initial State**.
  - Branches represent **Actions**.
  - Nodes represent **States** in the **state space**.
- **Key Components of Search Tree:**
  - Expanded: set of Lavender Nodes.
  - Frontier: Generated node but not yet expanded. (Green nodes).
  - Reached nodes = Expanded nodes + Frontier nodes

e.g. Search Tree



3 partial search trees for finding a route from **Arad** to **Bucharest**.

Data Structures

Data structure help to keep track of the search tree.

A node with four components:

- *node*.**State**: the state to which the node corresponds.
- *node*.**Parent**: the node in the tree that generated this node.
- *node*.**Action**: the action that was applied to the parent's state to generate this node;
- *node*.**P-Cost**: the total cost of the path from the initial state to this node.

Data structure of frontier:

- **Empty**(*frontier*)
- **Pop**(*frontier*)
- **Top**(*frontier*)
- **Add**(*node*, *frontier*)

Three type of queue:

- **Priority queue**
- **FIFO queue**
- **LIFO queue**

Best First Search

The Core Concept:
Best-first Search is a general search strategy that chooses the "most promising" node from the Frontier for expansion.

- **Evaluation Function:** It uses a function $f(n)$ to calculate the "desirability" of a node.
- **Selection Logic:** The algorithm always selects the node with the minimum $f(n)$ value.
- **Data Structure:** The Frontier is implemented as a Priority Queue, sorted in ascending order of $f(n)$.

Best-first Search

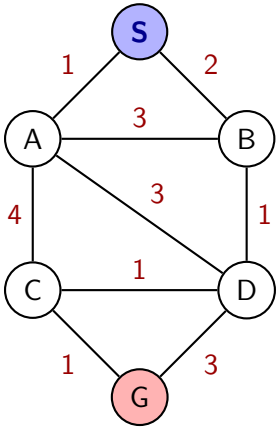
```
func Best-First-Search(problem, f) returns Solution node or Failure
    node ← Node(State = problem.Initial)
    frontier ← a priority Queue by f, node as an element
    reached ← lookup table, with key problem.Initial and value node
    while not Empty (frontier) do
        node ← Pop (frontier)
        if problem.Goal(node.State) then
            return node
        foreach child ∈ Expand (problem, node) do
            s ← child.State
            if s ∉ reached or child.P-Cost < reached[s].P-Cost then
                reached ← child
                add child to frontier
    return Failure
```

function Expand

```
func Expand(problem, node) yields nodes
    s ← node.State
    foreach action ∈ problem.Action(s) do
        s' ← problem.Result(s, action)
        cost ← node.P-Cost + problem.A-Cost(s, action, s')
        yields Node(State = s', Parent = node, Action = action,
                    P-Cost = cost)
```

Example

Best-first Search using Priority Queue.



Best-First Search

No.	Reached	
	Expanded	Frontier [Priority Queue]
0		S(0)
1	**S(0)**	A(1) B(2)
2	A(1)	B(2) C(5) D(4)
3	B(2)	C(5) ~~D(4)~~ D(3)
4	D(3)	~~C(5)~~ C(4) G(6)
5	C(4)	~~G(6)~~ G(5)
6	**G(5)**	

$S \rightarrow B(2) \rightarrow D(3) \rightarrow C(4) \rightarrow G(5)$

Uninformed Search Strategies (Blind Search)

1. **Definition:**

   Uninformed search strategies have no information about the distance from the current state to the goal. They only know:

   - How to generate successor nodes.
   - How to distinguish a goal state from a non-goal state.

Uninformed Search Strategies (Blind Search)

2. **Key Strategies to Cover:**

   - **Breadth-First Search (BFS):** Expands the shallowest nodes first.
   - **Uniform-Cost Search (UCS):** Expands the node with the lowest path cost $g(n)$.
   - **Depth-First Search (DFS):** Expands the deepest nodes first.
   - **Depth-Limited & Iterative Deepening Search:** Hybrids to handle large state spaces.

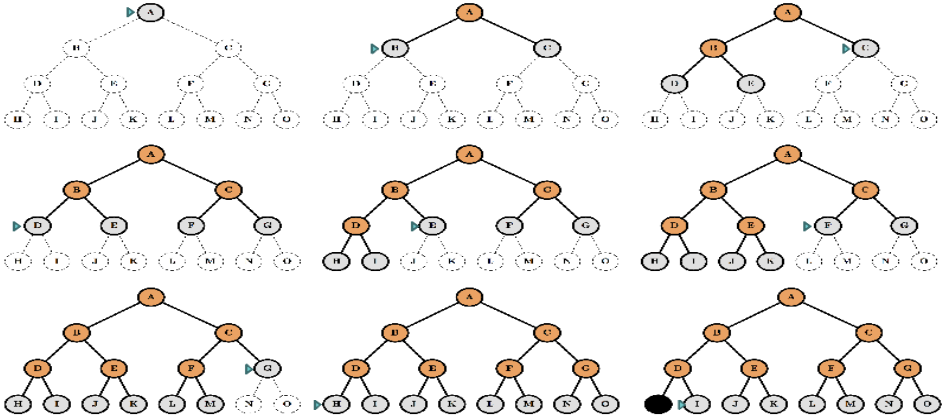
Breadth-First Search

Expand the root node first, then all successors of the root, then their successors, and so on.

- **Mechanism:** Uses a FIFO (First-In-First-Out) queue. New nodes (at the same level) go to the back of the queue.
- **Goal Test:** Applied when a node is generated (not when it is expanded).
- **Performance:**
  - **Completeness:** Yes (if the branching factor b is finite).
  - **Optimality:** Yes (if all step costs are identical, e.g., 1).
  - **Time & Space Complexity:** $O(b^d)$, where d is the depth of the shallowest solution.

Breadth First Search (BFS)

BFS: Expand **shallowest** first.



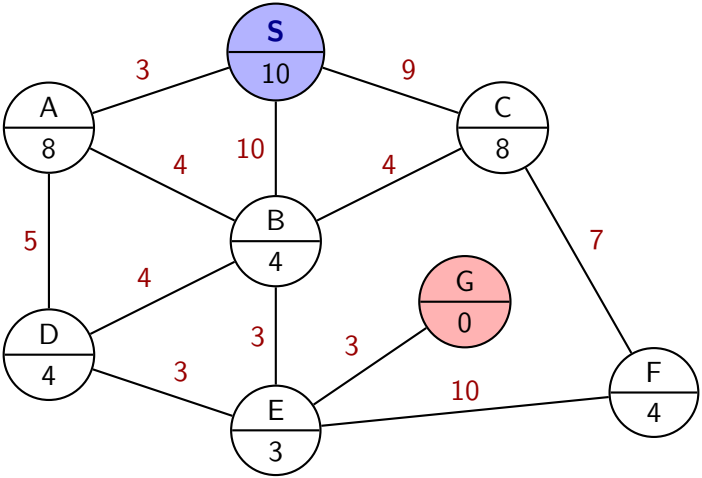
BFS Search algo.

```
func Breadth-First-Search(problem) returns Solution node or Failure
    node ← Node(problem.Initial)
    if problem.Goal (node.State) then
        return Solution (node)
    frontier ← a FIFO queue with node as an element
    reached ← problem.Initial
    while not Empty (frontier) do
        node ← Pop (frontier)
        foreach child ∈ Expand (problem, node) do
            s ← child.State
            if problem.Goal(s) then
                return child
            if s ∉ reached then
                add s to reached
                add child to frontier
    return Failure
```

Expand Function

- Instead of modifying **Expand** function, it is also use in **Breadth-First-Search** function to increase efficiency.
- BFS checks whether a node is a target as soon as it is created (immediately after calling **Expand**), instead of waiting until it is removed from the queue as in conventional **Best-First-Search**.

Graph 01



BFS for Graph 01

Breadth First Search

No.	Reached	
	Expanded	Frontier [FIFO Queue]
0		S
1	S	A B C
2	A	B C D
3	B	C D E
4	C	D E F
5	D	E F
6	E	F G Solution
$S \rightarrow B \rightarrow E \rightarrow G$		

Uniform-Cost Search (UCS)

Uniform Cost search is the Best First search, also call **Dijkstra's algorithm**.

func `Uniform-Cost-Search`(*problem*) **returns** **Solution** or **Failure**

Best-First-Search(*problem*, Path-Cost)

Path-Cost: $f(n) = g(n)$

Depth-first Search (DFS)

Always expands the deepest node in the current frontier.

- **Mechanism:** Uses a LIFO (Last-In-First-Out) stack.
- **Pros:** Very low memory requirement **(**$O(bm)$**)**.
- **Cons:** Not complete (can get stuck in an infinite loop) and not optimal.

DFS Search

```
func Depth-First-Search(problem) returns Solution node or Failure
    node ← Node(problem.Initial)
    frontier ← a LIFO queue with node as an element
    reached ← problem.Initial
    while not Empty (frontier) do
        node ← Pop (frontier)
        if problem.Goal (node.State) then
            return Solution (node)
        foreach child ∈ Expand (problem, node) do
            s ← child.State
            if s ∉ reached then
                add s to reached
                add child to frontier
    return Failure
```

DFS for Graph 01

Depth First Search

No.	Reached	
	Expanded	Frontier [LIFO Queue]
0		S(0)
1	S(0)	C(1) B(1) A(1)
2	A(1)	C(1) B(1) D(2)
3	D(2)	C(1) B(1) E(3)
4	E(3)	C(1) B(1) F(4) G(4)
5	G(4)	

$S(0) \rightarrow A(1) \rightarrow D(2) \rightarrow E(3) \rightarrow G(4)$

Depth Limited Search

1. **The Concept**

   Depth-Limited Search is a variation of Depth-First Search (DFS) that imposes a fixed maximum depth limit, l. Nodes at depth l are treated as if they have no successors.

   - **Purpose:** To prevent DFS from wandering down an infinite path in state spaces that are infinitely deep or contain loops.
   - **Mechanism:** It performs DFS but stops exploring further once the current depth reaches the limit l.

Depth-Limited Search

2. **Possible Outcomes**

When DLS terminates, it can return one of three results:

- **Solution:** The goal was found within the limit.
- **Failure:** The goal does not exist in the state space (exhausted search).
- **Cutoff:** No solution was found within the depth limit l.

Depth Limited Search

3. **Performance Analysis**

   - **Completeness:** No, if the shallowest goal is at a depth $d > l$.
   - **Optimality:** No, if the limit $l > d$ (it might find a non-optimal solution first).
   - **Time Complexity:** $O(b^l)$.
   - **Space Complexity:** $O(bl)$ (Memory efficient, just like DFS).

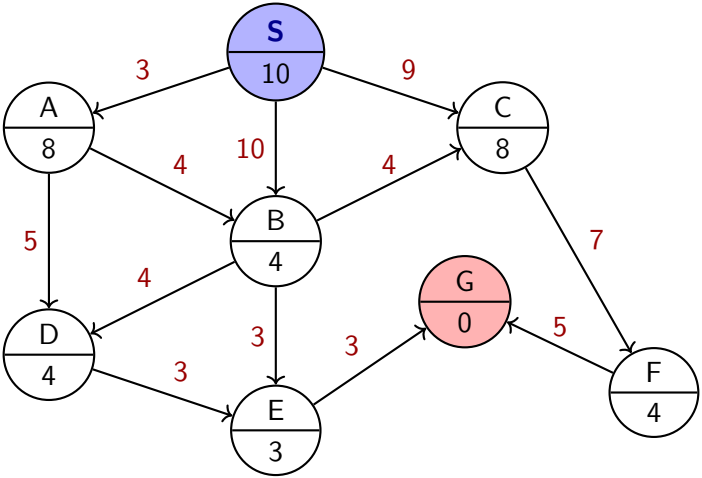
Depth Limited Search

```
func Depth_Limited_Search(problem, l) returns Solution node or Failure or Cutoff
    node ← Node(problem.Initial)
    frontier ← a LIFO queue with node as an element
    result ← Failure
    while not Empty (frontier) do
        node ← Pop(frontier)
        if problem.Goal(node.State) then
            return node
        if Depth (node) > l then
            result ← Cutoff
        else
            if not Cycle (node) then
                foreach child ∈ Expand (problem, node) do
                    add child to frontier
    return result
```

Depth Limited Search

```
func Cycle(problem, node) returns True or False
    curr ← node.Parent
    while curr is not null do
        if curr.State == node.State) then
            return True
        else
            curr ← curr.Parent
    return False
```

Graph 02



DLS for Graph 02 with depth = 1

Depth Limited Search $l = 1$

No.	Reached	
	Expanded	Frontier [LIFO Queue]
0		S(0)
1	S(0)	C(1) B(1) A(1)
2	A(1)	C(1) B(1) D(2) B(2)
3	B(2)	**Cutoff**
4	D(2)	**Cutoff**
5	B(1)	C(1) E(2) D(2) C(2)
6	C(2)	**Cutoff**
7	D(2)	**Cutoff**
8	E(2)	**Cutoff**
9	C(1)	F(2)
10	F(2)	**Cutoff**
Empty *Frontier*: Return Cutoff		

DLS for Graph 02 with depth = 2

Depth Limited Search $l = 2$

No.	Reached	
	Expanded	Frontier [LIFO Queue]
0		S(0)
1	S(0)	C(1) B(1) A(1)
2	A(1)	C(1) B(1) D(2) B(2)
3	B(2)	C(1) B(1) D(2) E(3) D(3) C(3)
4	C(3)	**Cutoff**
5	D(3)	**Cutoff**
6	E(3)	**Cutoff**
7	D(2)	C(1) B(1) E(3)
8	E(3)	**Cutoff**
9	B(1)	C(1) E(2) D(2) C(2)
10	C(2)	C(1) E(2) D(2) F(3)

DLS for Graph 02 with depth = 2

Depth Limited Search $l = 2$

No.	Reached	
	Expanded	Frontier [LIFO Queue]
11	F(3)	**Cutoff**
12	D(2)	C(1) E(2) E(3)
13	D(2)	C(1) E(2) E(3)
14	E(3)	**Cutoff**
15	E(2)	C(1) G(3)
16	G(3)	Solution
S(0) → B(1) → E(2) → G(3)		

DLS for Graph 02 with depth = 3

Depth Limited Search $l = 3$

No.	Reached	
	Expanded	Frontier [LIFO Queue]
0		S(0)
1	S(0)	C(1) B(1) A(1)
2	A(1)	C(1) B(1) D(2) B(2)
3	B(2)	C(1) B(1) D(2) E(3) D(3) C(3)
4	C(3)	C(1) B(1) D(2) E(3) D(3) F(4)
5	F(4)	**Cutoff**
6	D(3)	C(1) B(1) D(2) E(3) E(4)
7	E(4)	**Cutoff**
8	E(3)	C(1) B(1) D(2) G(4)
9	G(4)	Solution
S(0) → A(1) → B(2) → E(3) → G(4)		

Iterative deepening Search

```
func Interative_Deepening_Search(problem) returns Solution node or Failure
    for depth = 0 to ∞ do
        result ← Depth_Limited_Search(problem, depth)
        if result ≠ Cutoff then
            return result
```

Informed Search Strategies (Heuristic Search)

1. **Concept**

   Informed search strategies use domain-specific knowledge to find solutions more efficiently than uninformed strategies.

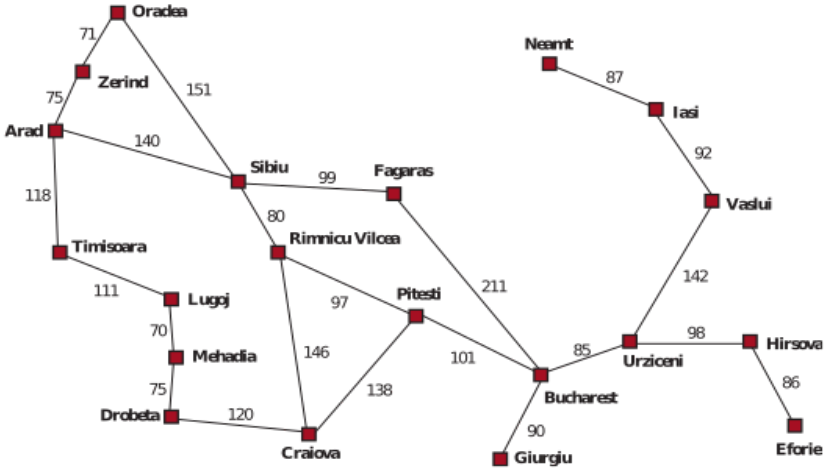
   - **Heuristic Function** $h(n)$: The most common form of "knowledge."
     - $h(n)$ estimated cost of the cheapest path from the current state at node n to a goal state.
     - If n is a goal node, then $h(n) = 0$.
   - **Goal:** Use $h(n)$ to guide the search toward the goal, reducing the number of nodes expanded.

Informed Search Strategies (Heuristic Search)

2. **Two Main Algorithms:**
   - **Greedy Best-First Search:** Only cares about the "Future" h: $f(n) = h(n)$.
   - **A* Search:** Cares about both the "Past" g and the "Future" h: $f(n) = g(n) + h(n)$.

Greedy Best-First Search

Example: Finding a route from **Arad** to **Bucharest**.



Greedy Best-First Search

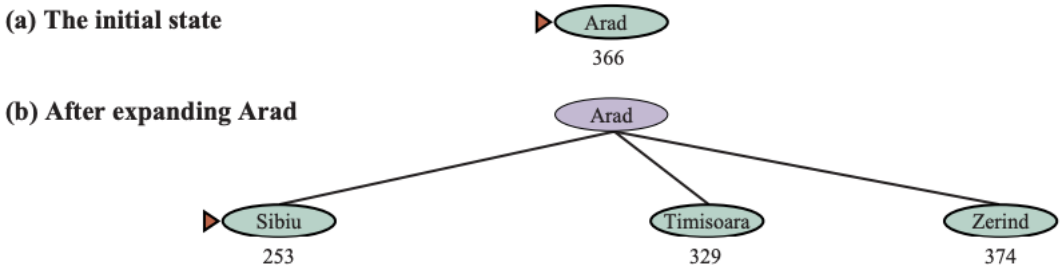
Straight-line distance heuristic: h_{SLD}.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Straight-line distance from **City** to **Bucharest**.

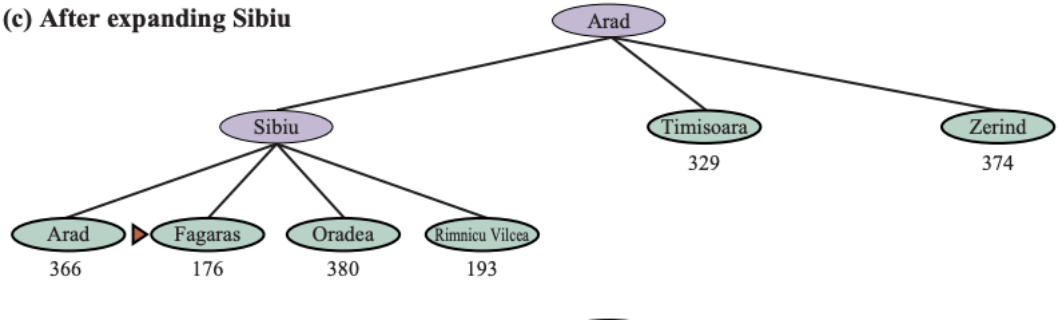
Greedy Best-First Search

$h_{SLD}(Arad) = 366$; $h_{SLD}(Sibiu) = 253$
$h_{SLD}(Timisoara) = 329$; $h_{SLD}(Zerind) = 366$



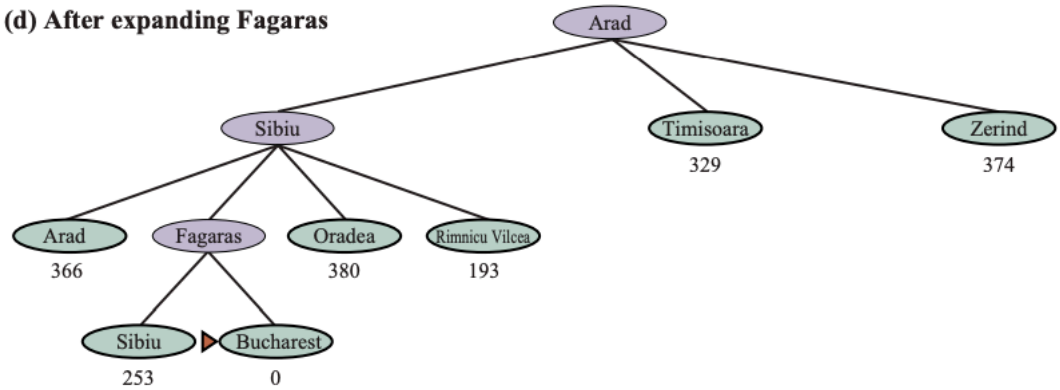
Greedy Best-First Search

$h_{SLD}(Arad) = 366$; $h_{SLD}(Fagaras) = 176$
$h_{SLD}(Oradea) = 380$; $h_{SLD}(RimnicuVilcea) = 193$



Greedy Best-First Search

$h_{SLD}(Sibiu) = 253;$ $h_{SLD}(Bucharest) = 0$



Greedy-Best-First Search

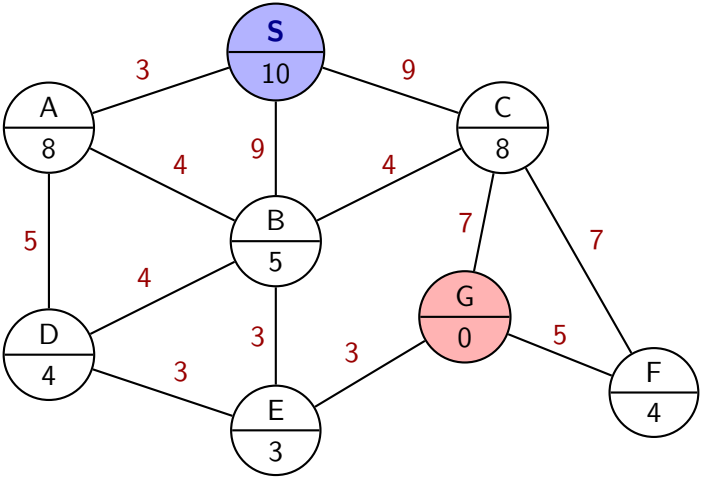
- **Greedy Best-First** search Using **Best-First-Search** algo. with $f(n) = h(n)$

```
func Greedy-Best-First-Search(problem, f) returns Solution node or Failure
    node ← Node(State = problem.Initial)
    frontier ← a priority Queue by f, node as an element
    reached ← lookup table, with key problem.Initial and value node
    while not Empty (frontier) do
        node ← Pop (frontier)
        if problem.Goal(node.State) then
            return node
        foreach child ∈ Expand (problem, node) do
            s ← child.State
            if s ∉ reached or child.h() < reached[s].h() then
                reached ← child
                add child to frontier
    return Failure
```

update function Expand

```
func Expand(problem, node) yields nodes
    s ← node.State
    foreach action ∈ problem.Action(s) do
        s' ← problem.Result(s, action)
        cost ← problem.h()(s, action, s')
        yields Node(State = s', Parent = node, Action = action, h() = cost)
```

Graph 03



Gready BFS for Graph 03

Greedy Best-First Search

No.	Reached	
	Expanded	Frontier [Priority Queue]
0		S(10)
1	**S(10)**	A(8) B(5) C(8)
2	B(5)	A(8) C(8) D(4) E(3)
3	E(3)	A(8) C(8) D(4) G(0)
4	**G(0)**	A(8) C(8) D(4)
$S \rightarrow B \rightarrow E \rightarrow G$		

A* Search

Concept: A* minimizes the total estimated cost by combining the cost to reach the node and the estimated cost to the goal.

- **Evaluation Function:** $f(n) = g(n) + h(n)$
  - $g(n)$: The cost so far to reach node n (The Past).
  - $h(n)$: The estimated cost from n to the goal (The Future).
  - $f(n)$: The estimated cost of the cheapest solution through n.
- **The Logic:** A* is identical to Uniform-cost search (UCS) except that it uses $g + h$ instead of just g.

Conditions for Optimality

For A* to find the optimal solution, the heuristic $h(n)$ must satisfy certain properties:

1. **Admissibility** (Tính chấp nhận được): An admissible heuristic never overestimates the cost to reach the goal.
   - $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost.
   - e.g. Straight-line distance is always $\leq$ actual road distance.

Conditions for Optimality

2. **Consistency** (Tính nhất quán/Đơn điệu): For every node n and every successor n' generated by action a:
   - $h(n) \leq c(n, a, n') + h(n')$
   - This ensures that $f(n)$ values are non-decreasing along any path.

A* Consistency

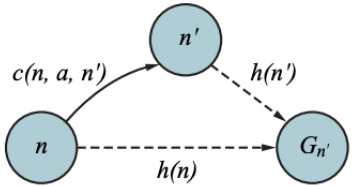
– Node n : $h(n)$

– Node n' is successor of n : $h(n')$

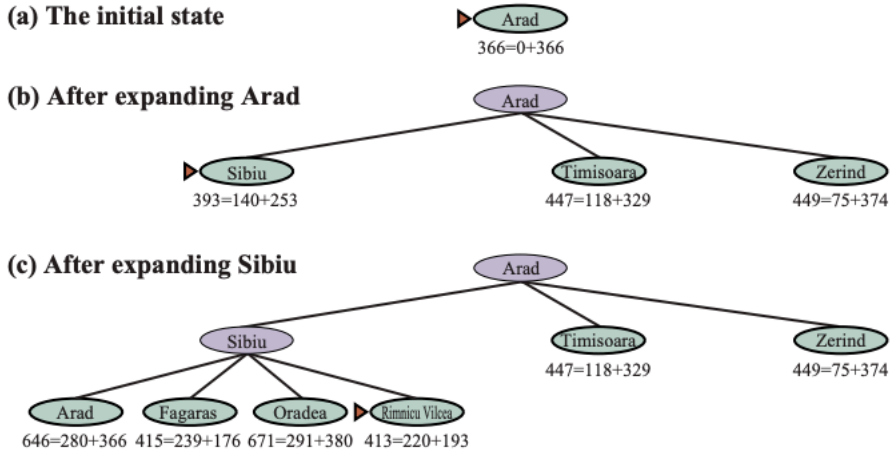
– $n \rightarrow n'$: $c(n, a, n')$

Triangle inequality:

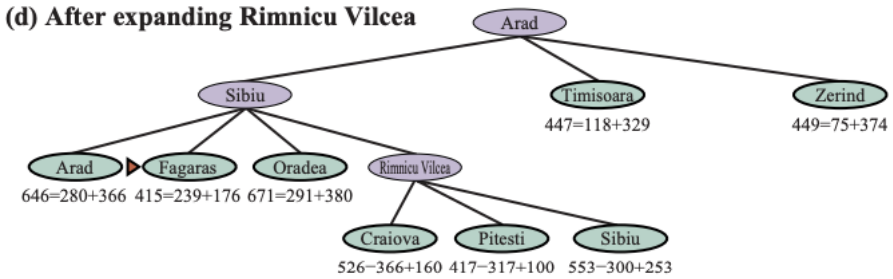
$$h(n) \leqslant c(n, a, n') + h(n')$$



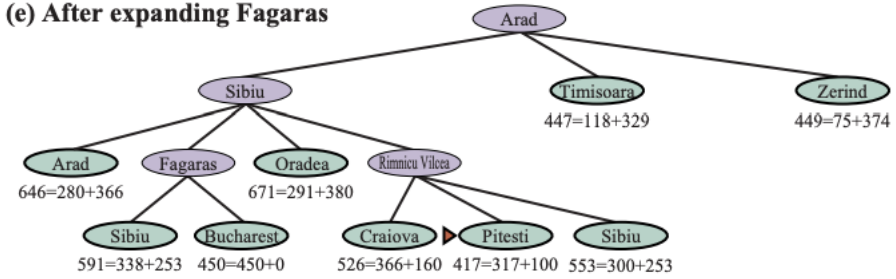
A* Search


(a) The initial state
Arad
366=0+366
(b) After expanding Arad
Arad
Sibiu
Timisoara
Zerind
393=140+253
447=118+329
449=75+374
(c) After expanding Sibiu
Arad
Sibiu
Timisoara
Zerind
447=118+329
449=75+374
Arad
Fagaras
Oradea
Rimnicu Vilcea
646=280+366
415=239+176
671=291+380
413=220+193

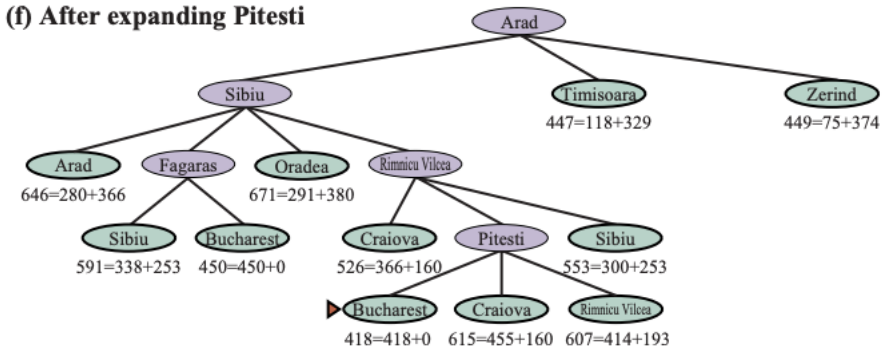
A* Search


(d) After expanding Rimnicu Vilcea
Arad
Sibiu
Timisoara
Zerind
447=118+329
449=75+374
Arad
Fagaras
Oradea
Rimnicu Vilcea
646=280+366
415=239+176
671=291+380
Craiova
Pitesti
Sibiu
526−366+160
417−317+100
553−300+253

A* Search


(e) After expanding Fagaras
Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
Oradea
671=291+380
Rimnicu Vilcea
Sibiu
591=338+253
Bucharest
450=450+0
Craiova
526=366+160
Pitesti
417=317+100
Sibiu
553=300+253

A* Search


(f) After expanding Pitesti
Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
Oradea
671=291+380
Rimnicu Vilcea
Sibiu
591=338+253
Bucharest
450=450+0
Craiova
526=366+160
Pitesti
Sibiu
553=300+253
Bucharest
418=418+0
Craiova
615=455+160
Rimnicu Vilcea
607=414+193

A* Search

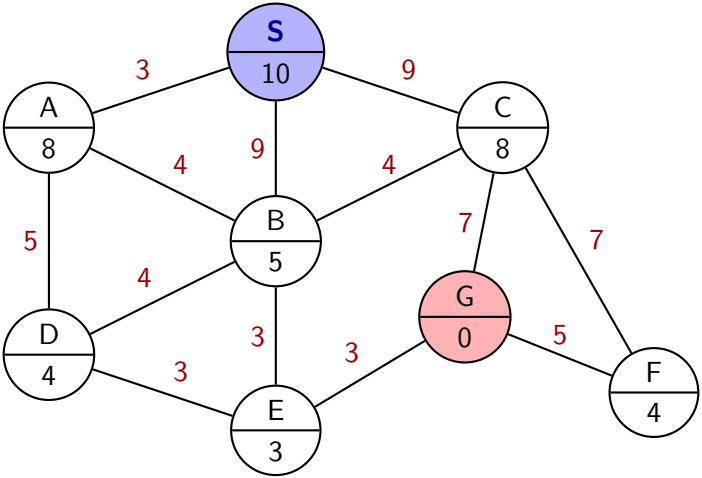
- **A* search** algo. using **Best-First-Search** algo. with $f(n) = g(n) + h(n)$

```
func A-Star-Search(problem, f) returns Solution node or Failure
    node ← Node(State = problem.Initial)
    frontier ← a priority Queue by f, node as an element
    reached ← lookup table, with key problem.Initial and value node
    while not Empty (frontier) do
        node ← Pop (frontier)
        if problem.Goal(node.State) then
            return node
        foreach child ∈ Expand (problem, node) do
            s ← child.State
            if problem.Goal(s) then
                return child
            if s ∉ reached or child.P-Cost < reached[s].P-Cost then
                reached ← child
                add child to frontier
    return Failure
```

update function Expand

func Expand(*problem*, *node*) **yields** *nodes*
  $s \leftarrow$ *node*.**State**
  foreach *action* $\in$ *problem*.**Action**(s) **do**
    $s' \leftarrow$ *problem*.**Result**(s, *action*)
    cost $\leftarrow$ *node*.**h()** + *problem*.**A-Cost**(s, *action*, s')
    yields Node(**State** $= s'$, **Parent** $=$ *node*, **Action** $=$ *action*, **P-Cost** $=$ *cost*)

Graph 03



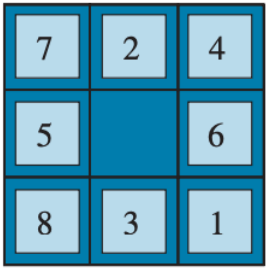
A* for Graph 03

A* Search

No.	Reached	
	Expanded	Frontier [Priority Queue]
0		S(0+10=10)
1	**S(0+10=10)**	A(3+8=11) B(9+5=14) C(9+8=17)
2	A(3+8=11)	~~B(9+5=14)~~ C(9+8=17) D(8+4=12) B(7+5=12)
3	B(7+5=12)	C(9+8=17) D(8+4=12) E(10+3=13)
4	D(8+4=12)	C(9+8=17) E(10+3=13)
5	E(10+3=13)	C(9+8=17) G(13+0=13)
6	**G(13+0=13)**	C(9+8=17)
$S \rightarrow A(3) \rightarrow B(7) \rightarrow E(10) \rightarrow G(13)$		

Heuristic Functions

e.g.



Start State



Goal State

Heuristic Functions

There are two common heuristic function:

• h_1 = the number of misplaced tiles (blank not included). For figure above, all eight tiles are out of position, so the start state has $h_1 = 8$.

• h_2 = the sum of the distances of the tiles from their goal positions. Manhattan distance.

$$h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$$

Beyond Classical Search

Concept: In classical search, we assume the environment is observable, deterministic, and the path to the goal matters. In complex environments, we face:

- **Large or Continuous State Spaces:** Where we cannot store a search tree.
- **Optimization Problems:** Where only the goal state matters, not the path taken to get there.
- **Non-deterministic environments:** Where actions can have multiple possible outcomes.

Local Search Algorithms

Definition: Local search algorithms operate using a single current node (rather than a frontier) and generally move only to neighbors of that node.

- **Pros:** Use very little memory (constant amount).
  - Can find reasonable solutions in infinite or large state spaces.
- **Key Algorithms:**
  1. **Hill-Climbing Search** (The "Greedy" approach).
  2. **Simulated Annealing** (Escaping local maxima).
  3. **Genetic Algorithms** (Stochastic hill-climbing with parallel populations).

Hill Climbing Search

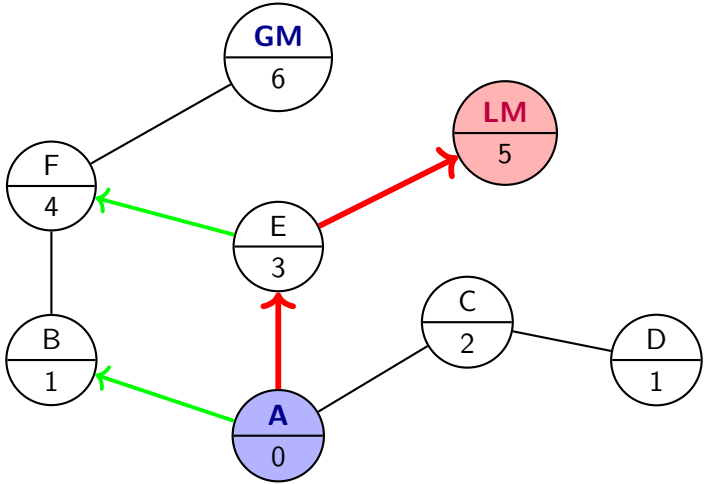
Concept: It is a loop that continually moves in the direction of increasing value (uphill). It terminates when it reaches a "peak" where no neighbor has a higher value.

- **The Problem of Local Maxima:** The algorithm can get stuck on a "shoulder" or a local peak that is not the highest point in the entire landscape (**Global Maximum**).
- **Variations:**
  - **Stochastic hill-climbing:** Chooses at random from among the uphill moves.
  - **Random-restart hill-climbing:** "If at first you don't succeed, try, try again" from a new random location.

Hill-Climbing Algorithm

```
func Hill-Climbing(problem) returns a state that is a local maximum
    current ← problem.Initial
    while true do
        neighbor ← a highest valued successor state of current
        if Value(neighbor) ⩽ Value(current) then
            return current
        current ← neighbor
```

e.g. Hill-Climbing Problems - Local Maximum



e.g. Hill-Climbing Problems - Local Maximum

Hill-Climbing h = Elevation

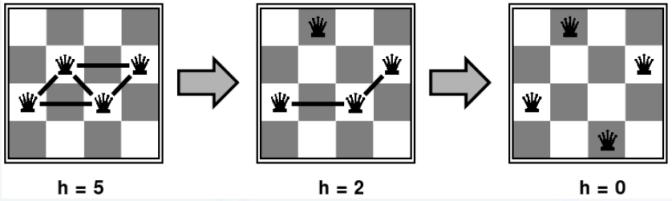
No.	Reached	
	Expanded	Frontier [Priority Queue]
0		**A(0)**
1	**A(0)**	B(1) C(2) E(3)
2	E(3)	S(4) LM(5)
3	LM(5)	
$A \rightarrow E \rightarrow LM$		

e.g. 4-Queens

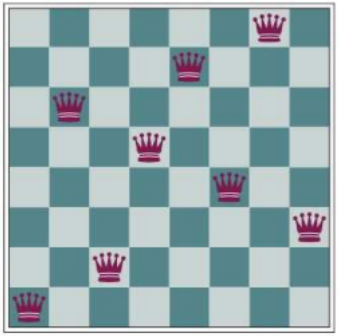


- Heuristic cost function h: the number of pairs of queens that are attacking each other.
- $h = 3$ for this board.
- Queen move within its column and update h.
- $h = 1$ is the best.
- The Hill-climbing algorithm will pick one of these.

e.g. 4-Queens



e.g. 8-Queens



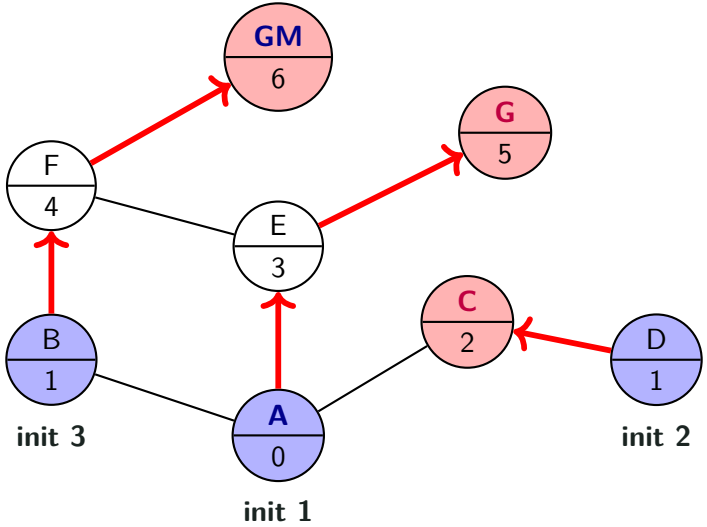
- $h = 1$ for this board.

e.g. 8-Queens



- $h = 17$
- After update, $h = 12$ is the best.

e.g. Random Restart Hill-Climbing



Simulated Annealing

Concept: Inspired by statistical mechanics (the cooling of metals), this algorithm allows "bad moves" to escape local maxima.

- **How it works:**
  - At first, the "temperature" is high, and the agent accepts many bad moves (exploration).
  - Over time, the temperature decreases, and the agent becomes more selective (exploitation).
- **Significance:** If the cooling schedule is slow enough, the algorithm is guaranteed to find the global optimum.

Simulated Annealing Algorithm

Goal: Find the global maximum/minimum by starting with high exploration and gradually moving to exploitation.

1. **Initialize:** Choose an initial state $current_state$.
   - Set an initial high temperature T.
   - Define a Cooling Schedule (how T decreases over time).

Simulated Annealing Algorithm

2. **Loop** (until T reaches a minimum threshold or time is up):
   - **Step A: Generate a Neighbor**. Select a random successor of the current state ($next_state$).
   - **Step B: Calculate Energy Change** ΔE.

     $\Delta E = Value(next) - Value(current)$ (for maximization).
   - **Step C:** Decision Making.
     - If $\Delta E > 0$ (the neighbor is better): **Always Accept** the move.
     - If $\Delta E \leq 0$ (the neighbor is worse): **Accept with Probability** $P = e^{\Delta E/T}$.
   - **Step D: Update**. If accepted, $current_state = next_state$.
3. **Cool Down:** Reduce T according to the cooling schedule (e.g., $T = T \times 0.99$).
4. **Output:** Return the current state as the best solution found.

Understanding the Acceptance Probability P

- **Why do we accept "Bad Moves"?**
  - At High T: The probability P is close to 1. The agent behaves like a Random Walk, jumping freely to explore the entire landscape and escape local traps.
  - At Low T: The probability P becomes very small. The agent behaves like Hill-Climbing, only accepting moves that improve the result.
- **The Probability Formula:**

$$P(Accept) = e^{\frac{\Delta E}{T}}$$

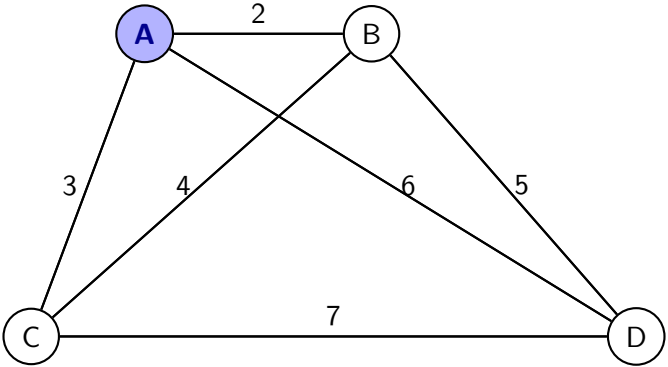
  - If T is very large $\rightarrow P \approx e^0 \approx 1$.
  - If T is very small $\rightarrow P \approx e^{-\infty} \approx 0$.

Simulated Annealing Algorithm

func `Simulated-Annealing`(*problem*, ***schedule***) **returns** a solution state

```
current ← problem.Initial
for t = 1 to ∞ do
    T ← schedule(t)
    if T = 0 then
        return current
    next ← a randomly selected successor of current
    ΔE ← Value(current) - Value(next)
    if ΔE > 0 then
        current ← next
    else
        current ← next only with probability e^(ΔE/T)
```

e.g. Traveling Salesperson Problem (TSP)



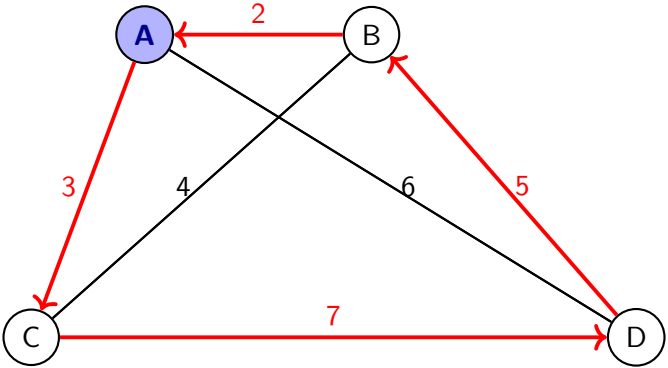
e.g. Traveling Salesperson Problem (TSP)

Loop $t = 100$; Series of random: 0.1, 0.9, 0.5, 0.8, 0.7, 0.3, 0.6, 0.5, ...
$T_0 = 100$; T rate $= 0.95$; $P = e^{\Delta/T}$
if $\Delta \leqslant 0$ and $P \leqslant$ *Random*: Current $\leftarrow$ Next

Simulated Annealing: TSP

No.	Current	Next	Δ	T	P	Random	Solution
1	ABCDA(19)	ABDCA(17)	2	100	-	0.1	Next
2	ABDCA(17)	ACBDA(18)	-1	95	0.98	0.9	Next
3	ACBDA(18)	ACDBA(17)	1	90	-	0.5	Next
4	ACDBA(17)	ADBCA(19)	-2	85	0.97	0.8	Current
5	ACDBA(17)	ADCBA(19)	-2	81	0.97	0.7	Current
6	ACDBA(17)	-	-	-	-	-	-
-	-	-	-	-	-	-	-

e.g. Traveling Salesperson Problem (TSP)



Genetic Algorithms (GA) – The Evolution of Solutions

Concept: A variant of stochastic beam search inspired by **Charles Darwin's theory of natural evolution**. Instead of modifying one state, we maintain a **population** of states and let them "evolve."

- **Key Analogy:**
  - State = Individual (Organism).
  - Value/Quality = Fitness (Khả năng thích nghi).
  - Features = DNA/Chromosomes (Nhiễm sắc thể).
- **Core Idea:** The "fittest" individuals survive and pass their traits to the next generation.

The 5 Main Phases of GA

To run a Genetic Algorithm, we follow these 5 steps in a loop:

1. **Initial Population:** Start with a set of k randomly generated individuals.
2. **Fitness Function:** A function that assigns a score to each individual (Higher is better).
3. **Selection:** Select parents based on their fitness (Better individuals have a higher chance to breed).
4. **Crossover (Recombination):** Combine parts of two parents to create offspring.
5. **Mutation:** Randomly flip small parts of the offspring's DNA to maintain genetic diversity.

Crossover and Mutation – Creating Diversity

1. **Crossover:** This is the most crucial part of GA. We pick a crossover point and swap the "DNA" of two parents.
   - e.g. Parent A: 111|11 + Parent B: 000|00 $\rightarrow$ Child: 111|00.
2. **Mutation:**

   Without mutation, the population might become too similar, getting stuck in a local optimum.
   - e.g. Child: 11100 $\rightarrow$ Mutated Child: 111**1**0.

Genetic Algorithms (GA)

```
func Genetic-Algorithm(population, fitness) returns an individual
    repeat
        weights ← Weighted-By(population, fitness)
        pupulation_2 ← empty list
        for i = 1 to Size(population) do
            parent_1, parent_2 ← Weighted-Random-Choices(population, weights, 2)
            child ← Reproduce(parent_1, parent_2)
            if small random probability then
                child ← Mutate(child)
            add child to population_2
        population ← population_2
    until some individual is fit enough, or enough time has elapsed
    return the best individual in population, according to fitness
```

Genetic Algorithms (GA) cont.

func `Preproduce`($parent_1, parent_2$) **returns** an individual
  $n \leftarrow$ Length($parent_1$)
  $c \leftarrow$ random number from 1 to n
  return Append(Substring($parent_1$, $1, c$), Substring($parent_2$, $c+1, n$))

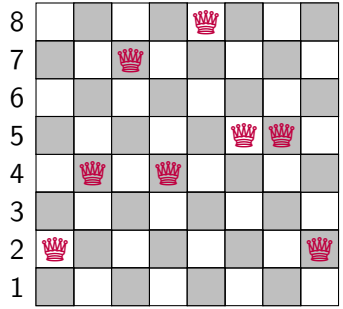
e.g. 8-Queens

How do we represent 8-Queens in a Genetic Algorithm?

- **State (Individual):** A string of 8 digits (e.g., 24748552), where each digit represents the row position of a queen in a column.
- **Fitness Function:** The number of non-attacking pairs of queens (Maximum is 28).
- **Selection:** Use a "Roulette Wheel" where the slice of the wheel is proportional to the fitness score.

Case Study: 8-Queens

- **Individual**: Each individual (**gene**) is encoded by a string (characters or numbers) e.g. 8-Queens

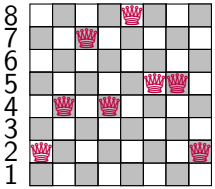


Gene = Encoded: **(2 4 7 4 8 5 5 2)**

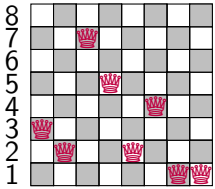
Case Study: 8-Queens

- **Population:** Subset of $\sum(individuals - Chromosomes)$

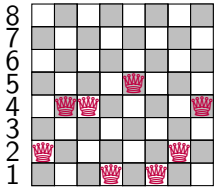
  e.g. 8-Queens



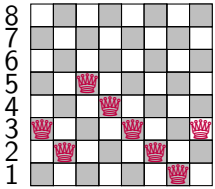
(2 4 7 4 8 5 5 2)



(3 2 7 5 2 4 1 1)



(2 4 4 1 5 1 2 4)

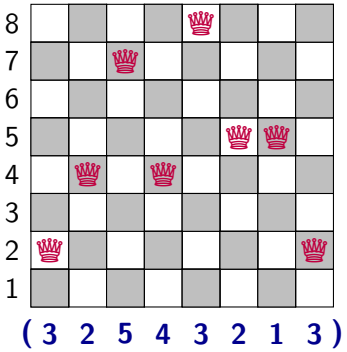


(3 2 5 4 3 2 1 3)

Case Study: 8-Queens

- **Fitness:** Score function of individual F_i

e.g. 8-Queens:
Fitness = number of non attacking pairs of queens. **Final solution:** $(8 * 7)/2 = 28$.



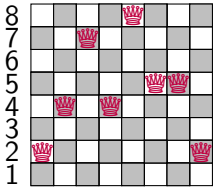
- Queen 1: 6
- Queen 2: 5
- Queen 3: 4
- Queen 4: 4
- Queen 5: 3
- Queen 6: 1
- Queen 7: 1
- Queen 8: 0

Fitness score $F_1 = 24$

Case Study: 8-Queens

- **Normalized fitness to probabilities:** $P_i = F_i / \sum_1^n F_i$

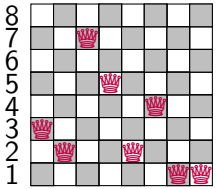
e.g. 8-Queens:



(2 4 7 4 8 5 5 2)

$F_1 = 24$

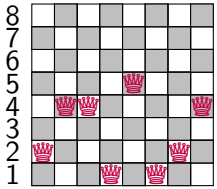
$P_1 = 31\%$



(3 2 7 5 2 4 1 1)

$F_2 = 23$

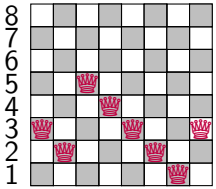
$P_2 = 29\%$



(2 4 4 1 5 1 2 4)

$F_3 = 20$

$P_3 = 26\%$

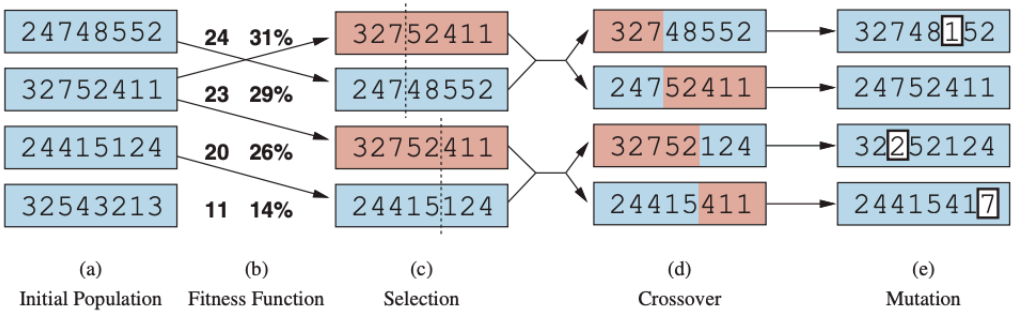


(3 2 5 4 3 2 1 3)

$F_4 = 11$

$P_4 = 14\%$

Genetic Algorithms (GA)



The 3-Airport Problem

- **Goal:** Determine the locations for 3 new airports anywhere on the map of Romania such that the **sum of squared straight-line distances** from each city on the map to its nearest airport is minimized.
- **State Space:** This is defined by the coordinates of the three airports: $(x_1, y_1), (x_2, y_2), (x_3, y_3)$. Because there are six variables, this constitutes a **six-dimensional space**. Moving through this space corresponds to changing the position of one or more airports on the map.
- **Objective Function** f: This is the function to be minimized (in this case, the sum of squared distances). If C_i is the set of cities whose nearest airport is airport i, the formula is expressed as:

$$f(x_1, y_1, x_2, y_2, x_3, y_3) = \sum_{i=1}^{3} \sum_{c \in C_i} ((x_i - x_c)^2 + (y_i - y_c)^2)$$

Geometric Representation

State-Space Landscape in Continuous Spaces

- **Mathematical Formulation**
  - A state is defined as a vector of continuous real-valued variables:

$$x = (x_1, x_2, ..., x_n) \in \mathbb{R}^n$$

  - Evaluation function (Objective function or Cost function):$f(x)$
- **Geometric Intuition**
  - In a 2D variable space (x_1, x_2), the state space forms a continuous coordinate plane.
  - The function $f(x_1, x_2)$ defines a **surface landscape** consisting of hills (local/global maxima) and valleys (local/global minima).
  - **Goal:** Find the state x that maximizes or minimizes $f(x)$.

Naive Approach - Discretization

Breaking Down Continuous into Discrete

- **The Basic Idea:**
  - Convert a continuous space into a discrete one by dividing the space using a fixed step size δ.
  - Example: Instead of exploring all real numbers $x \in [0, 1]$, evaluate a finite set $x \in \{0.0, 0.1, 0.2, ..., 1.0\}$ with $\delta = 0.1$.
- **Major Limitations**
  - If δ is too large: The algorithm might jump right over the optimal peak (missing the solution).
  - If δ is too small: The state space explodes exponentially, causing extreme performance lag.
- **Empirical Local Search:** Move a small step of $\pm\delta$ along each coordinate axis and choose the best direction.

Optimization using Derivatives

Gradient-Based Search

- **Why use Gradients?**
  - Instead of guessing random directions, we use calculus to compute the exact direction of the steepest slope.
- **The Gradient Vector ∇f**
  - The gradient is a vector containing the partial derivatives of the function with respect to each variable:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, ..., \frac{\partial f}{\partial x_n} \right)$$

  - **Geometric Meaning:** The gradient vector ∇f always points in the direction of the steepest ascent (fastest increase) of the function at the current point.

The State Update Rule

Learning Rate and Step Size

- **Maximization (Gradient Ascent - Climbing Up)**

$$x \leftarrow x + \alpha \nabla f(x)$$

- **Minimization (Gradient Descent - Moving Down)**

$$x \leftarrow x - \alpha \nabla f(x)$$

- **The Role of Step Size / Learning Rate α**
  - If α is too small: The algorithm takes tiny steps and converges very slowly.
  - If α is too large: The algorithm will overshoot, bouncing back and forth between hillsides without ever reaching the optimum.

e.g. (1-Variable Case)

Gradient Descent in Action

- **Problem:** Minimize the cost function $f(x) = x^2$ given:
  - Initial state: $x_0 = 2$
  - Step size: $\alpha = 0.2$
- **Step 0: Find the Gradient**

$$\nabla f(x) = f'(x) = 2x$$

- **Iteration 1 ($t = 1$):**
  - Gradient value at current position: $\nabla f(x_0) = 2 \times 2 = 4$
  - Update position (minimization $\rightarrow$ subtract):

$$x_1 = x_0 - \alpha \nabla f(x_0) = 2 - 0.2 \times 4 = 1.2$$

e.g. (1-Variable Case)

- **Iteration 2 ($t = 2$):**
  - New gradient value: $\nabla f(x_1) = 2 \times 1.2 = 2.4$
  - Update position:

$$x_2 = x_1 - \alpha \nabla f(x_1) = 1.2 - 0.2 \times 2.4 = 0.72$$

- **Iteration 3 ($t = 3$):**
  - New gradient value: $\nabla f(x_2) = 2 \times 0.72 = 1.44$
  - Update position:

$$x_3 = x_2 - \alpha \nabla f(x_2) = 0.72 - 0.2 \times 1.44 = 0.432$$

Observation: The value of x steadily decreases across iterations:
$2 \to 1.2 \to 0.72 \to 0.432$, converging accurately toward the global minimum at $x = 0$.

e.g. (2-Variable Case)

Gradient Descent in Action

- **Problem:** Minimize the cost function $f(x, y) = x^2 + 2y^2$ given:
  - Initial state: $(x_0, y_0) = (2, 1)$
  - Step size: $\alpha = 0.1$
- **Step 0: Compute Partial Derivatives for the Gradient Vector**

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right) = (2x, 4y)$$

- **Iteration 1 ($t = 1$):**
  - Gradient at starting point $(2, 1)$: $\nabla f(2, 1) = (2 \times 2, 4 \times 1) = (4, 4)$
  - Update coordinates:

$$x_1 = x_0 - \alpha \frac{\partial f}{\partial x} = 2 - 0.1 \times 4 = 1.6$$

$$y_1 = y_0 - \alpha \frac{\partial f}{\partial y} = 1 - 0.1 \times 4 = 0.6$$

  - $\Rightarrow$ New position: $(x_1, y_1) = (1.6, 0.6)$

e.g. (2-Variable Case)

- **Iteration 2 ($t = 2$):**
  - Gradient at new point $(1.6, 0.6)$: $\nabla f(1.6, 0.6) = (2 \times 1.6, 4 \times 0.6) = (3.2, 2.4)$
  - Update position:

$$x_2 = x_1 - \alpha \nabla f(x_1) = 1.2 - 0.2 \times 2.4 = 0.72$$

- **Iteration 3 ($t = 3$):**
  - New gradient value: $\nabla f(x_2) = 2 \times 0.72 = 1.44$
  - Update coordinates:

$$x_2 = x_1 - \alpha \frac{\partial f}{\partial x} = 1.6 - 0.1 \times 3.2 = 1.28$$

$$y_2 = y_1 - \alpha \frac{\partial f}{\partial y} = 0.6 - 0.1 \times 2.4 = 0.36$$

  - $\Rightarrow$ New position: $(x_2, y_2) = (1.28, 0.36)$

Insight: The true minimum is at $(0, 0)$. Both variables simultaneously decay toward the origin point: $(2, 1) \to (1.6, 0.6) \to (1.28, 0.36)$.

Introduction to Nondeterminism

- **Deterministic vs. Nondeterministic Environments**
  - **Deterministic:** Action A in State S always leads to a single, predictable State S'.
  - **Nondeterministic:** Action A in State S can yield multiple possible outcomes. The environment introduces uncertainty, hardware errors, or adversarial disruptions.
- **The Evolution of a "Plan"**
  - In deterministic search, a plan is a static sequence: $[A_1, A_2, A_3]$.
  - In nondeterministic search, a plan matures into a **Strategy** or a Contingency Plan structured with conditional statements (if-then-else).

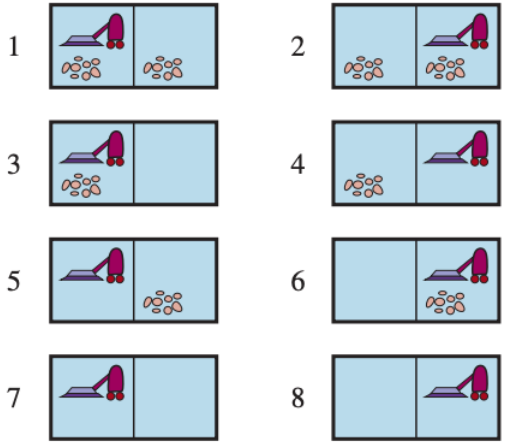
From Sequences to Conditional Plans

In a deterministic environment, a solution is a fixed sequence of actions. However, in a nondeterministic environment, the agent doesn't know for sure which state it will transition to.

- **Belief State:** Instead of tracking a single physical state, the agent maintains a **belief state**, which is a set of all physical states it believes are currently possible
- **Conditional Plan:** A solution in this context is a **conditional plan** (also known as a **contingency plan** or strategy). It specifies different actions depending on the percepts the agent receives during execution (e.g., "if the light is red, stop; if it is green, go")

The Erratic Vacuum World

The eight possible states of the vacuum world; states 7 and 8 are goal states.



The Erratic Vacuum World

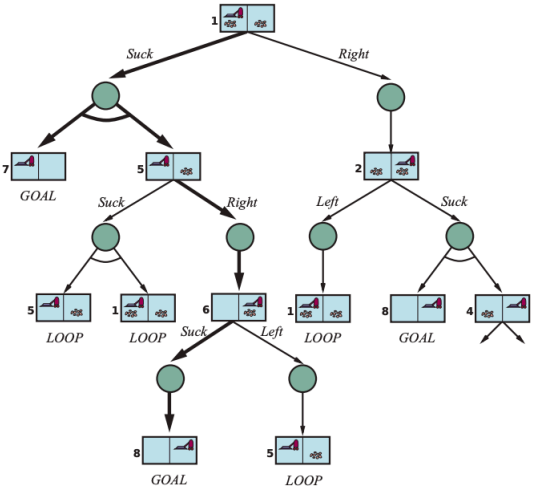
In the erratic vacuum world, the Suck action works as follows:

- When applied to a dirty square the action cleans the square and sometimes cleans up dirt in an adjacent square, too.
- When applied to a clean square the action sometimes deposits dirt on the carpet
- *Results* function that returns a set of possible outcome states.
  e.g. *Suck* action in state 1 cleans up either just the current location, or both locations:

$$Results(1, Suck) = \{5, 7\}$$

- If we start in state 1, no single sequence of actions solves the problem, but the following **conditional plan** does:
- $[Suck, \textbf{if } State = 5 \textbf{ then } [Right,\ Suck] \textit{ else } [\,]\,]$.

AND-OR Search Tree



AND-OR Search Tree

To find these contingent solutions, the search tree must account for both the agent's choices and the environment's possible responses.

- **OR Nodes:** These represent the agent's own choices between available actions in a given state
- **AND Nodes:** These represent the **environment's choice** of outcomes for a specific action. In a nondeterministic world, a single action can lead to a set of possible states, all of which the agent's plan must be prepared to handle.
- **Definition of a Solution:** A solution for an AND–OR search problem is a subtree that
  1. Has a **goal node at every leaf**.
  2. Specifies **exactly one action** at each of its OR nodes.
  3. Includes **every possible outcome branch** at each of its AND nodes.

The AND–OR–Search Algorithm

```
func AND-OR-Search(problem) returns a conditional plan, or failure
    return OR-Search(problem, problem.Initial, [ ])

func OR-Search(problem, state, path) returns a conditional plan, or failure
    if problem.Goal(state) then
        return the empty plan
    if Cycle(state, path) then
        return failure
    foreach action ∈ problem.Actions(state) do
        plan ← AND-Search(problem, Results(state, action), [state] + [path])
        if plan ≠ failure then
            return [action] + [plan]
```

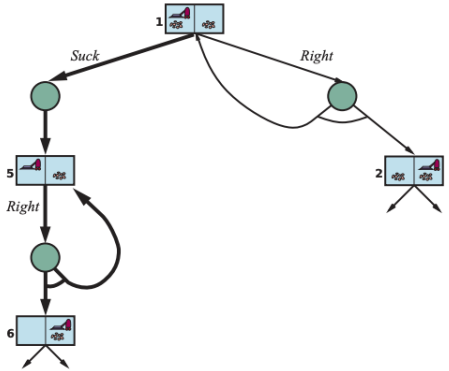
The AND–OR–Search Algorithm

func `AND-Search`(*problem*, *states*, *path*) **returns** a *conditional plan*, or *failure*

```
foreach s_i ∈ states do
    plan_i ← OR-Search(problem, s_i, path)
    if plan_i = failure then
        return failure
return [ if s_1 then plan_1 else if s_2 then plan_2 else ... if s_{n−1} then plan_{n−1} else plan_n ]
```

- An algorithm for searching **AND-OR** graphs generated by nondeterministic environments.
- A solution is a conditional plan that considers every nondeterministic outcome and makes a plan for each one.

Try, try again



Part of the search graph for a slippery vacuum world, where we have shown (some) cycles explicitly. All solutions for this problem are cyclic plans because there is no way to move reliably.

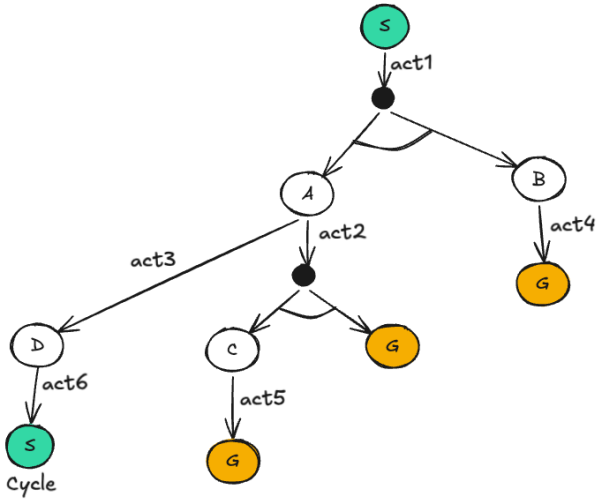
Example 1. Nondeterministic State

Consider a state space defined by the following components:

- **States:** S, A, B, C, D, G (where G is the **goal state**).
- **Initial State:** S.
- **Goal Test (Goal):** Returns true if the state is G.
- **Transition Model (Results):**
  1. At S, action act_1 leads to $\{A, B\}$.
  2. At A, action act_2 leads to $\{C, G\}$.
  3. At A, action act_3 leads to $\{D\}$.
  4. At B, action act_4 leads to $\{C, G\}$.
  5. At C, action act_5 leads to $\{G\}$.
  6. At D, action act_6 leads to $\{S\}$ (forming a **cycle**).

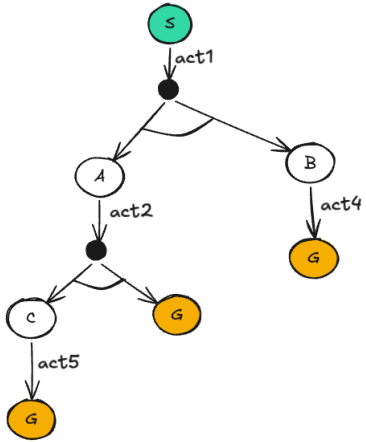
Task: Execute the **AND-OR-Search** algorithm to find a conditional plan that leads the agent from S to G.

Example 1. AND-OR Search Tree



Example 1. Sub Tree Plan

$[act_1$ *if A then* $[act_2,$ *if C then* $[act_5]]$ *else if B then* $[act_4]]$



Example 2. Nondeterministic State

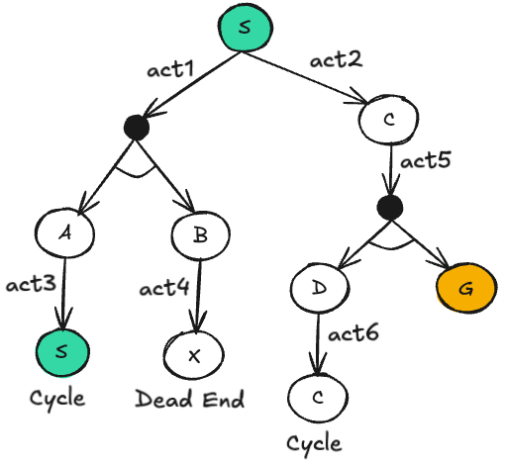
Consider a state space defined by the following components:

- **States:** S, A, B, C, D, X, G (where G is the **goal state**).
- **Initial State:** S.
- **Goal Test (Goal):** Returns true if the state is G.
- **Transition Model (Results):**
  1. At S, action act_1 leads to $\{A, B\}$.
  2. At S, action act_2 leads to $\{C\}$.
  3. At A, action act_3 leads to $\{S\}$.
  4. At B, action act_4 leads to $\{X\}$.
  5. At C, action act_5 leads to $\{D, G\}$.
  6. At D, action act_6 leads to $\{C\}$.

Task: Execute the **AND-OR-Search** algorithm to find a conditional plan that leads the agent from S to G.

Example 2. AND-OR Search Tree

There is no conditional plan that guarantees a certain journey to G from S.



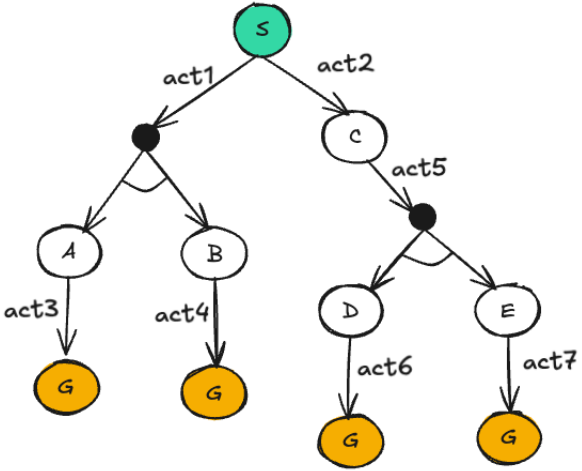
Example 3. Nondeterministic State

Consider a state space defined by the following components:

- **States:** S, A, B, C, D, E, G (where G is the **goal state**).
- **Initial State:** S.
- **Goal Test (Goal):** Returns true if the state is G.
- **Transition Model (Results):**
  1. At S, action act_1 leads to $\{A, B\}$.
  2. At S, action act_2 leads to $\{C\}$.
  3. At A, action act_3 leads to $\{G\}$.
  4. At B, action act_4 leads to $\{G\}$.
  5. At C, action act_5 leads to $\{D, E\}$.
  6. At D, action act_6 leads to $\{G\}$.
  7. At E, action act_7 leads to $\{G\}$.

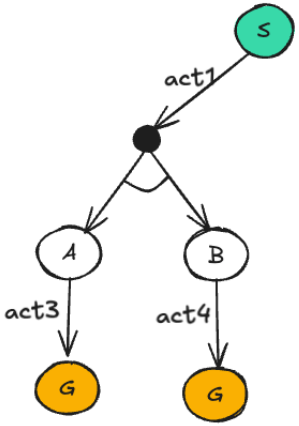
Task: Execute the **AND-OR-Search** algorithm to find a conditional plan that leads the agent from S to G.

Example 3. AND-OR Search Tree



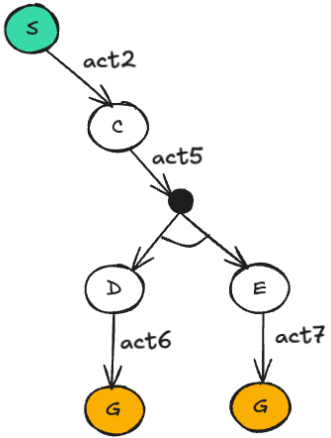
Example 3. Sub Tree Plan 1.

$[act_1$ *if A then* $[act_2,$ *if C then* $[act_5]]$ *else if B then* $[act_4]]$



Example 3. Sub Tree Plan 2.

$[act_1$ *if A then* $[act_2,$ *if C then* $[act_5]]$ *else if B then* $[act_4]]$

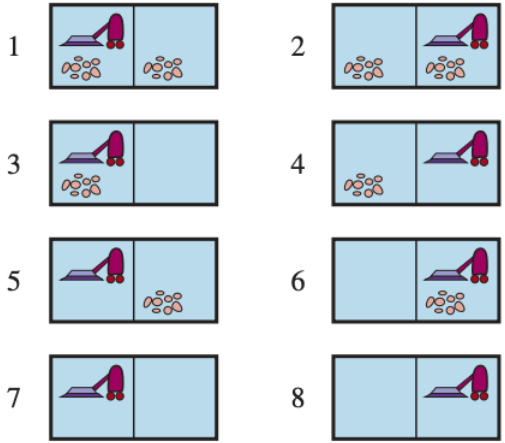


Searching with no observation

- When the agent's percepts provide no information at all, is called a sensorless problem.
- Sometimes a sensorless plan is better even when a conditional plan with sensing is available.
- e.g., doctors often prescribe a broad-spectrum antibiotic rather than using the conditional plan of doing a blood test, then waiting for the results to come back, and then prescribing a more specific antibiotic.

Sensorless version of the (deterministic) vacuum world

Assume that the agent knows the geography of its world, but not its own location or the distribution of dirt.



Sensorless version of the (deterministic) vacuum world

- Initial belief state is $\{1, 2, 3, 4, 5, 6, 7, 8\}$.
- If the agent moves *Right* it will be in one of the states $\{2, 4, 6, 8\}$.
- After $[\mathit{Right}, \mathit{Suck}]$ the agent will always end up in one of the states $\{4, 8\}$.
- Finally, after $[\mathit{Right}, \mathit{Suck}, \mathit{Left}, \mathit{Suck}]$ the agent is guaranteed to reach the goal state 7, no matter what the start state.
- The solution to a sensorless problem is a sequence of actions, not a conditional plan.
- In belief-state space, the problem is fully observable because the agent always knows its own belief state.
- Furthermore, the solution (if any) for a sensorless problem is always a sequence of actions.

The belief-state problem

- Using the existing Search algorithms if we transform the underlying physical problem into a belief-state problem, in which we search over belief states rather than physical states.
- The original problem, P, has components $Actions_P$, $Result_P$ etc., and the belief-state problem has the following components:
  - **States:** The belief-state space contains every possible subset of the physical states. If P has N states, then the belief-state problem has 2^N belief states, although many of those may be unreachable from the initial state.
  - **Initial state:** Typically the belief state consisting of all states in P, although in some cases the agent will have more knowledge than this.

The belief-state problem

- - **Actions:** Suppose the agent is in belief state $b = \{s_1, s_2\}$, but $Actions_P(s_1) \neq Actions_P(s_2)$; then it is safe to take the union of all the actions in any of the physical states in the current belief state b:
$$Actions(b) = \bigcup_{s \in b} Actions_P(s)$$

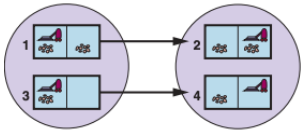
- **Transition model:**
For deterministic actions, the new belief state has one result state for each of the current possible states (although some result states may be the same):
$$b' = Result(b, a) = \{s' : s' = Result_p(s, a) \text{ and } s \in b\}$$
With nondeterminism, the new belief state consists of all the possible results of applying the action to any of the states in the current belief state:
$$\begin{aligned} b' = Result(b, a) &= \{s' : s' = Result_p(s, a) \text{ and } s \in b\} \\ &= \bigcup_{s \in b} Result_p(s, a) \end{aligned}$$

The belief-state problem

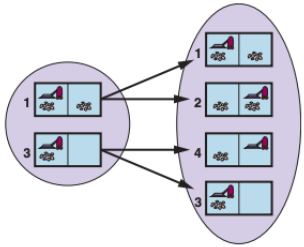
- **Goal test:** The agent possibly achieves the goal if any state s in the belief state satisfies the goal test of the underlying problem, $Goal_P(s)$.
- **Action cost:** We assume that the cost of an action is the same in all states and so can be transferred directly from the underlying physical problem.

The belief-state problem

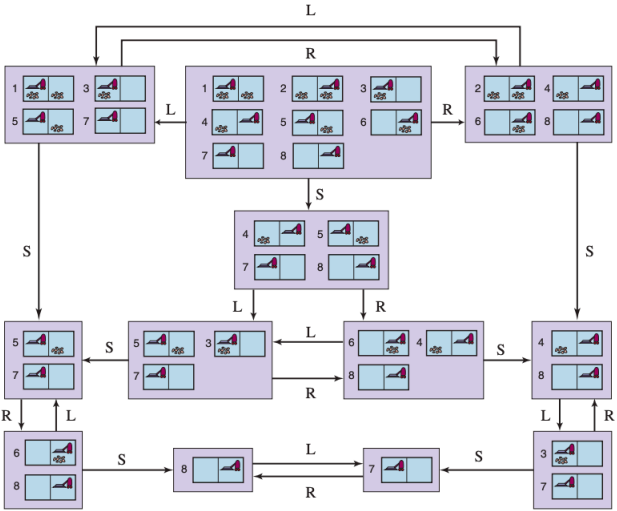
- $Result(\{1,3\}, Right) = \{2,4\}$:



- $Result(\{1,3\}, Right) = \{1,2,3,4\}$ in the slippery version:

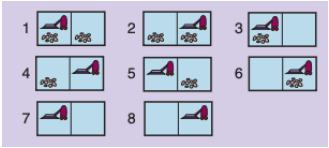


Seaching for reachable belief-state space (deterministic)



Seaching for reachable belief-state space (deterministic)

- Belief-state space: $2^8 = 256$ possible belief states.
- Initial belief-state: $\{1, 2, 3, 4, 5, 6, 7, 8\}$



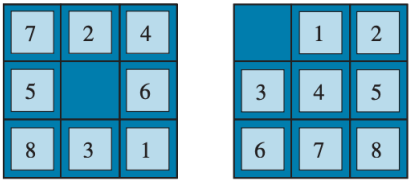
- $[\mathit{Initial}, \mathit{Right}] = \{2, 4, 6, 8\}$
- $[\mathit{Initial}, \mathit{Left}] = \{1, 3, 5, 7\}$
- ...
- There are only 12 reachable belief states to find out $\{7\}$ or $\{8\}$
- $[\mathit{Initial}, \mathit{Right}, \mathit{Suck}, \mathit{Left}, \mathit{Suck}] = \{7\}$

Key Takeaways

- **Coercion Over Observation:** A sensorless (blind) agent can still formulate a successful plan to achieve its goal without any perceptual feedback. By executing a coercive sequence of actions e.g., $[Right, Suck, Left, Suck]$.
- **Belief-State Reduction:** The size of the belief state can be reduced or guided into known states. The agent transitions from complete ignorance (top box with 8 physical states) down to singleton goal states ([7] or [8]).

Searching in partially observable environments

- Many problems can not be solved without sensing.
- For example, the sensorless 8-puzzle is impossible.
- We can solve 8-puzzles if we can see just the upper-left corner square.
- The solution involves moving each tile in turn into the observable square and keeping track of its location from then on.



Searching in partially observable environments

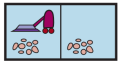
- $Percept(s)$ returns the percept received by the agent in a given state.
- If sensing is nondeterministic, then we can use a $Percepts$ function that returns a set of possible percepts.
- For fully observable problems:
  $Percept(s) = s$ for every state s,
- and for sensorless problems $Percept(s) = null$.

Local-sensing Vacuum World

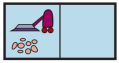
Consider a local-sensing vacuum world, in which the agent has:

- **Position sensor:** Yields the percept *L* in the left square, and *R* in the right square.
- **Dirt sensor:** Yields *Dirty* when the current square is dirty and *Clean* when it is clean.

- State 1: $Percept(1) = [L, Dirty]$

1 

- State 3 will also: $Percept(3) = [L, Dirty]$

3 

The 3-Stage Transition Model

To expand our search tree using And-Or Belief Search, the transition model for $Results(b, a)$ is broken down into three logical layers:

1. **The prediction stage:** Computes the belief state resulting from the action:
   $Result(b, a)$: Exactly as we did with sensorless problems.
   $\hat{b} = Result(b, a)$: To emphasize that this is a prediction.
   $Predict(b, a)$ as a synonym for $Result(b, a)$.
   So, we use:
   $$\hat{b} = Predict(b, a).$$
2. **The possible percepts stage:** Computes the set of percepts that could be observed in the predicted belief state (using the letter o for observation):
   $$\text{Possible-Percepts}(\hat{b}) = \{o : o = Percept(s) \text{ and } s \in \hat{b}\}$$

The 3-Stage Transition Model

3. **The update stage** For each possible percept, the belief state that would result from the percept. The updated belief state b_o, is the set of states in $\hat{b}$ that could have produced the percept:

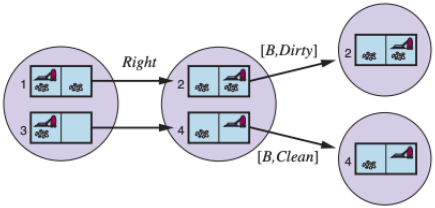
$$b_o = \mathit{Update}(\hat{b}, o) = \{s : o = \mathit{Percept}(s) \text{ and } s \in \hat{b}\}$$

Putting these three stages together, we obtain the possible belief states resulting from a given action and the subsequent possible percepts:

$$\mathit{Result}(b, a) = \{b_o : b_o = \mathit{Update}(\mathit{Predict}(b, a), o) \text{ and } o \in \text{Possible-Percepts}(\mathit{Predict}(b, a))\}$$

Example 1: Vacuum World

In the deterministic world: *A*: Left Room; *B*: Right Room.



- **Initial Belief State** $b = \{1, 3\}$: Agent is in Room A; Room A is Dirty; Room B's status is unknown.
- **Action** *Right*: Moves to Room B with 100% certainty.

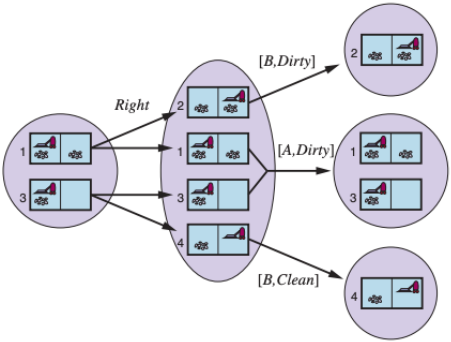
Example 1: Vacuum World

In the deterministic world:

- **Step 1: Predict:** $\hat{b} = \mathit{Predict}(b, \mathit{Right})$
  - $1 \xrightarrow{\mathit{Right}} 2$
  - $3 \xrightarrow{\mathit{Right}} 4$
  - $\Rightarrow \hat{b} = \{2, 4\}$ Uncertainty remains at 2 states.
- **Step 2: Possible-Percepts:**
  $\text{Possible-Percepts}(\hat{b}) = \{o : o = \mathit{Percept}(s) \text{ and } s \in \hat{b}\}$
  $\text{Possible-Percepts}(\{2, 4\}) = \{[B, \mathit{Dirty}], [B, \mathit{Clean}]\}$
- **Step 3: Update:** $b_o = \mathit{Update}(\hat{b}, o) = \{s : o = \mathit{Percept}(s) \text{ and } s \in \hat{b}\}$
  - If percept is $[B, \mathit{Dirty}] \Rightarrow b_o = \{2\}$ (Uncertainty eliminated / Singleton).
  - If percept is $[B, \mathit{Clean}] \Rightarrow b_o = \{4\}$ (Uncertainty eliminated / Singleton).

Example 1: Vacuum World

In the slippery world: *A*: Left cell; *B*: Right cell.



- **Initial Belief State** $b = \{1, 3\}$: Agent is in Room A; Room A is Dirty; Room B's status is unknown.
- **Action** *Right*: Agent might succeed or slip and stay in Room A.

Example 1: Vacuum World

In the slippery world: A: Left cell; B: Right cell.

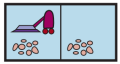
- **Step 1: Predict:** $\hat{b} = \mathit{Predict}(b, \mathit{Right})$
  - $1 \xrightarrow{\mathit{Right}} \{1, 2\}$
  - $3 \xrightarrow{\mathit{Right}} \{3, 4\}$
  - $\Rightarrow \hat{b} = \{1, 2, 3, 4\}$ Uncertainty doubles due to slippage.
- **Step 2: Possible-Percepts:**
  $\text{Possible-Percepts}(\hat{b}) = \{o : o = \mathit{Percept}(s) \text{ and } s \in \hat{b}\}$
  $\text{Possible-Percepts}(\{1, 2, 3, 4\}) = \{[A, \mathit{Dirty}], [B, \mathit{Dirty}], [B, \mathit{Clean}]\}$
- **Step 3: Update:** $b_o = \mathit{Update}(\hat{b}, o) = \{s : o = \mathit{Percept}(s) \text{ and } s \in \hat{b}\}$
  - If percept is $[B, \mathit{Dirty}] \Rightarrow b_o = \{2\}$.
  - If percept is $[B, \mathit{Clean}] \Rightarrow b_o = \{4\}$.
  - If percept is $[A, \mathit{Dirty}] \Rightarrow b_o = \{1, 3\}$
    (Slippage confirmed; agent stays in Room A).

Example 2: Vacuum World

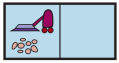
Local-sensing Vacuum World

- **Position sensor:** Yields the percept *A* in the left square, and *B* in the right square.
- **Dirt sensor:** Yields *Dirty* when the current square is dirty and *Clean* when it is clean.

- State 1: $Percept(1) = [A, Dirty]$

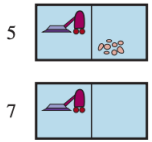
1 

- State 3 will also: $Percept(3) = [A, Dirty]$

3 

Example 2: Vacuum World

1. **Deterministic Environment:**
   - **Initial Belief State** $b = \{1, 3\}$: Agent is in Room A; Room A is Dirty; Room B's status is unknown.
   - **Action:** *Suck* (Succeeds with 100% certainty; cleans Room A).
   - **Step 1: Predict:** $\hat{b} = \mathit{Predict}(b, \mathit{Suck})$
     - $1 \xrightarrow{\mathit{Suck}} 5$
     - $3 \xrightarrow{\mathit{Right}} 7$
     - $\Rightarrow \hat{b} = \{5, 7\}$ Room A is now guaranteed to be Clean.



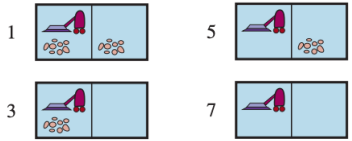
Example 2: Vacuum World

1. **Deterministic Environment:**
   - **Step 2: Possible-Percepts:**
     $\text{Possible-Percepts}(\hat{b}) = \{o : o = \textit{Percept}(s) \text{ and } s \in \hat{b}\}$
     $\text{Possible-Percepts}(\{5, 7\}) = \{[A, \textit{Clean}]\}$
   - **Step 3: Update:** $b_o = \textit{Update}(\hat{b}, o) = \{s : o = \textit{Percept}(s) \text{ and } s \in \hat{b}\}$
     - Since there is only one possible percept, no AND-branching occurs.
     - Filter $\hat{b}$ for $[A, \textit{Clean}] \Rightarrow b_o = \{5, 7\}$.
   - Takeaway: The agent knows Room A is clean, but strategic uncertainty about Room B remains completely unchanged.

Example 2: Vacuum World

2. Slippery Environment (Non-deterministic/Faulty Vacuuming)

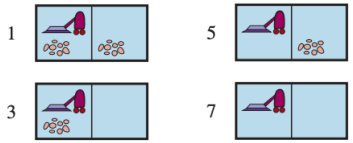
- **Initial Belief State** $b = \{1, 3\}$: Agent is in Room A; Room A is Dirty; Room B's status is unknown.
- **Action** *Suck*: Agent might succeed or slip and stay in Room A.
- **Step 1: Predict:** $\hat{b} = \mathit{Predict}(b, \mathit{Right})$
  - Success path: $\{1, 3\} \xrightarrow{\mathit{Suck}} \{5, 7\}$
  - Failure path: $\{1, 3\} \xrightarrow{\mathit{Suck}} \{1, 3\}$
  - $\Rightarrow \hat{b} = \{1, 3, 5, 7\}$ Uncertainty doubles due to potential action failure.



Example 2: Vacuum World

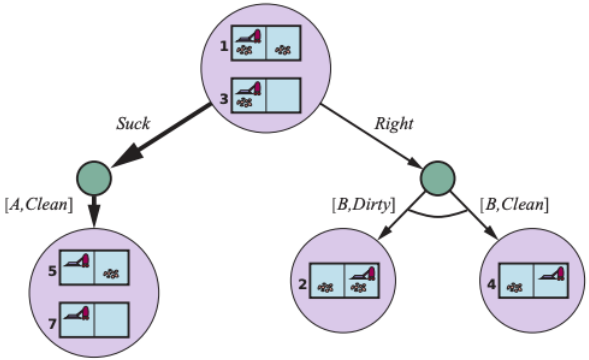
2. Slippery Environment (Non-deterministic/Faulty Vacuuming)

- **Step 2: Possible-Percepts:**
  Possible-Percepts($\hat{b}$) $= \{o : o = \mathit{Percept}(s) \text{ and } s \in \hat{b}\}$
  From $\hat{b} = \{1, 3, 5, 7\}$, the possible observations are: $\{[A, \mathit{Dirty}], [A, \mathit{Clean}]\}$.
- **Step 3: Update:** $b_o = \mathit{Update}(\hat{b}, o) = \{s : o = \mathit{Percept}(s) \text{ and } s \in \hat{b}\}$
  The search tree splits into 2 AND-branches based on the execution outcome:
  - If percept is $[A, \mathit{Dirty}] \Rightarrow b_o = \{1, 3\}$ Action failed; state reverted.
  - If percept is $[A, \mathit{Clean}] \Rightarrow b_o = \{5, 7\}$ Action succeeded; Room A is clean.



Solving partially observable problems

The first level of the AND–OR search tree for a problem in the local-sensing vacuum world; Suck is the first action in the solution.



Solving partially observable problems

- **Core Mechanism: AND-OR Search in Belief-State Space:** To solve partially observable problems, the RESULTS function (derived from the physical transition model and the PERCEPT sensor model) is used to transform the physical problem into a search within the belief-state space.
- **Applying the Algorithm:** The AND-OR-SEARCH algorithm can be applied directly to this formulation to find a solution.
- **Solution Structure:** The result is a conditional plan. Unlike plans for observable environments, this plan tests belief states rather than actual physical states, as the agent cannot know its exact physical location.

Searching the Belief-State Space with AND-OR Search

Once the 3-stage transition model RESULTS(b, a) is defined, we can treat a partially observable problem as a standard AND-OR Graph Search problem, but operating purely in the space of belief states.

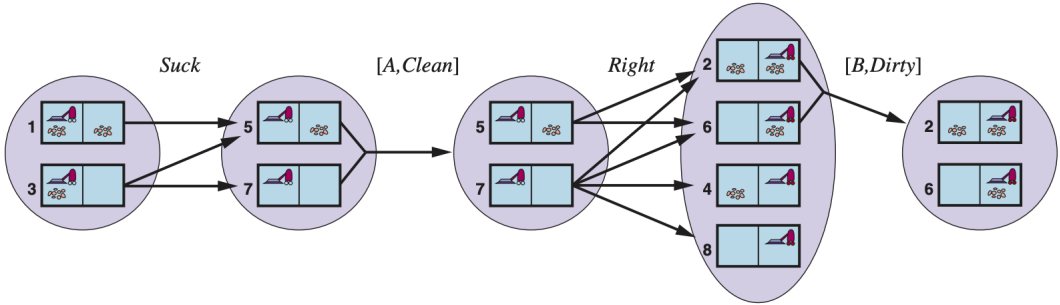
- **Key Components of the Formulation:**
  - **Initial State:** The initial belief state b_0 containing all physical states consistent with the first percept received by the agent.
  - **Actions:** ACTIONS$(b) = \bigcup_{s \in b}$ ACTIONS(s). An action is legal in a belief state only if it is legal in all underlying physical states.
  - **Transition Model:** RESULTS(b, a) returns a set of potential updated belief states $\{b_o\}$, one for each possible percept.
  - **Goal Test:** GOAL-TEST(b) returns True if and only if every physical state $s \in b$ satisfies the underlying physical goal condition.
- **The Solution Structure:** The search yields a Conditional Plan (a tree of actions using if-then-else logic based on observations), rather than a linear sequence of movements.

Agent for partially observable environments

- **Belief State Maintenance:** Instead of simply reacting to current percepts, the agent must maintain a belief state - a set of all logically possible physical states it could be in.
- **Prediction-Update Cycle:** As the agent acts and senses, it updates its belief state b to a new state b' using the formula: $b' = UPDATE(PREDICT(b, a), o)$
- **Recursive State Estimation:** This process is known as a recursive state estimator because it computes the current state from the previous one and the latest percept, without needing to re-examine the entire history of observations. This function is also called monitoring, filtering, or state estimation.
- **Speed Requirements:** The agent must be able to compute these updates at least as fast as new percepts arrive from the environment to avoid "falling behind".

Agent for partially observable environments

Two prediction–update cycles of belief-state maintenance in the kindergarten vacuum world with local sensing.



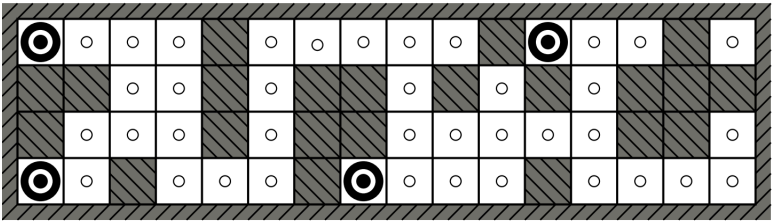
Example: Robot Localization

This example in a discrete environment with deterministic sensors and nondeterministic actions.

- Robot is placed in the maze-like environment.
- Robot is equipped with four sonar sensors:
  - bit vector: north, east, south, and west.
  - e.g., 1011 means there are obstacles to the north, south, and west, but not east.
- We assume that the sensors give perfectly correct data, and that the robot has a correct map of the environment.
- Robot's navigational system is broken, so when it executes a *Right* action, it moves randomly to one of the adjacent squares. The robot's task is to determine its current location.

Example: Robot Localization

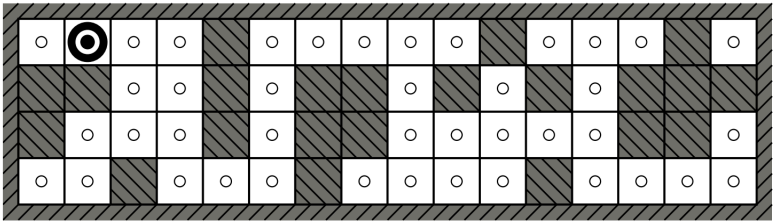
- Suppose the robot has just been switched on, and it does not know where it is.
- Initial belief state b consists of the set of all locations.
- The robot then receives the percept 1011 and does an update using the equation $b_o = \mathit{UPDATE}(1011)$, yielding the 4 locations:



Example: Robot Localization

- Next the robot executes a *Right* action, but the result is nondeterministic.
- The new belief state, $b_a = PREDICT(b_o, Right)$, contains all the locations that are one step away from the locations in b_o.
- Second percept: 1010, robot does $UPDATE(b_a, 1010)$ and finds that the belief state has collapsed down to the single location:

$$UPDATE(PREDICT(UPDATE(b, 1011), Right), 1010)$$



Online Search Agents and Unknown Environments

Unlike **offline search** algorithms, which compute a complete solution before taking a single action; **online search** agents interleave computation, sensing, and acting.

- **Operational Mechanism**: The agent performs an action, then observes the environment to receive a percept of the new state, and uses this information to determine the next action.
- **Suitability:** This approach is essential for dynamic or nondeterministic environments, as well as unknown environments where the agent lacks a complete model of the world.
- **Key Challenges:**
  - **Unknown Environments:** The agent does not know which states exist or the outcomes of its actions $RESULT(s, a)$. Actions must be used as "experiments" to learn the environment's physics.
  - **Dead Ends:** These are states from which no path leads to a goal.

Online Search Agents and Unknown Environments

- **Performance Metric:** Online agents are often evaluated by their competitive ratio, which compares the actual cost incurred by the agent to the cost of an optimal path if the map were known in advance.
- **Core Algorithms:**
  - **Online DFS Agent:** Explores the world using a depth-first strategy, requiring the ability to backtrack in the real world (undoing actions) when a path fails.
  - **LRTA*:** A "somewhat-greedy" algorithm that combines hill climbing with a memory of state cost estimates $H(s)$, which are updated through experience.

Online DFS Agent Algorithms

func ONLINE-DFS-AGENT(*problem*, s') **returns** an action

  s, a, the previous state and action, initially null
  result, a table mapping (s, a) to s', initially empty
  untried, a table mapping s to a list of untried actions
  unbacktracked, a table mapping s to a list of states never backtracked to

  if *problem.IS-GOAL*(s') **then**
    return *stop*

  if s' *is a new state (not in untried)* **then**
    untried$[s'] \leftarrow$ *problem.ACTIONS*(s')

  if s *is not null* **then**
    result$[s, a] \leftarrow s'$
    add s to the front of *unbacktracked*$[s']$

  ...

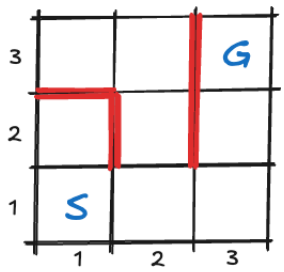
Online DFS Agent Algorithms

func ONLINE-DFS-AGENT($problem, s'$) **returns** an action

- ...
- **if** $untried[s']$ *is empty* **then**
  - **if** $unbacktracked[s']$ *is empty* **then**
    - **return** *stop*
  - $a \leftarrow$ an action b such that $result[s', b] = POP(unbacktracked[s'])$
  - $s' \leftarrow null$
- **else**
  - $a \leftarrow POP(untried[s'])$
- $s \leftarrow s'$
- **return** a

Example: Maze problem

A simple maze problem. The agent starts at S and must reach G but knows nothing of the environment.

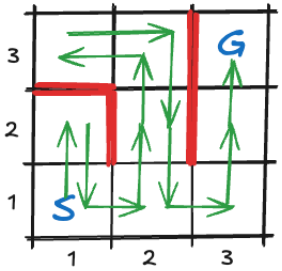


Up

Left Right

Down

Up -> Right -> down -> Left



Example: Maze problem - Online DFS Agent

Step	s	a	s'	$untried[s']$	$result[s, a]$	$unbacktracked[s']$
0	Null	Null	**(1,1)**	[(1,1)]: **[Up, Right]**		
1	(1,1)	**Up**	(1,2)	[(1,1)]: **[Right]** [(1,2)]: Null	[(1,1), **Up**]: (1,2)	[(1,2)]: (1,1)
2	(1,2)	**Down**	(1,1)	[(1,1)]: **[Right]**	[(1,2), **Down**]: (1,1)	
3	(1,1)	**Right**	(2,1)	[(2,1)]: **[Up, Right]**	[(1,1), **Right**]: (2,1)	[(2,1)]: (1,1)
4	(2,1)	**Up**	(2,2)	[(2,1)]: **[Right]** [(2,2)]: **[Up]**	[(2,1), **Up**]: (2,2)	[(2,2)]: (2,1)
5	(2,2)	**Up**	(2,3)	[(2,1)]: **[Right]** [(2,2)]: Null [(2,3)]: **[Left]**	[(2,1), **Up**]: (2,2) [(2,2), **Up**]: (2,3)	[(2,2)]: (2,1) [(2,3)]: (2,2)
6	(2,3)	**Left**	(1,3)	[(2,1)]: **[Right]** [(2,2)]: Null [(2,3)]: Null [(1,3)]: Null	[(2,1), **Up**]: (2,2) [(2,2), **Up**]: (2,3) [(2,3), **Left**]: (1,3)	[(2,2)]: (2,1) [(2,3)]: (2,2) [(1,3)]: (2,3)

Example: Maze problem - Online DFS Agent

Step	s	a	s'	$untried[s']$	$result[s, a]$	$unbacktracked[s']$
7	(1,3)	**Right**	(2,3)	[(2,1)]: [**Right**] [(2,2)]: Null [(2,3)]: Null	[(2,1), **Up**]: (2,2) [(2,2), **Up**]: (2,3)	[(2,2)]: (2,1) [(2,3)]: (2,2)
8	(2,3)	**Down**	(2,2)	[(2,1)]: [**Right**] [(2,2)]: Null	[(2,1), **Up**]: (2,2)	[(2,2)]: (2,1)
9	(2,2)	**Down**	(2,1)	[(2,1)]: [**Right**]		
10	(2,1)	**Right**	(3,1)	[(2,1)]: Null [(3,1)]: [**Up**]	[(2,1), **Right**]: (3,1)	[(3,1)]: (2,1)
11	(3,1)	**Up**	(3,2)	[(3,1)]: Null [(3,2)]: [**Up**]	[(3,1), **Up**]: (3,2)	[(3,2)]: (3,1)
12	(3,2)	**Up**	**(3,3)**	**Goal, Stop**		

Learning Real Time A^*

func $LRTA^*$-AGENT($problem, s', h$) **returns** an action

s, a, the previous state and action, initially null

$result$, a table mapping (s, a) to s', initially empty

H, a table mapping s to a cost estimate, initially empty

if $problem.IS\text{-}GOAL(s')$ **then**

  return $stop$

if s' *is a new state (not in* H *)* **then**

  $H[s'] \leftarrow h(s')$

if s *is not null* **then**

  $result[s, a] \leftarrow s'$

  $H(s) \leftarrow \min_{b \in \text{ACTIONS}(s)} \text{LRTA*-COST}(\text{problem}, s, b, \text{result}[s, b], H)$

$a \leftarrow \operatorname{argmin}_{b \in \text{ACTIONS}(s')} \text{LRTA*-COST}(\text{problem}, s', b, \text{result}[s', b], H)$

$s \leftarrow s'$

return a

Learning Real Time A^*

func $LRTA^*$-COST($problem, s, a, s', H$) **returns** a cost estimate
- **if** s' *is undefined* **then**
  - **return** $h(s)$
- **else**
  - **return** $problem$.ACTION-COST$(s, a, s') + H[s']$

Example: Maze problem - Learning Real Time A^*

To run this algorithm, we need to add a straight-line heuristic function (Manhattan distance) from each cell to the destination $G(3,3)$.

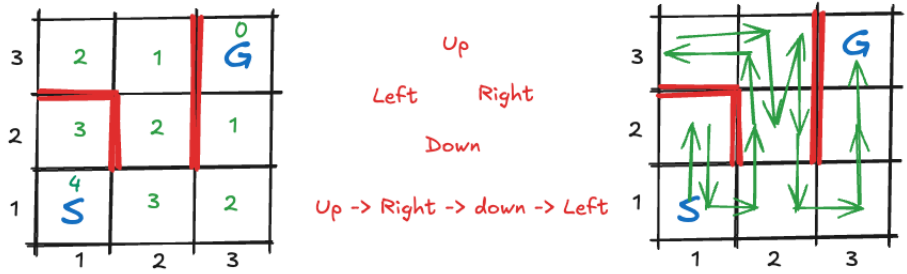
- The original heuristic function $h(s)$ (Manhattan distance to $(3,3)$):
  - Row 3: $h(1,3) = 2$; $h(2,3) = 1$; $h(3,3) = 0$
  - Row 2: $h(1,2) = 3$; $h(2,2) = 2$; $h(3,2) = 1$
  - Row 1: $h(1,1) = 4$; $h(2,1) = 3$; $h(3,1) = 2$
- Action Cost: Every valid move (**Up**, **Right**, **Down**, **Left**) costs 1.
- The order of priority when argmin is duplicated is: **Up** $\rightarrow$ **Right** $\rightarrow$ **Down** $\rightarrow$ **Left**.

Analyze the cost function LRTA*-COST(s, a, s', H)

- If cell s' has never been visited ($result[s, a]$ does not exist): Return $h(s) =$ Manhattan distance of that cell.
- If cell s' has been visited before: Return $1 + H[s']$ (where $H[s']$ is the previously updated/learned value).

Example: Maze problem - Learning Real Time A^*

A simple maze problem. The agent starts at S and must reach G but knows nothing of the environment.



Example: Maze problem - Learning Real Time A^*

Step	s	a	s'	$H(s')$	Update $H(s)$	Choose a at s' argmin[b : Cost]
0	Null	Null	**(1,1)**	$[(1,1)] = 4$		**Up(1,2):** $h(1,1) = 4$ **Right(2,1):** $h(1,1) = 4$
1	(1,1)	**Up**	(1,2)	$[(1,2)] = 3$	$[(1,1)]$ **Min:** **Up(1,2):** $1 + H(1,2) = 4$ **Right(2,1):** $h(1,1) = 4$	**Down:** $1 + H[(1,1)] = 5$
2	(1,2)	**Down**	(1,1)	$[(1,1)] = 4$	$[(1,2)]$ **Min:** **Down(1,1):** $1 + H(1,1) = 5$	**Up(1,2):** $1 + H(1,2) = 6$ **Right(2,1):** $h(1,1) = 4$
3	(1,1)	**Right**	(2,1)	$[(2,1)] = 3$	$[(1,1)]$ **Min:** **Up(1,2):** $1 + H(1,2) = 6$ **Right(2,1):** $1 + H(2,1) = 4$	**Up(2,2):** $h(2,1) = 3$ **Right(3,1):** $h(2,1) = 3$

Example: Maze problem - Learning Real Time A^*

Step	s	a	s'	$H(s')$	Update $H(s)$	Choose a at s' argmin$[b : \text{Cost}]$
4	(2,1)	**Up**	(2,2)	$[(2,2)] = 2$	$[(2,1)]$ **Min:** **Up(2,2):** $1 + H(2,2) = 3$ **Right(3,1):** $h(2,1) = 3$	**Up(2,3):** $h(2,2) = 2$ **Down(2,1):** $1 + H(2,1) = 4$
5	(2,2)	**Up**	(2,3)	$[(2,3)] = 1$	$[(2,2)]$ **Min:** **Up(2,3):** $1 + H(2,3) = 2$ **Down(1,1):** $1 + H(1,1) = 5$	**Down(2,2):** $1 + H(2,2) = 3$ **Left(1,3):** $h(2,3) = 1$
6	(2,3)	**Left**	(1,3)	$[(1,3)] = 2$	$[(2,3)]$ **Min:** **Down(2,2):** $1 + H(2,2) = 3$ **Left(1,3):** $1 + H(1,3) = 3$	**Right(2,3):** $1 + H(2,3) = 4$
7	(1,3)	**Right**	(2,3)		$[(1,3)]$ **Min:** **Right(2,3):** $1 + H(2,3) = 4$	**Down(2,2):** $1 + H(2,2) = 3$ **Left(1,3):** $1 + H(1,3) = 5$

Example: Maze problem - Learning Real Time A^*

Step	s	a	s'	$H(s')$	Update $H(s)$	Choose a at s' argmin[b : Cost]
8	(2,3)	**Down**	(2,2)		[(2,3)] **Min:** **Down(2,2):** $1 + H(2,2) = 3$ **Left(1,3):** $1 + H(1,3) = 5$	**Up(2,3):** $1 + H(2,3) = 4$ **Down(2,1):** $1 + H(2,1) = 4$
9	(2,2)	**Up**	(2,3)		[(2,2)] **Min:** **Up(2,3):** $1 + H(2,3) = 4$ **Down(2,1):** $1 + H(2,1) = 4$	**Down(2,2):** $1 + H(2,2) = 5$ **Left(1,3):** $1 + H(1,3) = 5$
10	(2,3)	**Down**	(2,2)		[(2,3)] **Min:** **Down(2,2):** $1 + H(2,2) = 5$ **Left(1,3):** $1 + H(1,3) = 5$	**Up(2,3):** $1 + H(2,3) = 6$ **Down(2,1):** $1 + H(2,1) = 4$
11	(2,2)	**Down**	(2,1)		[(2,2)] **Min:** **Up(2,3):** $1 + H(2,3) = 6$ **Down(2,1):** $1 + H(2,1) = 4$	**Up(2,2):** $1 + H(2,2) = 5$ **Right(3,1):** $h(2,1) = 3$

Example: Maze problem - Learning Real Time A^*

Step	s	a	s'	$H(s')$	Update $H(s)$	Choose a at s' argmin[b : Cost]
12	(2,1)	**Right**	(3,1)	[(3,1)]=2	[(2,1)] **Min:** **Up(2,2):** $1 + H(2,2) = 5$ **Right(3,1):** $1 + H(3,1) = 3$ **Left(1,1):** $1 + H(1,1) = 5$	**Up(3,2):** $h(3,1) = 2$ **Left(2,1):** $1 + H(2,1) = 4$
13	(3,1)	**Up**	(3,2)	[(3,2)]=1	[(3,1)] **Min:** **Up(3,2):** $1 + H(3,2) = 2$ **Left(2,1):** $1 + H(2,1) = 4$	**Up(3,3):** $h(3,2) = 1$ **Down(3,1):** $1 + H(3,1) = 3$
14	(3,2)	**Up**	**(3,3)**	**Goal, Stop**		

Constraint Satisfaction Problems (CSP)

Concept: A search problem where the state is defined by a set of variables, and the goal is to find a value for each variable that satisfies all constraints.

The Three Components X, D, C

- Variables $X = \{X_1, X_2, \ldots, X_n\}$: The things we need to assign values to.
- Domains $D = \{D_1, D_2, \ldots, D_n\}$: The set of allowable values for each variable. $D_i = \{v_1, v_2, ..., v_k\}$
- Constraints C: The rules that restrict the combinations of values.

Constraint Satisfaction Problems (CSP)

Goal: To find a complete (all variables assigned) and consistent (no constraints violated) assignment.

e.g.

$X = \{X_1, X_2\}$. $\quad D_1 = D_2 = \{1, 2, 3\}$, $\quad C = \{X_1 > X_2\}$

Solusion: $\langle (X_1, X_2), \{(3, 1), (3, 2), (2, 1)\} \rangle$

Case study: Map Coloring



Australia map coloring problem

Case study: Map coloring

$$X = \Big\{ WA, NT, Q, NSW, V, SA, T \Big\}. \qquad D_i = \Big\{ red, green, blue \Big\}.$$

$$C = \Big\{ SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V,$$

$$WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V \Big\}.$$

$$SA \neq WA := \Big\langle (SA, WA), SA \neq WA \Big\rangle.$$

$SA \neq WA$ can be: $\Big\{ (red, green), (red, blue), (green, red),$

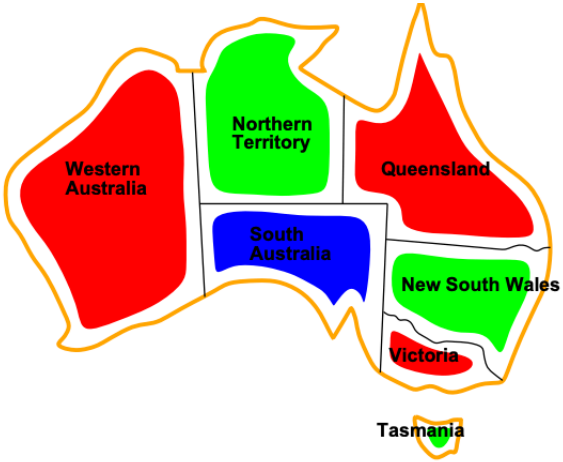
$$(green, blue), (blue, red), (blue, green) \Big\}.$$

one solutions: $\Big\{ WA = red, NT = green, Q = red, NSW = green, V = red,$

$$SA = blue, T = green \Big\}.$$

Case study: Map coloring

One solution for Australia map coloring problem:



Variations on the CSP

- **Variable**
  1. **Discrete**
     - finite domains, size $d \Rightarrow O(d^n)$ complete assignments
     - infinite domains (integers, strings, etc.)

       e.g., job scheduling, variables are start/end days for each job
  2. **Continuous domains**, e.g., start/end times for Hubble Telescope observations

Variations on the CSP

- **Constraint**
  1. **Unary constraint** e.g., $\langle (SA), SA \neq green \rangle$
  2. **Binary constraint** e.g., $\left\langle (SA, WA), SA \neq WA \right\rangle$
  3. **Global constraint** involving an arbitrary number of variables.

     Alldiff: all of the variables involved in the constraint must have different values.

     e.g., Sudoku problem, cryptarithmetic puzzles.

Real-World CSP

- Assignment problems: e.g., who teaches what class
- Timetabling problems: e.g., which class is offered when and where?
- Hardware configuration
- Transportation scheduling
- Factory scheduling
- Floorplanning
- Fault diagnosis
- ... lots more!
- Many real-world problems involve real-valued variables ...

Constraint Graphs

Definition: A visual representation of a CSP.

- **Nodes:** Represent the variables.
- **Edges:** Represent the constraints between variables.

Why use a Graph?

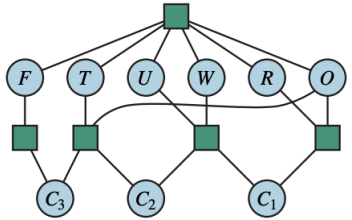
- It helps us visualize the complexity of the problem.
- We can decompose the problem into smaller, independent sub-problems (e.g., Tasmania is an independent node in the Australia map).

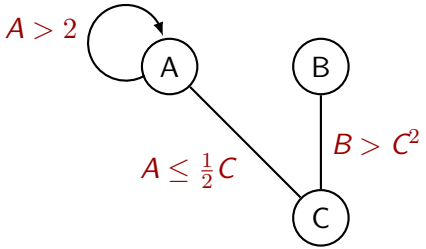
Constraint Graphs

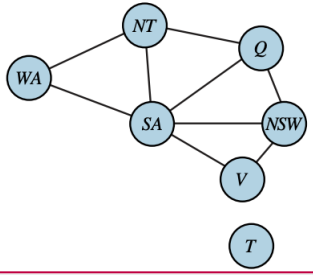
Variables $\rightarrow$ **Vertices**

Constraints $\rightarrow$ **Edges**

- **Unary**: Self-edges
- **Binary**: regular edges
- **n-ary**: hyperedges (hypergraphs)







e.g. CSP

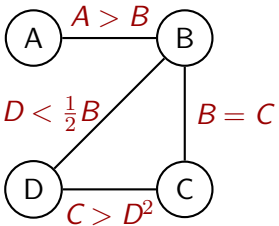
CSP

$$X = \{A, B, C, D\}$$

$$D_A = \{1, 2, 4\} \; ; \; D_B = \{1, 3, 5\} \; ; \; D_C = \{3, 5\} \; ; \; D_D = \{0, 1, 2\}$$

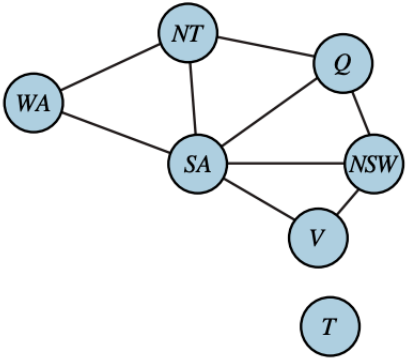
$$C = \{A > B \; ; B = C \; ; D < \tfrac{1}{2}B \; ; C > D^2\}$$

Constraint Graphs



e.g. Australia Map





e.g. Cryptarithmetic Puzzles

$X = \{F, T, U, W, R, O\}$

$D = \{0, 1, 2, ..., 9\}$

Global constraint *Alldiff* $\{F, T, U, W, R, O\}$

n-ary constraints

$O + O = R + 10.C_1$

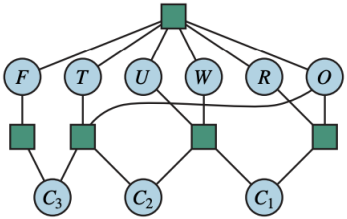
$C_1 + W + W = U + 10.C_2$

$C_2 + T + T = O + 10.C_3$

$C_3 = F$

C_1, C_2, C_3: auxiliary variables.

$$\begin{array}{cccc} & T & W & O \\ + & T & W & O \\ \hline F & O & U & R \end{array}$$



Hypergraph

Ordinary nodes (circles)
Hypernodes (squares)

Constraint Propagation: Inference in CSP

Concept: In regular search, an agent can only search. In CSPs, an agent can infer (deduce) new information.

1. **What is Constraint Propagation?**
   - Using constraints to reduce the number of legal values for a variable.
   - This, in turn, can reduce the legal values for another variable, and so on.
2. **The Goal:** To achieve Local Consistency before or during the search process.

Introduction to Local Consistency

Concept: Instead of searching through the entire state space, we can use Inference to enforce consistency in small parts of the constraint graph.

- **Definition:** Local consistency is a property where a subset of variables satisfies specific constraints independently of the rest of the problem.
- **The Core Intuition:** If a value v for variable X cannot satisfy a constraint with its neighbors, then v can never be part of a global solution.

Introduction to Local Consistency

- **Why enforce Local Consistency?**
  1. **Pruning:** It eliminates inconsistent values from domains early.
  2. **Early Detection:** It can detect if a problem has no solution before searching.
  3. **Simplification:** It transforms a complex problem into a simpler one by reducing domain sizes.

Types of Local Consistency

1. **Node Consistency:**

   A single variable (node) is consistent if all values in its domain satisfy the variable's unary constraints.

2. **Arc Consistency (The most popular):**

   A variable X_i is arc-consistent with respect to another variable X_j if for every value in D_i, there is at least one value in D_j that satisfies the binary constraint between them.

Types of Local Consistency

3. **Path Consistency:**

   A pair of variables $\{X_i, X_j\}$ is path-consistent with respect to a third variable X_m if, for every assignment $\{X_i = a, X_j = b\}$ that satisfies the constraint C_{ij}, there is an assignment for X_m that satisfies both C_{im} and C_{mj}.

Types of Local Consistency

4. k-**Consistency:**

   A CSP is k-Consistent if, for any set of $k-1$ variables and for any consistent assignment to them, a consistent value can always be assigned to any k-th variable.

   - **1-Consistency** is **Node Consistency**.
   - **2-Consistency** is **Arc Consistency**.
   - **3-Consistency** is **Path Consistency**.

Node Consistency

- A single variable (corresponding to a node in the CSP graph) is node-consistent if all the values in the variable's domain satisfy the variable's unary constraints.
- A graph is node-consistent if every variable in the graph is node-consistent.
- e.g. if South Australians dislike green $\Rightarrow$ SA reduce domain $\{red, blue\}$

Arc Consistency

- In binary constraint C, X, Y : variables. (X, Y) is Arc concistency if:
  $\forall x \in Dom(X), \exists y \in Dom(Y) : (x, y) \in C$.
- A graph is arc consistent if every variable is arc consistent with every other variable.
- e.g.
  Constraint $Y = X^2$,
  $Dom(X) = \{0, 1, 2, 3\}$,
  $Dom(Y) = \{0, 1, 4, 9\}$,
  CSP is arc-consistent.

AC-3 Algorithm

func AC-3(*csp*) **returns** *false* if an inconsistency is found and true otherwise

- *queue* ← a queue of arcs, initially all the arcs in *csp*
- **while** *queue is not empty* **do**
  - (X_i, X_j) ← Pop(*queue*)
  - **if** *Revise(csp,* X_i, X_j*)* **then**
    - **if** *size of* $D_i = 0$ **then**
      - **return** *false*
    - **foreach** X_k *in* X_i*.Neighbors -* $\{X_j\}$ **do**
      - add (X_k, X_i) to *queue*
- **return** *true*

AC-3 Algorithm

func `Revise(`*csp*, X_i, X_j`)` **returns** true iff we revise the domain of X_i

- *revised* $\leftarrow$ *false*
- **foreach** x *in* D_i **do**
  - **if** *no value* y *in* D_j *allows* (x, y) *to satisfy the constraint between* X_i *and* X_j **then**
    - delete x from D_i
    - *revised* $\leftarrow$ *true*
- **return** *revised*

e.g. AC-3

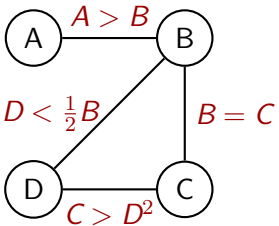
CSP : $X = \{A, B, C, D\}$

$D_A = \{1, 2, 4\}$, $D_B = \{1, 3, 5\}$, $D_C = \{3, 5\}$, $D_D = \{0, 1, 2\}$

$C = \{A > B\ ; B = C\ ; D < \frac{1}{2}B\ ; C > D^2\}$

AC-3

1. **Constraint Graph**



e.g. AC-3

2. **AC-3**

$\mathcal{D}_X$	(X_i, X_j)	$Queue\ (FIFO)$
$A\{1,2,4\}$, $B\{1,3,5\}$ $C\{3,5\}$, $D\{0,1,2\}$		$AB(A > B)$, $BA(B < A)$, $BC(B = C)$, $CB(C = B)$ $CD(C > D^2)$, $DC(D^2 < C)$, $BD(\frac{1}{2}B > D)$, $DB(D < \frac{1}{2}B)$
$A\{2,4\}$, $B\{1,3,5\}$ $C\{3,5\}$, $D\{0,1,2\}$	$AB(A > B)$	$BA(B < A)$, $BC(B = C)$, $CB(C = B)$, $CD(C > D^2)$ $DC(D^2 < C)$, $BD(\frac{1}{2}B > D)$, $DB(D < \frac{1}{2}B)$
$A\{2,4\}$, $B\{1,3\}$ $C\{3,5\}$, $D\{0,1,2\}$	$BA(B < A)$	$BC(B = C)$, $CB(C = B)$, $CD(C > D^2)$, $DC(D^2 < C)$ $BD(\frac{1}{2}B > D)$, $DB(D < \frac{1}{2}B)$, $CB(C = B)$, $DB(D < \frac{1}{2}B)$
$A\{2,4\}$, $B\{3\}$ $C\{3,5\}$, $D\{0,1,2\}$	$BC(B = C)$	$CB(C = B)$, $CD(C > D^2)$, $DC(D^2 < C)$, $BD(\frac{1}{2}B > D)$ $DB(D < \frac{1}{2}B)$, $CB(C = B)$, $DB(D < \frac{1}{2}B)$, $AB(A > B)$ $DB(D < \frac{1}{2}B)$

e.g. AC-3

$\mathcal{D}_X$	(X_i, X_j)	$Queue\ (FIFO)$
$A\{2,4\}$, $B\{3\}$ $C\{3\}$, $D\{0,1,2\}$	$CB(C = B)$	$CD(C > D^2)$, $DC(D^2 < C)$, $BD(\frac{1}{2}B > D)$, $DB(D < \frac{1}{2}B)$ $CB(C = B)$, $DB(D < \frac{1}{2}B)$,$AB(A > B)$, $DB(D < \frac{1}{2}B)$, $DC(D^2 < C)$
$A\{2,4\}$, $B\{3\}$ $C\{3\}$, $D\{0,1,2\}$	$CD(C > D^2)$	$DC(D^2 < C)$, $BD(\frac{1}{2}B > D)$, $DB(D < \frac{1}{2}B)$, $CB(C = B)$ $DB(D < \frac{1}{2}B)$,$AB(A > B)$, $DB(D < \frac{1}{2}B)$,$DC(D^2 < C)$
$A\{2,4\}$, $B\{3\}$ $C\{3\}$, $D\{0,1\}$	$DC(D^2 < C)$	$BD(\frac{1}{2}B > D)$, $DB(D < \frac{1}{2}B)$, $CB(C = B)$, $DB(D < \frac{1}{2}B)$ $AB(A > B)$, $DB(D < \frac{1}{2}B)$, $DC(D^2 < C)$, $BD(\frac{1}{2}B > D)$
$A\{2,4\}$, $B\{3\}$ $C\{3\}$, $D\{0,1\}$	$BD(\frac{1}{2}B > D)$	$DB(D < \frac{1}{2}B)$, $CB(C = B)$, $DB(D < \frac{1}{2}B)$, $AB(A > B)$ $DB(D < \frac{1}{2}B)$, $DC(D^2 < C)$, $BD(\frac{1}{2}B > D)$
$A\{2,4\}$, $B\{3\}$ $C\{3\}$, $D\{0,1\}$	$DB(D < \frac{1}{2}B)$	$CB(C = B)$, $DB(D < \frac{1}{2}B)$, $AB(A > B)$ $DB(D < \frac{1}{2}B)$, $DC(D^2 < C)$, $BD(\frac{1}{2}B > D)$
$A\{2,4\}$, $B\{3\}$ $C\{3\}$, $D\{0,1\}$	$CB(C = B)$	$DB(D < \frac{1}{2}B)$, $AB(A > B)$, $DB(D < \frac{1}{2}B)$ $DC(D^2 < C)$, $BD(\frac{1}{2}B > D)$

e.g. AC-3

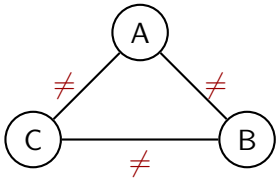
$\mathcal{D}_X$	(X_i, X_j)	$Queue\ (FIFO)$
$A\{2,4\}$, $B\{3\}$ $C\{3\}$, $D\{0,1\}$	$DB(D < \frac{1}{2}B)$	$AB(A > B)$, $DB(D < \frac{1}{2}B)$, $DC(D^2 < C)$, $BD(\frac{1}{2}B > D)$
$A\{4\}$, $B\{3\}$ $C\{3\}$, $D\{0,1\}$	$AB(A > B)$	$DB(D < \frac{1}{2}B)$, $DC(D^2 < C)$, $BD(\frac{1}{2}B > D)$
$A\{4\}$, $B\{3\}$ $C\{3\}$, $D\{0,1\}$	$DB(D < \frac{1}{2}B)$	$DC(D^2 < C)$, $BD(\frac{1}{2}B > D)$
$A\{4\}$, $B\{3\}$ $C\{3\}$, $D\{0,1\}$	$DC(D^2 < C)$	$BD(\frac{1}{2}B > D)$
$A\{4\}$, $B\{3\}$ $C\{3\}$, $D\{0,1\}$	$BD(\frac{1}{2}B > D)$	$Empty \Rightarrow Consistency$
$\mathcal{D}_A = \{4\}$, $\mathcal{D}_B = \{3\}$, $\mathcal{D}_C = \{3\}$, $\mathcal{D}_D = \{0,1\}$ $\mathcal{C} = \{A > B\ ; B = C\ ; D < \frac{1}{2}B\ ; C > D^2\}$		

Path Consistency

CSP

$X = \{A, B, C\}$,
$D_{A,B,C} = \{1, 2\}$,
$C = \{A \neq B, B \neq C, C \neq A\}$

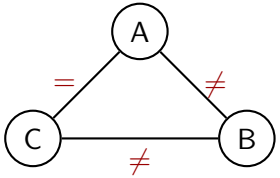
Arc-consistency but **not Path-consistency**



CSP

$X = \{A, B, C\}$,
$D = \{1, 2\}$,
$C = \{A \neq B, B \neq C, C = A\}$

Path-Consistency



Global Constraints

Concept: A global constraint is a constraint that involves an arbitrary number of variables. It is not necessarily "global" to the whole problem, but it captures a pattern that occurs frequently.

1. **The Alldiff Constraint**
   - **Definition:** All variables in the set must have distinct values.
   - e.g. **Sudoku:** Each row, column, and 3×3 block must satisfy an Alldiff constraint.
   - **Inference Advantage:** If we have 3 variables $\{X_1, X_2, X_3\}$ with domains $\{red, blue\}$, Alldiff can immediately detect a failure.
     - Simple Arc Consistency (binary) might take longer to realize that 3 variables cannot share only 2 colors.

2. **The AtMost Constraint (Resource Constraint)**

   - e.g. In a scheduling problem, "At most 10 hours of work assigned to a staff member."
   - **Inference:** Helps prune domains based on numerical bounds (Bound Consistency).

Sudoku

- Sudoku board: 81 squares, some of which are initially filled with digits from 1 to 9.
- The puzzle is to fill in all the remaining squares such that no digit appears twice in any row, column, or 3×3 box.
- A row, column, or box is called a unit.

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

(a)

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

(b)

Standard search formulation

States are defined by the values assigned so far

- **initial state**: the empty assignment, {}
- **Successor function**: assign a value to an unassigned variable that does not conflict with current assignment.

  $\Rightarrow$ fail if no legal assignments (not fixable!)

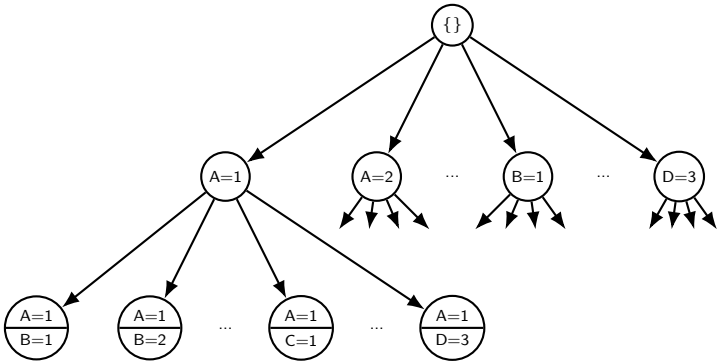
Standard search formulation

- **Goal test**: the current assignment is complete
  1. This is the same for all CSPs
  2. Every solution appears at depth n with n variables

     $\Rightarrow$ use **depth-first search**

e.g. DFS

$\mathcal{X} = \{A, B, C, D\}$; $\mathcal{D} = \{1, 2, 3\}$



Depth 1: 4 variable $\times$ 3 = 12 states.

Depth 2: 3 variable $\times$ 3 = 9 states.

Depth 3: 2 variable $\times$ 3 = 6 states.

Depth 4: 1 variable $\times$ 3 = 3 states (leaf level).

Improving Efficiency with Backtracking Search

Concept: After applying constraint propagation, if the problem isn't solved yet, we use **Backtracking Search** (a specialized DFS for CSPs).

Heuristics to Speed Up Backtracking:

1. **Variable Ordering (Which to assign next?):**
   - **Minimum Remaining Values (MRV):** Choose the variable with the fewest "legal" values left (also called the "Fail-fast" strategy).

Improving Efficiency with Backtracking Search

2. **Value Ordering (In what order to try values?):**
   - **Least Constraining Value (LCV):** Choose the value that leaves the most options open for neighboring variables.
3. **Interleaving Search and Inference:**
   - **Forward Checking:** Whenever a variable X is assigned, look ahead at its neighbors and delete inconsistent values.

Backtracking Search

func BACKTRACKING-SEARCH(*csp*) **returns** a solution or *failure*
    return BACKTRACK(*csp*, {})

Backtracking Search

```
func BACKTRACK(csp, assignment) returns a solution or failure
    if assignment is complete then
        return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)
    foreach value ∈ ORDER-DOMAIN-VALUES(csp, var , assignment) do
        if value is consistent with assignment then
            add { var = value } to assignment
            inferences ← INFERENCE(csp, var , assignment)
            if inferences ≠ failure then
                add inferences to csp
                result ← BACKTRACK(csp, assignment)
                if result ≠ failure then
                    return result
                remove inferences from csp
            remove { var = value } from assignment
    return failure
```

Backtracking Search

- $assignment = \{variable = value, ...\}$
- $var \leftarrow$ **SELECT-UNASSIGNED-VARIABLE**$(csp, assignment)$

  minimum-remaining-values (MRV) heuristic: var was choosed so that remain with fewest "legal" values. It base on **degree heuristic**, so we choose with **highest degree**.

- $value \in$ **ORDER-DOMAIN-VALUES**$(csp, var, assignment)$

  least-constraining-value heuristic: $value$ was choosed so that it make highest choices for the neighboring variables in the constraint

- **INFERENCE**$(csp, var, assignment)$

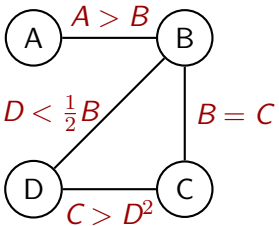
  forward checking: After $X_i \leftarrow value$, the INFERENCE calls AC-3, but instead of a queue of all arcs in the CSP, we start with only the arcs (X_j, X_i) for all X_j that are unassigned variables that are neighbors of X_i

e.g. CSP

csp : $\mathcal{X} = \{A, B, C, D\}$

$\mathcal{D}_A = \{1, 2, 4\}$; $\mathcal{D}_B = \{1, 3, 5\}$; $\mathcal{D}_C = \{3, 5\}$; $\mathcal{D}_D = \{0, 1, 2\}$

$\mathcal{C} = \left\{A > B\,; B = C\,; D < \frac{1}{2}B\,; C > D^2\right\}$



e.g. Backtrack without Hueristic

$\{A = 1\}$

$\{A = 1, B = 1\}$ inconsistent with $A > B$

$\{A = 1, B = 3\}$ inconsistent with $A > B$

$\{A = 1, B = 5\}$ inconsistent with $A > B$

No valid for B, return $result = failure$, Backtrack to $\{A = next\ value\}$

$\{A = 2\}$

$\{A = 2, B = 1\}$

$\{A = 2, B = 1, C = 3\}$ inconsistent with $B = C$

$\{A = 2, B = 1, C = 5\}$ inconsistent with $B = C$

$\{A = 2, B = 1, C = 7\}$ inconsistent with $B = C$

No valid for C, return $result = failure$, Backtrack to $\{A = 2, B = next\ value\}$

e.g. backtrack without hueristic

$\{A = 2, B = 3\}$ inconsistent with $A > B$

$\{A = 2, B = 5\}$ inconsistent with $A > B$

No valid for B, return $result = failure$, Backtrack to $\{A = next\ value\}$

$\{A = 4\}$

$\{A = 4, B = 1\}$

$\{A = 4, B = 1, C = 3\}$ inconsistent with $B = C$

$\{A = 4, B = 1, C = 5\}$ inconsistent with $B = C$

$\{A = 4, B = 1, C = 7\}$ inconsistent with $B = C$

No valid for C, return $result = failure$, Backtrack to $\{A = 4, B = next\ value\}$

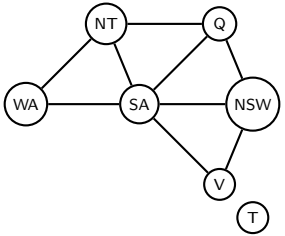
e.g. Backtrack without hHueristic

$\{A = 4, B = 3\}$

$\{A = 4, B = 3, C = 3\}$

$\{A = 4, B = 3, C = 3, D = 0\}$ *assignment* **Full**

e.g. Australia map



$\{WA = R\}$

$\{WA = R, NT = R\}$ inconsistent with $WA \neq NT$

$\{WA = R, NT = G\}$

$\{WA = R, NT = G, SA = R\}$ inconsistent with $WA \neq SA$

$\{WA = R, NT = G, SA = G\}$ inconsistent with $NT \neq SA$

e.g. backtrack without hueristic

$\{WA = R, NT = G, SA = B\}$

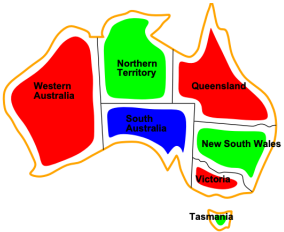
$\{WA = R, NT = G, SA = B, Q = R\}$

$\{WA = R, NT = G, SA = B, Q = R, NSW = R\}$ inconsistent with $Q \neq NSW$

$\{WA = R, NT = G, SA = B, Q = R, NSW = G\}$

$\{WA = R, NT = G, SA = B, Q = R, NSW = G, V = R\}$

$\{WA = R, NT = G, SA = B, Q = R, NSW = G, V = R, T = R|G|B\}$ *assignment* **Full**

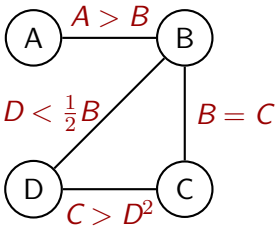


e.g. csp

csp : $\mathcal{X} = \{A, B, C, D\}$

$\mathcal{D}_A = \{1, 2, 4\}$; $\mathcal{D}_B = \{1, 3, 5\}$; $\mathcal{D}_C = \{3, 5\}$; $\mathcal{D}_D = \{0, 1, 2\}$

$\mathcal{C} = \left\{A > B\,; B = C\,; D < \frac{1}{2}B\,; C > D^2\right\}$



e.g. Backtrack with Hueristic

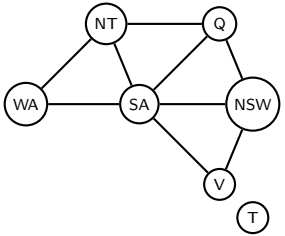
$\{B = 3\}$ **minimum-remaining-values (MRV), least-constraining-value**

$\Rightarrow dom(A) = \{4\}$, $dom(B) = \{3\}$, $dom(C) = \{3\}$, $dom(D) = \{0, 1\}$

$\{B = 3, A = 4, C = 3\}$

$\{B = 3, A = 4, C = 3, D = 0\}$ *assignment* **Full**

e.g. Australia map



$\{SA = R\}$. heuristic: **minimum-remaining-values (MRV), least-constraining-value**

$\Rightarrow SA\{R\}$ $WA\{GB\}$, $NT\{GB\}$, $Q\{GB\}$, $NSW\{GB\}$, $V\{GB\}$, $T\{RGB\}$

$\{SA = R, WA = R\}$ inconsistent with $SA \neq WA$

e.g. Backtrack with Hueristic

$\{SA = R, WA = G\}$, heuristic: **(MRV), least-constraining-value**

$\Rightarrow SA\{R\}$ $WA\{G\}$, $NT\{B\}$, $Q\{G\}$, $NSW\{B\}$, $V\{G\}$, $T\{RGB\}$

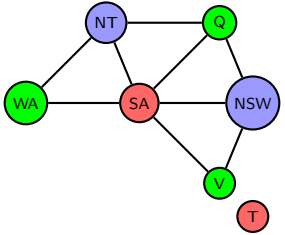
$\{SA = R, WA = G, NT = B\}$, heuristic: **(MRV), least-constraining-value**

$\{SA = R, WA = G, NT = B, Q = G\}$, heuristic: ...

$\{SA = R, WA = G, NT = B, Q = G, NSW = B\}$, heuristic: ...

$\{SA = R, WA = G, NT = B, Q = G, NSW = B, V = G\}$, heuristic: ...

$\{SA = R, WA = G, NT = B, Q = G, NSW = B, V = G, T = R|G|B\}$ *assignment* **Full**



Local Search for CSP

Concept: Unlike Backtracking which builds a solution one variable at a time, Local Search starts with a **complete assignment** for all variables and iteratively modifies values to eliminate constraint violations.

- **Initial State:** Every variable is assigned a value (often randomly).
- **Objective Function:** Typically the number of **conflicted constraints** (we want to minimize this to 0).
- **Key Advantages:** Extremely low memory requirements.
  - Can find solutions in massive state spaces where Backtracking would fail.

The Min-Conflicts Heuristic

The Algorithm:

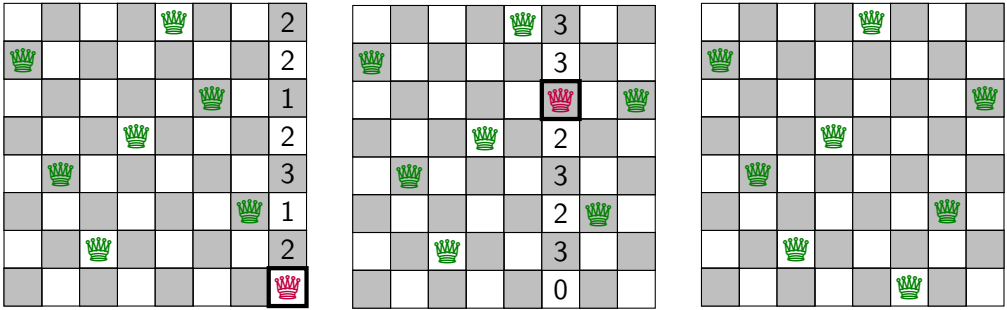
1. **Select:** Randomly pick a conflicted variable X.
2. **Move:** Assign v to X that minimizes the total number of conflicts in the entire problem.
3. **Repeat:** Continue until a solution is found or a maximum number of steps is reached.

Performance: For many problems (like the n-Queens problem), Min-Conflicts can find a solution in almost constant time, even for $n = 1,000,000$!

Min-Conflicts

```
func MIN-CONFLICTS(csp, max_steps) returns a solution or failure
    input: csp, a constraint satisfaction problem
    max_steps, the number of steps allowed before giving up
    current ← an initial complete assignment for csp
    for i = 1 to max_steps do
        if current is a solution for csp then
            return current
        var ← a randomly chosen conflicted variable from csp.VARIABLES
        value ← the value v for var that minimizes
          CONFLICTS(csp, var, v, current)
        set var = value ∈ current
    return failure
```

e.g. 8-Queens



MIN-CONFLICTS local search algorithm for CSP.

initial state: chosen randomly or by a greedy assignment minimal-conflict value.

each variable in turn: **CONFLICTS** function counts the number of constraints violated.

Introduction to Knowledge-Based Agents

1. **Limitations of Simple Reflex Agents**
   - They only react to the current percept (The "Here and Now").
   - They lack a deep understanding of the environment and cannot "reason" about things they don't see.

Introduction to Knowledge-Based Agents

2. What is a Knowledge-Based Agent (KBA)?

- **The Core Idea:** An agent that maintains an internal Knowledge Base (KB) to represent the world.
- **Functionality:** It uses Inference to derive new information from what it already knows and what it perceives.
- **Flexibility:** KBAs can be told new goals or learn new facts without needing to be "re-programmed."

The Knowledge Base (KB)

1. **Definition**

   A Knowledge Base is a set of Sentences (not English sentences, but formal representations) expressed in a Knowledge Representation Language.

2. **Two Fundamental Operations**

   - **TELL:** Providing the agent with new information (Background knowledge or current percepts).
   - **ASK:** Querying the agent to find out what it should do next or what is true about the world.

The Knowledge Base (KB)

3. **The Reasoning Process**

   - When the agent is ASKed a question, the Inference Engine processes the sentences in the KB to produce an answer, even if that answer wasn't explicitly TELLed to it before.

The "TELL-ASK" Cycle (Algorithm)

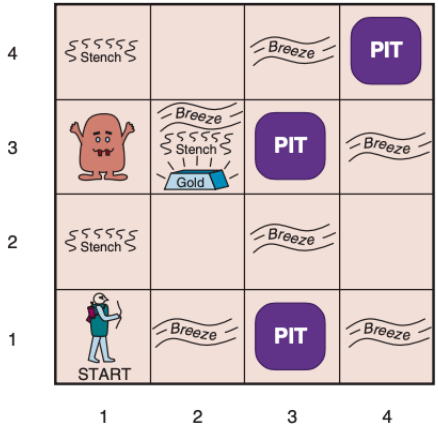
To behave intelligently, the agent follows this loop for every time step t:

1. **TELL** the **KB** about the current percept at time t.
2. **ASK** the **KB** what action should be performed at time t.
3. **TELL** the **KB** that the chosen action has been executed.

Analogy: Think of a doctor.

- **TELL**: "The patient has a fever." (Percept)
- **ASK**: "What is the diagnosis?" (Inference)
- **TELL**: "I am prescribing paracetamol." (Action taken)

The Wumpus World



Move around a square board looking for **Gold** while avoiding **Pits** and the **Wumpus**.

The Wumpus World

Performance Measure

- **+1000** reward **points** if the agent **comes out** of the cave **with the gold**.
- **-1000** **points** penalty for being **eaten** by the Wumpus or **falling** into the **pit**.
- **-1** for each action, and **-10** for using an shoot **arrow**.
- The **game ends** if either agent **dies** or **came out of the cave**.

The Wumpus World

Environment

- A **4*4 grid** of rooms.
- The agent **initially** in room **square [1, 1]**, **facing toward the right**.
- Location of **Wumpus and gold** are chosen randomly **except the first square [1,1]**.
- Each square of the cave can be a **pit** with **probability 0.2** except the **first square**.

The Wumpus World

Actuators

- Turn Left.
- Turn Right.
- Move Forward.
- Grab.
- Release.
- Shoot.

Sensors

The agent has five sensors, each of which gives a **single bit of information**:

$[Stench, Breeze, Glitter, Bump, Scream]$ or *None* for its position.

The Wumpus World

1. *Stench*: In the squares directly (not diagonally) adjacent to the **wumpus**, the agent will perceive a **Stench**.
2. *Breeze*: In the squares directly adjacent to a **pit**, the agent will perceive a **Breeze**.
3. *Glitter*: In the square where the **gold** is, the agent will perceive a **Glitter**.

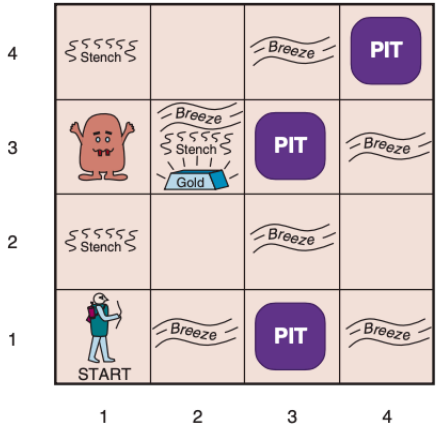
Sensors

The agent has five sensors, each of which gives a **single bit of information**:

$$[Stench, Breeze, Glitter, Bump, Scream]$$ or *None* for its position.

4. $Bump$: When an agent walks into a **wall**, it will perceive a **Bump**.
5. $Scream$: When the wumpus is **killed**, it emits a woeful **Scream** that can be perceived anywhere in the cave.

Wumpus World, Initial



Wumpus World, First step



(a) (b)

(a) The initial situation, after percept:
[None, None, None, None, None].

(b) After moving to [2,1], perceiving:
[None, Breeze, None, None, None].

Wumpus World, Two later stages



(a) (b)

(a) After moving to [1,1] and then [1,2], and perceiving:
[Stench,None,None,None,None].

(b) After moving to [2,2] and then [2,3], and perceiving:
[Stench,Breeze,Glitter,None,None].

Fundamentals of Logic

Logic is a formal language for representing information such that conclusions can be drawn.

Fundamental concepts of Logic

- **Sentences**: A technical term. It describe the logic.
- **Knowledge bases**: A set of sentences.
- **Syntax**: Syntax of the representation language.
- **Semantics**: Meaning of sentence.
- **Truth**: The semantics of sentences.
- **Standard logics**: **true** or **false** - there is no "in between."

Model - Entailment - Inference

- **Model**: Mathematical abstractions. It is in place of "possible world" and assign the Truth value (true or false) for every relevant sentence
- **Satisfaction**: If sentence α is true in model m, saying m satisfies α or sometimes m is a **model** of α
- $M(\alpha)$: Set of all **models** of α
- **Entailment** a sentence *follows logically* from another sentence:

$$\alpha \models \beta$$

Sentence α **entails** the sentence β **if and only if**, in every model in which α is **true**, β is also **true**.

$$\alpha \models \beta \quad \textit{if and only if} \quad M(\alpha) \subseteq M(\beta)$$

Fundamentals of Logic

- **Inference** is an algorithm i can derive α from *KB*, we write:

$$KB \vdash_i \alpha$$

Pronounce: "α **is derived from** KB **by** i" or "i **derives** α **from** KB"

Soundness: i is sound if whenever $KB \vdash_i \alpha$, it is also true that $KB \models \alpha$

Completeness: i is complete if whenever $KB \models \alpha$, it is also true that $KB \vdash_i \alpha$

Propositional Logic

- **Syntax** of propositional logic defines the allowable sentences.
- **Proposition symbol**: P, Q, R, $W_{1,3}$ and *FacingEast* ...
- **Atomic sentences** consists of a single proposition symbol.
- **Complex sentences** are constructed from simpler sentences, using parentheses and operators called **logical connectives**.
- five **connectives** in common use:
  - $\neg$ (not). **negation** of.
  - $\wedge$ (and). **conjunction**.
  - $\vee$ (or). **disjunction**.
  - $\Rightarrow$ (implies). sometimes written $\supset$ or $\rightarrow$. **rules** or **if - then**
  - $\Leftrightarrow$ (if only if - iff). **biconditional**

BNF (Backus–Naur Form)

$$
\begin{aligned}
\textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} &\rightarrow \textit{True} \mid \textit{False} \mid P \mid Q \mid R \mid \ldots \\
\textit{ComplexSentence} &\rightarrow (\textit{Sentence}) \\
&\mid \neg \textit{Sentence} \\
&\mid \textit{Sentence} \wedge \textit{Sentence} \\
&\mid \textit{Sentence} \vee \textit{Sentence} \\
&\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
&\mid \textit{Sentence} \Leftrightarrow \textit{Sentence} \\
\text{OPERATOR PRECEDENCE} &: \neg, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{aligned}
$$

BNF grammar of sentences in propositional logic.

Semantics

The semantics defines the rules for determining the truth of a sentence.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

Truth tables for the five logical connectives.

In any model m:

- $\neg P$ is *true* iff P is *false* in m.
- $P \wedge Q$ is *true* iff both P and Q are *true* in m.
- $P \vee Q$ is *true* iff either P or Q is *true* in m.
- $P \Rightarrow Q$ is *true* unless P is *true* and Q is *false* in m.
- $P \Leftrightarrow Q$ is *true* iff P and Q are both *true* or both *false* in m.

The Simple Inference Procedure (Model Checking)

1. **The Concept of Model Checking**
   Inference by Model Checking is the most basic way to answer the question: "Does the Knowledge Base **KB** entail a sentence α?" ($KB \models \alpha$)
2. The 3-Step Procedure:
   - 2.1 **Generate:** List all **possible Models** (all possible combinations of **True/False** for the symbols involved).
   - 2.2 **Filter:** Identify the models where the **KB is True**. (A model satisfies the KB only if it satisfies every sentence within it).
   - 2.3 **Verify:** In those specific models, check if the sentence α **is also True**.
     - If α is True in ALL models where **KB** is **True** $\rightarrow KB \models \alpha$ (Confirmed).
     - If there is even one model where **KB** is **True** but α is **False** $\rightarrow KB \not\models \alpha$.
3. **Complexity Note:**
   - For n logical symbols, there are 2^n models to check.
   - This procedure is Sound and Complete for Propositional Logic.

Implementing Inference – The Truth Table Method

1. **What is a Truth Table in Logic?**
   A Truth Table is a complete enumeration of all possible Models for a given set of logical symbols. It is the most direct implementation of the Model Checking procedure.
2. **Why use Truth Tables for Inference?**
   - **Visualization:** It provides a clear, row-by-row view of every possible "world."
   - **Systematic:** It ensures no model is overlooked.
   - **Proving Entailment:**
     - Each **Row** = A Model.
     - Each **Column** = A Sentence (**KB** or α).
     - **The Rule:** If α is **True** in every row where the **KB** column is **True**, then $KB \models \alpha$.
3. **Limitations:**
   - If we have n symbols, we need 2^n rows.
   - e.g. 20 variables $\rightarrow$ 1,048,576 rows. (This is why we will later need more efficient algorithms like DPLL or Resolution).

Truth-table algorithm

function TT-ENTAILS?(KB, α) **returns** *true* or *false*
  inputs: KB, the knowledge base, a sentence in propositional logic
    α, the query, a sentence in propositional logic

  $symbols \leftarrow$ a list of the proposition symbols in KB and α
  return TT-CHECK-ALL($KB, \alpha, symbols, \{\}$)

function TT-CHECK-ALL($KB, \alpha, symbols, model$) **returns** *true* or *false*
  if EMPTY?($symbols$) **then**
    if PL-TRUE?($KB, model$) **then return** PL-TRUE?($\alpha, model$)
    else return *true* // when KB is false, always return true
  else
    $P \leftarrow$ FIRST($symbols$)
    $rest \leftarrow$ REST($symbols$)
    return (TT-CHECK-ALL($KB, \alpha, rest, model \cup \{P = true\}$)
      and
      TT-CHECK-ALL($KB, \alpha, rest, model \cup \{P = false\}$)

TT: Truth-Table algorithm for deciding propositional entailment.
PL-TRUE?: Propositional Logic True?

KB for Wumpus World

Symbols for each $[x, y]$ location

- $P_{x,y}$ is $true$ if there is a **Pit** in $[x, y]$.
- $W_{x,y}$ is $true$ if there is a **Wumpus** in $[x, y]$, dead or alive.
- $B_{x,y}$ is $true$ if there is a **Breeze** in $[x, y]$.
- $S_{x,y}$ is $true$ if there is a **Stench** in $[x, y]$.
- $L_{x,y}$ is $true$ if the agent is in **Location** $[x, y]$.

KB for Wumpus World

Scenario: The Agent is in Square $[1,1]$. There is no breeze in $[1,1]$, but the Agent wants to know if there are **Pits** in the adjacent squares $[2,1]$ and $[1,2]$.

1. **Building the Knowledge Base (KB):**
   - **Rule 1:** The Agent perceives no breeze in $[1,1]$:

     $R_1 : \neg B_{1,1}$
   - **Rule 2:** A square is breezy if and only if there is a pit in an adjacent square. For $[1,1]$:

     $R_2 : B_{1,1} \Leftrightarrow (P_{2,1} \vee P_{1,2})$
   - $KB = R_1 \wedge R_2$
2. **Goal:**

   Check if the KB entails $\alpha_1(\neg P_{2,1})$ or $\alpha_2(\neg P_{1,2})$.

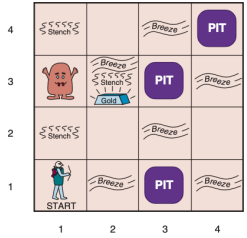
KB for Wumpus World

Truth table

$B_{1,1}$	$P_{2,1}$	$P_{1,2}$	$P_{2,1} \vee P_{1,2}$	R_1: $\neg B_{1,1}$	R_2: $B_{1,1} \Leftrightarrow (P_{2,1} \vee P_{1,2})$	$KB = R_1 \wedge R_2$	$\alpha_1(\neg P_{2,1})$	$\alpha_2(\neg P_{1,2})$
F	F	F	F	T	T	T	T	T
F	F	T	T	T	F	F	T	F
F	T	F	T	T	F	F	F	T
F	T	T	T	T	F	F	F	F
T	F	F	F	F	F	F	T	T
T	F	T	T	F	T	F	T	F
T	T	F	T	F	T	F	F	T
T	T	T	T	F	T	F	F	F

From Truth tabe we have:

- $M(KB) \subseteq M(\alpha_1(\neg P_{2,1}))$: $KB \models \alpha_1(\neg P_{2,1})$,
  No **Pit** at $[2, 1]$
- $M(KB) \subseteq M(\alpha_2(\neg P_{1,2}))$: $KB \models \alpha_2(\neg P_{1,2})$,
  No **Pit** at $[1, 2]$



Logical equivalences

Instead of checking all possible models (which is 2^n), we apply Inference Rules to the sentences in our **KB** to construct a **Proof**.

- **Logical equivalence**: α and β are logically equivalent if they are *true* in the same set of models: $\alpha \equiv \beta$

  $\alpha \equiv \beta$ *if and only if* $\alpha \models \beta$ and $\beta \models \alpha$

- **Validity**, **Tautology**:

  A sentence is valid if it is *true* **in all models**.

- **Deduction theorem**:

  α and β, $\alpha \models \beta$ *iff sentence* $(\alpha \Rightarrow \beta)$ is **valid**.

- **Satisfiability**: if the sentence is true in, or satisfied by, some model.

Logical equivalences

- **Validity and satisfiability are connected**:
  - α is **valid** iff $\neg\alpha$ is **unsatisfiable**.
  - α is **satisfiable** iff $\neg\alpha$ is **not valid**.

$$\alpha \models \beta \text{ iff the sentence } (\alpha \wedge \neg\beta) \textbf{ is unsatisfiable}.$$

- Proof by **contradiction**:
  - Proving β from α by checking the unsatisfiability of $(\alpha \wedge \neg\beta)$.
  - Assumes a sentence β to be false and shows that this leads to a **contradiction** with known axioms α.
  - It is meant the sentence $(\alpha \wedge \neg\beta)$ is unsatisfiable

Logical equivalences

Two kind of logical operator:

- **Entail, equivalences:** $\models$, $\equiv$
- **Connectives:** $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

Inference Rules and Proofs

Searching algorithms can be used to find a sequence of the steps.

Proof problem as follows:

- **INITIAL STATE**: the initial knowledge base.
- **ACTIONS**: the set of actions consists of all the inference rules applied to all the sentences that match the top half of the inference rule.
- **RESULT**: the result of an action is to add the sentence in the bottom half of the inference rule.
- **GOAL**: the goal is a state that contains the sentence we are trying to prove.

Proof by truth-table algorithm: It would be overwhelmed by the exponential explosion of models

Inference Rules

- **Modus Ponens** Give $\alpha \Rightarrow \beta$ and α, then β.

$$\frac{\alpha \Rightarrow \beta,\ \alpha}{\beta}$$

e.g. Give $(\mathit{WumpusAhead} \wedge \mathit{WumpusAlive}) \Rightarrow \mathit{Shoot}$
and $(\mathit{WumpusAhead} \wedge \mathit{WumpusAlive})$, then *Shoot*.

- **And-Elimination** from a $\wedge$ conjunction, any of the conjuncts can be inferred:

$$\frac{\alpha \wedge \beta}{\alpha}$$

e.g. Give $(\mathit{WumpusAhead} \wedge \mathit{WumpusAlive})$, then *WumpusAlive*.

Inference Rules

e.g.

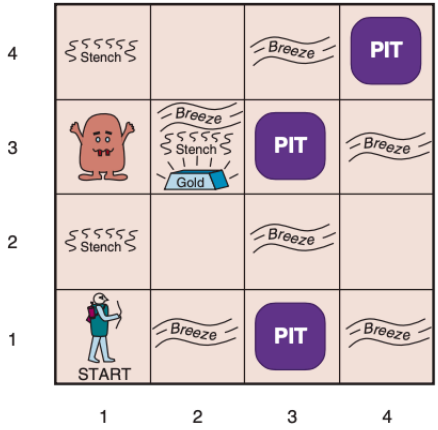
$$\frac{\alpha \Rightarrow \beta}{\neg\alpha \vee \beta} \quad , \quad \frac{\neg\alpha \vee \beta}{\alpha \Rightarrow \beta} \quad , \quad \frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}$$

monotonicity: increase as information.

Any sentences α and β,

if $KB \models \alpha$ **then** $KB \wedge \beta \models \alpha$

Inference for Wumpus World



Inference for Wumpus World

KB: $R_1 : \neg P_{1,1}$
$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
$R_4 : \neg B_{1,1}$ $R_5 : B_{2,1}$

1. "$\Leftrightarrow$" to R_2:
   $R_6 : (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
2. And-Elimination to R_6:
   $R_7 : ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
3. "$\Rightarrow$" to R_7:
   $R_8 : (\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1}))$
4. Modus Ponens with R_8 and the percept R_4 ($\neg B_{1,1}$):
   $R_9 : \neg(P_{1,2} \vee P_{2,1})$
5. De Morgan's rule, giving the conclusion:
   $R_{10} : \neg P_{1,2} \wedge \neg P_{2,1}$: **Neither [1,2] nor [2,1] contains a Pit.**

Proof by Resolution

- **The Resolution Inference Rule**
  Resolution is a powerful inference rule that yields a complete inference algorithm when combined with a search strategy.
- **Basic Rule:** If we have $(A \vee B)$ and $(\neg B \vee C)$, we can derive $(A \vee C)$.
- **Intuition:** Since B must be either True or False, one of the two clauses must contribute a true literal to the result.

Resolution Inference Rule

- **clause** disjunction of literal
- **Unit resolution rule**

  ℓ: literal

  ℓ_i and m: Complementary literals (one is the negation of the other)

$$\frac{\ell_1 \vee ... \vee \ell_k, \quad m}{\ell_1 \vee ... \vee \ell_{i-1} \vee \ell_{i+1} \vee ... \vee \ell_k}$$

- **Full resolution rule**

  ℓ_i and m_j: Complementary literals

$$\frac{\ell_1 \vee ... \vee \ell_k, \quad m_1 \vee ... \vee m_n}{\ell_1 \vee ... \vee \ell_{i-1} \vee \ell_{i+1} \vee ... \vee \ell_k \vee m_1 \vee ... \vee m_{j-1} \vee m_{j+1} ... \vee m_n}$$

  another represent:

$$\frac{\{p, r\}, \{\neg p, k\}}{\{r, k\}}$$

e.g. Wumpus World

• agent returns from [2,1] to [1,1] and then goes to [1,2]. Add the following facts to the KB:

$R_{11} : \neg B_{1,2}$
$R_{12} : B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3})$

• absence of pits in [2,2] and [1,3]

$R_{13} : \neg P_{2,2}$
$R_{14} : \neg P_{1,3}$

• apply biconditional elimination to R_3, followed by Modus Ponens with R_5, to obtain the fact that there is a pit in [1,1], [2,2], or [3,1]

$R_{15} : P_{1,1} \vee P_{2,2} \vee P_{3,1}$

e.g. Wumpus World

- a pit in one of [1,1], [2,2], and [3,1]? it's not in [2,2], then it's in [1,1] or [3,1].
- apply resolution rule: $\neg P_{2,2}$ in R_{13} with $P_{2,2}$ in R_{15}:

  $R_{16} : P_{1,1} \vee P_{3,1}$
- a pit in [1,1] or [3,1]? it's not in [1,1], then it's in [3,1].
- apply resolution rule: $\neg P_{1,1}$ in R_1 with $P_{1,1}$ in R_{16}:

  $R_{17} : P_{3,1}$: a pit in [3,1]

Conjunctive Normal Form (CNF)

Every sentence of propositional logic is logically equivalent to a conjunction of clauses.

Grammar for conjunctive normal form, Horn clauses, and definite clauses:

$$
\begin{aligned}
\mathit{CNFSentence} &\rightarrow \mathit{Clause}_1 \wedge \cdots \wedge \mathit{Clause}_n \\
\mathit{Clause} &\rightarrow \mathit{Literal}_1 \vee \cdots \vee \mathit{Literal}_m \\
\mathit{Fact} &\rightarrow \mathit{Symbol} \\
\mathit{Literal} &\rightarrow \mathit{Symbol} \mid \neg \mathit{Symbol} \\
\mathit{Symbol} &\rightarrow P \mid Q \mid R \mid \ldots \\
\mathit{HornClauseForm} &\rightarrow \mathit{DefiniteClauseForm} \mid \mathit{GoalClauseForm} \\
\mathit{DefiniteClauseForm} &\rightarrow \mathit{Fact} \mid (\mathit{Symbol}_1 \wedge \cdots \wedge \mathit{Symbol}_l) \Rightarrow \mathit{Symbol} \\
\mathit{GoalClauseForm} &\rightarrow (\mathit{Symbol}_1 \wedge \cdots \wedge \mathit{Symbol}_l) \Rightarrow \mathit{False}
\end{aligned}
$$

Converting to CNF

e.g. $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ converting to **CNF**

The steps are as follows:

1. Replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
   $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
2. Replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$
   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$
3. CNF requires $\neg$ to appear only in literals, so using:
   $\neg(\neg\alpha) \equiv \alpha$ (double-negation)
   $\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$ (De Morgan)
   $\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$ (De Morgan)
   the e.g. result: $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$
4. Apply the distributivity law $\wedge$ and $\vee$
   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$
   Sentence is now in **CNF**, as a conjunction of three clauses

Resolution algorithm

- Using the principle of proof by contradiction
- To show $KB \models \alpha$, we show that $(KB \wedge \neg\alpha)$ is **unsatisfiable**

Algorithm $(show\ KB \models \alpha)$

1. Convert $(KB \wedge \neg\alpha)$ into **CNF**
2. Apply resolution rule to the resulting clauses
   - Each pair that contains complementary literals is resolved to produce a new clause
   - added to the set if it is not already present
3. Loop step 2 until one of two things happens:
   - **No new clauses that can be added**: $KB \not\models \alpha$ or,
   - 2 clauses resolve to yield the **empty clause**: $KB \models \alpha$.

Resolution algorithm

PL-Resolve returns the set of all possible clauses obtained by resolving its two inputs.

func `PL-Resolution(`*KB*, α`)` **returns** *true* or *false*

- **inputs**: *KB*, the knowledge base, a sentence in propositional logic α, the query, a sentence in propositional logic
- *clauses* $\leftarrow$ the set of clauses in the **CNF** representation of $KB \wedge \neg\alpha$
- *new* $\leftarrow \{\}$
- **while** *true* **do**
  - **foreach** *pair of clauses* C_i, C_j *in clauses* **do**
    - *resolvents* $\leftarrow$ `PL-Resolution(`C_i, C_j`)`
    - **if** *resolvents contains the empty clause* **then**
      - **return** *true*
    - *new* $\leftarrow$ *new* $\cup$ *resolvents*
  - **if** *new* $\subseteq$ *resolvents* **then**
    - **return** *false*
  - *clauses* $\leftarrow$ *clauses* $\cup$ *new*

e.g.

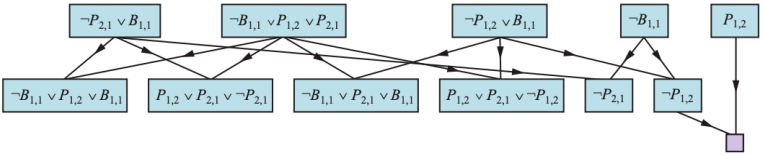
e.g. Agent is in [1,1], no breeze, so no pits in neighboring squares:

$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}) \qquad R_4 : \neg B_{1,1}$$

- $KB = R_2 \wedge R_4 = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$
- Prove $\alpha = \neg P_{1,2}$
- Convert $(KB \wedge \neg\alpha)$ into **CNF**

$(B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1} \wedge \neg(\neg P_{1,2})$

$(\neg P_{2,1} \vee B_{1,1}) \wedge (\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg B_{1,1}) \wedge (P_{1,2})$



Query $\neg P_{1,2}$, PL-RESOLUTION yield the **empty clause**.

So that the query is proven.

e.g. Resolution alg.

Consider the following six statements, all of which to be true:

1. If you go swimming you will get wet.
2. If it is raining and you are outside then you will get wet.
3. If it is warm and there is no rain then it is a pleasant day.
4. You are not wet.
5. You are outside.
6. It is a warm day.

Determine following statements must be true:

a. You are not swimming.

b. It is not raining.

c. It is a pleasant day.

e.g. Resolution alg.

In propositional logic:

1. $swimming \Rightarrow wet$
2. $(rain \wedge outside) \Rightarrow wet$
3. $(warm \wedge \neg rain) \Rightarrow pleasant$
4. $\neg wet$
5. $outside$
6. $warm$

Statements:

a. $\neg swimming$

b. $\neg rain$

c. $pleasant$

e.g. Resolution alg.

a) $\{1, 2, 3, 4, 5, 6\} \models \neg swimming$

Contradition method and convert to CNF:

1. $\neg swimming \vee wet$
2. $\neg rain \vee \neg outside \vee wet$
3. $\neg warm \vee rain \vee pleasant$
4. $\neg wet$
5. $outside$
6. $warm$
7. $swimming$

exercises:

b. $\neg rain$

PL-Resolution:

8. $(1) \wedge (4) : \neg swimming$
9. $(7) \wedge (8) : \blacksquare$

$\neg swimming$

e.g. The Power of False

$P \wedge \neg P \models Z$

We know: $P \wedge \neg P \equiv false$

$P \wedge \neg P \wedge \neg Z$

PL-Resolution:

$P \wedge \neg P : \blacksquare$

Z

Horn clauses and Definite clauses

- **Definite clause**: **disjunction** of literals of which **exactly one is positive**.

  e.g. $(\neg L_{1,1} \vee \neg Breeze \vee B_{1,1})$ is a definite clause,

  $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1})$ is not, because it has two positive clauses

- **Horn clause**: **disjunction** of literals of which **at most one is positive**.

  all definite clauses are Horn clauses.

  KB containing only definite clauses are interesting:

  1. Every definite clause can be written as an implication.

  e.g. $(\neg L_{1,1} \vee \neg Breeze \vee B_{1,1})$ can be written as the implication $(L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$

Horn clauses and Definite clauses

- **Horn clause**

  premise is call **body**

  conclusion is call **head**

  sentence consisting of a single positive literal is call **fact**

  e.g. $L_{1,1}$

  fact $L_{1,1}$ can be written in implication form: $True \Rightarrow L_{1,1}$

  2. Inference with Horn clauses can be done through the **forward-chaining** and **Backward-chaining** algorithms

  3. Entailment with Horn clauses in linear time with the size of knowledge base

Horn clauses and Definite clauses

- **Goal clauses**: clauses with no positive literals.
- **Modus Ponens** (for Horn Form): complete for Horn KBs

$$\frac{\alpha_1, ..., \alpha_n \Rightarrow \beta, \quad \alpha_1, ..., \alpha_n}{\beta}$$

FW and BW Chaining

Inference Engine

Apply **rules** of **KB** to **infer** new information.

Modus Ponens

e.g.

A	It is raining
$A \Rightarrow B$	if it is rainning i will carry an umbrella
B	I will carry an umbrellaq (new knowledge)

- **Forward Chaining**: Start with atomic sentences in the KB and applies inference rules (Modus Ponens) in the forward direction to extract more data until a goal is reached.
- **Backward Chaining**: Starts with the goal and works backward, chaining through rules to find known facts that support the goal.

FW and BW Chaining

e.g.

Forward Chaining

A	He exercises regularly.
$A \Rightarrow B$	if he is exercising regularly, he is fit.
B	He is fit.

Backward Chaining

B	He is fit.
$A \Rightarrow B$	if he is exercising regularly, he is fit.
A	He exercises regularly.

Forward chaining

function PL-FC-ENTAILS?(KB, q) **returns** *true* or *false*
  inputs: KB, the knowledge base, a set of propositional definite clauses
    q, the query, a proposition symbol
  count ← a table, where *count*[c] is initially the number of symbols in clause c's premise
  inferred ← a table, where *inferred*[s] is initially *false* for all symbols
  queue ← a queue of symbols, initially symbols known to be true in KB

  while *queue* is not empty **do**
    p ← POP(*queue*)
    if $p = q$ **then return** *true*
    if *inferred*[p] = *false* **then**
      inferred[p] ← *true*
      for each clause c in KB where p is in c.PREMISE **do**
        decrement *count*[c]
        if *count*[c] = 0 **then** add c.CONCLUSION to *queue*
  return *false*

The forward-chaining algorithm for propositional logic

- **Forward chaining is sound and complete for** *Horn KB*

e.g. Forward chaining

Goal Q

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L, \quad A, \quad B$

No.	Reached	Queue
0		A B

Inferred	
A	false
B	false
L	false
M	false
P	false
Q	false

Count	
$P \Rightarrow Q$	1
$L \wedge M \Rightarrow P$	2
$B \wedge L \Rightarrow M$	2
$A \wedge P \Rightarrow L$	2
$A \wedge B \Rightarrow L$	2

e.g. Forward chaining

Goal Q

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L.$ $A,$ B

No.	Reached	Queue
0		A B
1	A	B

Inferred	
A	**true**
B	false
L	false
M	false
P	false
Q	false

Count	
$P \Rightarrow Q$	1
$L \wedge M \Rightarrow P$	2
$B \wedge L \Rightarrow M$	2
$A \wedge P \Rightarrow L$	**1**
$A \wedge B \Rightarrow L$	**1**

e.g. Forward chaining

Goal Q

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L, \quad A, \quad B$

No.	Reached	Queue
0		A B
1	A	B
2	B	**L**

Inferred	
A	true
B	**true**
L	false
M	false
P	false
Q	false

Count	
$P \Rightarrow Q$	1
$L \wedge M \Rightarrow P$	2
$B \wedge L \Rightarrow M$	**1**
$A \wedge P \Rightarrow L$	1
$A \wedge B \Rightarrow L$	**0**

e.g. Forward chaining

Goal Q

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$. A, B

No.	Reached	Queue
0		A B
1	A	B
2	B	L
3	L	**M**

Inferred	
A	true
B	true
L	**true**
M	false
P	false
Q	false

Count	
$P \Rightarrow Q$	1
$L \wedge M \Rightarrow P$	**1**
$B \wedge L \Rightarrow M$	**0**
$A \wedge P \Rightarrow L$	1
$A \wedge B \Rightarrow L$	0

e.g. Forward chaining

Goal Q

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L.$ $A,$ B

No.	Reached	Queue
0		A B
1	A	B
2	B	L
3	L	M
4	M	**P**

Inferred	
A	true
B	true
L	true
M	**true**
P	false
Q	false

Count	
$P \Rightarrow Q$	1
$L \wedge M \Rightarrow P$	**0**
$B \wedge L \Rightarrow M$	0
$A \wedge P \Rightarrow L$	1
$A \wedge B \Rightarrow L$	0

e.g. Forward chaining

Goal Q

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L, \quad A, \quad B$

No.	Reached	Queue
0		A B
1	A	B
2	B	L
3	L	M
4	M	P
5	P	**L Q**

Inferred	
A	true
B	true
L	true
M	true
P	**true**
Q	false

Count	
$P \Rightarrow Q$	**0**
$L \wedge M \Rightarrow P$	0
$B \wedge L \Rightarrow M$	0
$A \wedge P \Rightarrow L$	**0**
$A \wedge B \Rightarrow L$	0

e.g. Forward chaining

Goal Q

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L, \quad A, \quad B$

No.	Reached	Queue
0		A B
1	A	B
2	B	L
3	L	M
4	M	P
5	P	L Q
6	L	Q

Inferred	
A	true
B	true
L	true
M	true
P	true
Q	false

Count	
$P \Rightarrow Q$	0
$L \wedge M \Rightarrow P$	0
$B \wedge L \Rightarrow M$	0
$A \wedge P \Rightarrow L$	0
$A \wedge B \Rightarrow L$	0

e.g. Forward chaining

Goal Q

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L, \quad A \wedge B \Rightarrow L, \quad A, \quad B$

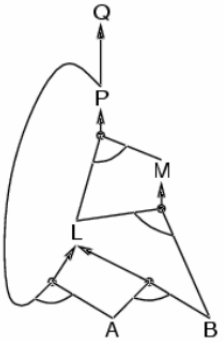
No.	Reached	Queue
0		A B
1	A	B
2	B	L
3	L	M
4	M	P
5	P	L Q
6	L	Q
7	**Q**	

Inferred	
A	true
B	true
L	true
M	true
P	true
Q	false

Count	
$P \Rightarrow Q$	0
$L \wedge M \Rightarrow P$	0
$B \wedge L \Rightarrow M$	0
$A \wedge P \Rightarrow L$	0
$A \wedge B \Rightarrow L$	0

e.g. Forward chaining

$$P \Rightarrow Q$$
$$L \wedge M \Rightarrow P$$
$$B \wedge L \Rightarrow M$$
$$A \wedge P \Rightarrow L$$
$$A \wedge B \Rightarrow L$$
$$A$$
$$B$$



Backward chaining

Idea:

Check whether a particular fact Q is *true*

Backward chaining

Given a fact Q to be "proven",

1. See if Q is already in the **KB**. If so, return *true*.
2. Find all implications, I, whose conclusion "matches" Q.
3. Recursively establish the premises of all i in I via backward chaining.

e.g. Backward chaining

Goal Q

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L, \quad A, \quad B$

No.	Reached	Goal stack
0		Q

Inferred	
A	false
B	false
L	false
M	false
P	false
Q	false

e.g. Backward chaining

Goal Q ✔

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L, \quad A, \quad B$

No.	Reached	Goal stack
0		Q
1	Q	$P \Rightarrow Q$

Inferred	
A	false
B	false
L	false
M	false
P	false
Q	false

e.g. Backward chaining

Goal Q ✔

$P \Rightarrow Q$ ✔

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L, \quad A, \quad B$

No.	Reached	Goal stack
0		Q
1	Q	$P \Rightarrow Q$
2	$P \Rightarrow Q$	$L \wedge M \Rightarrow P$

Inferred	
A	false
B	false
L	false
M	false
P	false
Q	false

e.g. Backward chaining

Goal Q ✔

$P \Rightarrow Q$ ✔

$L \wedge M \Rightarrow P$ ✔

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L, \quad A, \quad B$

No.	Reached	Goal stack
0		Q
1	Q	$P \Rightarrow Q$
2	$P \Rightarrow Q$	$L \wedge M \Rightarrow P$
3	$L \wedge M \Rightarrow P$	$A \wedge P \Rightarrow L, \quad B \wedge L \Rightarrow M$

Inferred	
A	false
B	false
L	false
M	false
P	false
Q	false

e.g. Backward chaining

Goal Q ✔

$P \Rightarrow Q$ ✔

$L \wedge M \Rightarrow P$ ✔

$B \wedge L \Rightarrow M$ ✔

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L, \quad A, \quad B$

No.	Reached	Goal stack
0		Q
1	Q	$P \Rightarrow Q$
2	$P \Rightarrow Q$	$L \wedge M \Rightarrow P$
3	$L \wedge M \Rightarrow P$	$A \wedge P \Rightarrow L, \quad B \wedge L \Rightarrow M$
4	$B \wedge L \Rightarrow M$	$A \wedge P \Rightarrow L, \quad B, \quad A \wedge B \Rightarrow L$

Inferred	
A	false
B	false
L	false
M	false
P	false
Q	false

e.g. Backward chaining

Goal Q ✔

$P \Rightarrow Q$ ✔

$L \wedge M \Rightarrow P$ ✔

$B \wedge L \Rightarrow M$ ✔

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$ ✔, A, B

No.	Reached	Goal stack
0		Q
1	Q	$P \Rightarrow Q$
2	$P \Rightarrow Q$	$L \wedge M \Rightarrow P$
3	$L \wedge M \Rightarrow P$	$A \wedge P \Rightarrow L$, $A \wedge B \Rightarrow L$
4	$B \wedge L \Rightarrow M$	$A \wedge P \Rightarrow L$ B, $A \wedge B \Rightarrow L$
5	$A \wedge B \Rightarrow L$	$A \wedge P \Rightarrow L$, B, A

Inferred	
A	false
B	false
L	false
M	false
P	false
Q	false

e.g. Backward chaining

Goal Q ✔

$P \Rightarrow Q$ ✔

$L \wedge M \Rightarrow P$ ✔

$B \wedge L \Rightarrow M$ ✔

$A \wedge P \Rightarrow L$, $A \wedge B \Rightarrow L$ ✔, A ✔, B

No.	Reached	Goal stack
0		Q
1	Q	$P \Rightarrow Q$
2	$P \Rightarrow Q$	$L \wedge M \Rightarrow P$
3	$L \wedge M \Rightarrow P$	$A \wedge P \Rightarrow L$, $A \wedge B \Rightarrow L$
4	$B \wedge L \Rightarrow M$	$A \wedge P \Rightarrow L$
5	$A \wedge B \Rightarrow L$	$A \wedge P \Rightarrow L$, B, A
6	A	$A \wedge P \Rightarrow L$, B

Inferred	
A	**true**
B	false
L	false
M	false
P	false
Q	false

e.g. Backward chaining

Goal Q ✔

$P \Rightarrow Q$ ✔

$L \wedge M \Rightarrow P$ ✔

$B \wedge L \Rightarrow M$ ✔

$A \wedge P \Rightarrow L$, $A \wedge B \Rightarrow L$ ✔, A ✔, B ✔

No.	Reached	Goal stack
0		Q
1	Q	$P \Rightarrow Q$
2	$P \Rightarrow Q$	$L \wedge M \Rightarrow P$
3	$L \wedge M \Rightarrow P$	$A \wedge P \Rightarrow L$, $A \wedge B \Rightarrow L$
4	$B \wedge L \Rightarrow M$	$A \wedge P \Rightarrow L$
5	$A \wedge B \Rightarrow L$	$A \wedge P \Rightarrow L$, B, A
6	A	$A \wedge P \Rightarrow L$, B
7	B	$A \wedge P \Rightarrow L$

Inferred	
A	true
B	**true**
L	false
M	false
P	false
Q	false

e.g. Backward chaining

Goal Q ✔

$P \Rightarrow Q$ ✔

$L \wedge M \Rightarrow P$ ✔

$B \wedge L \Rightarrow M$ ✔, $A \wedge P \Rightarrow L$ ✔, $A \wedge B \Rightarrow L$ ✔, A ✔, B ✔

No.	Reached	Goal stack
0		Q
1	Q	$P \Rightarrow Q$
2	$P \Rightarrow Q$	$L \wedge M \Rightarrow P$
3	$L \wedge M \Rightarrow P$	$A \wedge P \Rightarrow L$, $A \wedge B \Rightarrow L$
4	$B \wedge L \Rightarrow M$	$A \wedge P \Rightarrow L$
5	$A \wedge B \Rightarrow L$	$A \wedge P \Rightarrow L$, B, A
6	A	$A \wedge P \Rightarrow L$, B
7	B	$A \wedge P \Rightarrow L$
8	$A \wedge P \Rightarrow L$	$\{\ \}$

Inferred	
A	true
B	true
L	**true**
M	**true**
P	**true**
Q	**true**

DPLL Algorithm

Davis-Putnam-Logemann-Loveland (DPLL)

The DPLL algorithm for **checking satisfiability** of a sentence in propositional logic.

- **Early termination**:
  - A **clause** is true if any literal is true.
  - A **sentence** is false if any clause is false.
- **Pure symbol heuristic**:
  - **Pure symbol**: always appears with the same "sign" in all clauses.
  - e.g. $(A \vee \neg B)$, $(\neg B \vee \neg C)$, $(C \vee A)$. symbol A is pure; symbol B is pure; symbol C is impure.
  - **Make a pure symbol literal true**.
  - **Ignore clauses** that are already known to be **true** in the model constructed so far.
  - e.g. if the model contains $B = \mathit{false}$, then the clause $(\neg B \vee \neg C)$ is **true**, and in the remaining clauses C appears only as a positive literal; therefore C becomes **pure**.

DPLL Algorithm

- **Unit clause heuristic**:
  - **Unit clause**: only one literal in the clause.
  - The only literal in a unit clause must be **true**.
  - When a litteral assigned **false** by the model, the clause can simplifies by ignore that literal.
  - e.g. if the model contains $B = \mathit{true}$, then $(\neg B \vee \neg C)$ simplifies to $\neg C$, which is a unit clause.
  - Assigning one unit clause can create another unit clause.
  - e.g. when C is set to **false**, $(C \vee A)$ becomes a unit clause, causing **true** to be assigned to A.

DPLL Algorithm

function DPLL-SATISFIABLE?(s) **returns** *true* or *false*
  inputs: s, a sentence in propositional logic

  clauses $\leftarrow$ the set of clauses in the CNF representation of s
  symbols $\leftarrow$ a list of the proposition symbols in s
  return DPLL(*clauses*, *symbols*, { })

function DPLL(*clauses*, *symbols*, *model*) **returns** *true* or *false*

  if every clause in *clauses* is true in *model* **then return** *true*
  if some clause in *clauses* is false in *model* **then return** *false*
  P, *value* $\leftarrow$ FIND-PURE-SYMBOL(*symbols*, *clauses*, *model*)
  if P is non-null **then return** DPLL(*clauses*, *symbols* – P, *model* $\cup$ {P=*value*})
  P, *value* $\leftarrow$ FIND-UNIT-CLAUSE(*clauses*, *model*)
  if P is non-null **then return** DPLL(*clauses*, *symbols* – P, *model* $\cup$ {P=*value*})
  $P \leftarrow$ FIRST(*symbols*); *rest* $\leftarrow$ REST(*symbols*)
  return DPLL(*clauses*, *rest*, *model* $\cup$ {P=*true*}) **or**
      DPLL(*clauses*, *rest*, *model* $\cup$ {P=*false*})

e.g. DPLL

let $(C \vee D)$, $(C \vee \neg D)$, $(\neg D \vee A)$, $(A \vee B)$, $(\neg A \vee \neg B)$, Satisfy?

Solution:

Pure: (C): $C = true$

Clauses: $(\neg D \vee A)$, $(A \vee B)$, $(\neg A \vee \neg B)$

Pure: $(\neg D)$: $D = false$

Clauses: $(A \vee B)$, $(\neg A \vee \neg B)$

not pure or unit clause

$A = true$

Clauses: $(\neg B)$

Pure: $(\neg B)$: $B = false$

Model: $(A = true, B = false, C = true, D = false)$

Learning from Examples

- **Learning**: An agent is learning if it improves its performance after making observations about the world.

  Learning can range from the trivial, such as jotting down a shopping list, to the profound, as when Albert Einstein inferred a new theory of the universe.

- **Machine learning**: When the agent is a computer, we call it machine learning.

  A computer observes some data, builds a model based on data, and uses the model as both a hypothesis about the world and a piece of software that can solve problems.

Learning from Examples

Example:

- We want to learn the class, C, of a "family car."
- We have a set of examples of cars.
- We have a group of people that we survey to whom we show these cars.
- The people look at the cars and label them;
  - The cars that they believe are family cars are **positive examples**.
  - The cars that they are not believe are family cars are **negative examples**.
- Class learning is finding a description that is shared by all **positive examples** and none of the **negative examples**.

Learning from Examples

Let us denote price x_1 and engine power x_2

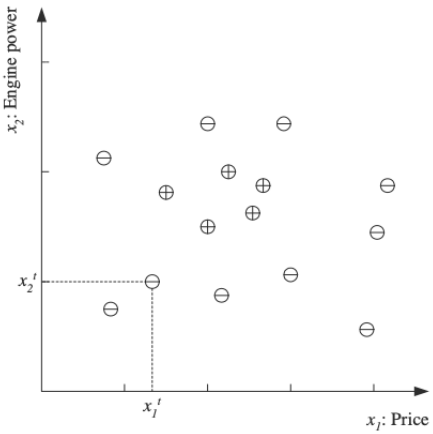
$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

and its label denotes its type

$$y = \begin{cases} 1 & \text{if } x \text{ is a positive example} \\ 0 & \text{if } x \text{ is a negative example} \end{cases}$$

Each car is represented by (x, y) and the training set contains N such examples:

$$\mathcal{D} = \{(x^i, y^i)\}_{i=1}^{N}$$



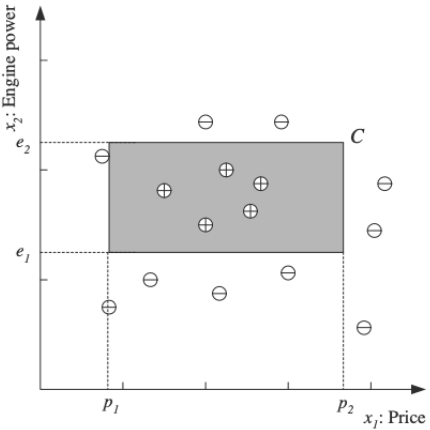
Learning from Examples

- Experts: Family cars within a rectangle C:

  $(p_1 \leqslant \text{price} \leqslant p_2)$ AND

  $(e_1 \leqslant \text{engine power} \leqslant e_2)$

  for suitable values of p_1, p_2, e_1, e_2.
- $\mathcal{H}$, the *hypothesis class*, the set of rectangles.
- $h \in \mathcal{H}$, the learning algorithm finds, to approximate **C** as closely as possible.



Learning from Examples

- Hypothesis h makes a prediction for an instance x such that

$$h(x) = \begin{cases} 1 & \text{if } h \text{ classifies } x \text{ as a positive example} \\ 0 & \text{if } h \text{ classifies } x \text{ as a negative example} \end{cases}$$

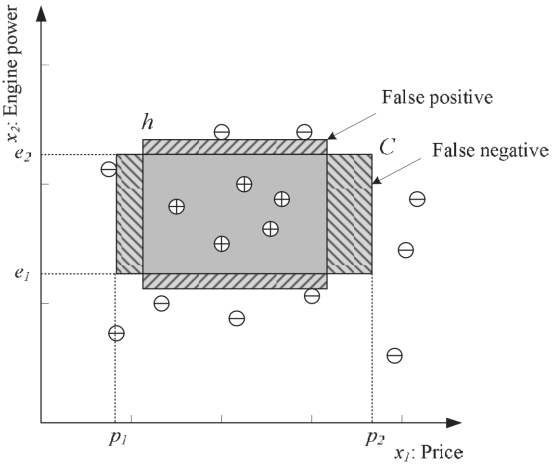
- In real life we do not know $C(x)$, so we cannot evaluate how well $h(x)$ matches $C(x)$.
- Empirical error is the proportion of training instances where predictions of h do not match the required values given in $\mathcal{D}$.
- The error of hypothesis h given the training set $\mathcal{D}$ is

$$E(h|\mathcal{D}) = \sum_{i=1}^{N} 1(h(x^i) \neq y^i)$$

where $1(a \neq b)$ is 1 if $a \neq b$ and is 0 if $a = b$

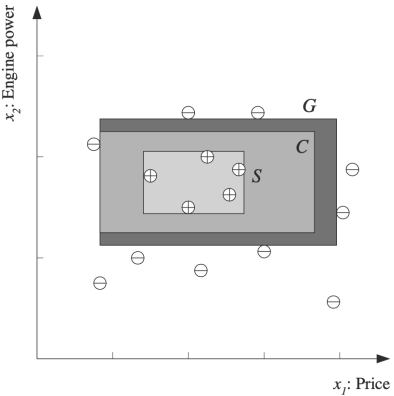
Learning from Examples

- C: Actual class.
- h: Induced hypothesis.
- Points $C = 1, h = 0$: **False negative**.
- Points $C = 0, h = 1$: **False positive**.
- Other Points: **True positives** and **True negatives** are correctly classified.



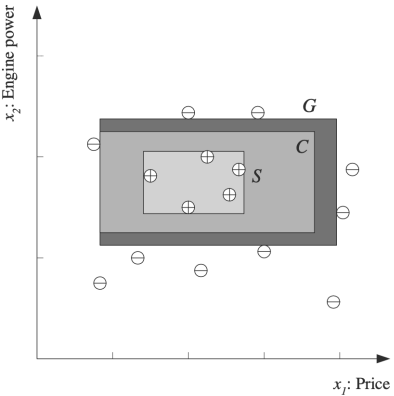
Learning from Examples

- x_1, x_2: Real valued, there are infinitely many such h.
- $(p_1^h, p_2^h, e_1^h, e_2^h)$: Hypothesis $h \in H$, we need to choose the best one.
- Future example x: Different candidate hypotheses may make different predictions.
- Generalization: How well hypothesis will correctly classify future examples that are not part of the training set.



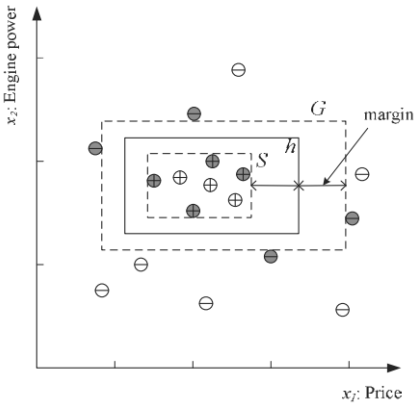
Learning from Examples

- **Most specific hypothesis** S: The tightest rectangle that includes all the positive examples and none of the negative examples.
- **Most general hypothesis** G: The largest rectangle we can draw that includes all the positive examples and none of the negative examples.



Learning from Examples

- Any $h \in H$ between S and G is a valid hypothesis with no error, said to be **consistent** with the training set, and such h make up the version space.
- **Margin**: The distance between S and G.
- Choose h halfway in margin.



Forms of Learning

Based on the nature of the output, we distinguish two main tasks:

- **Classification:**
  - Output: A discrete value from a finite set.
  - e.g. Binary: True, False; Multi-class: Sunny, Cloudy, Rainy
- **Regression:**
  - Output: A continuous numerical value (Integer or Real number).
  - e.g. Predicting tomorrow's temperature; Estimating traffic flow volume.

Main Types of Learning

Feedback is the corrective or informative signal that accompanies the input data. It dictates how the agent adjusts its internal model to improve performance.

The nature of the feedback determines the type of learning the agent will perform.

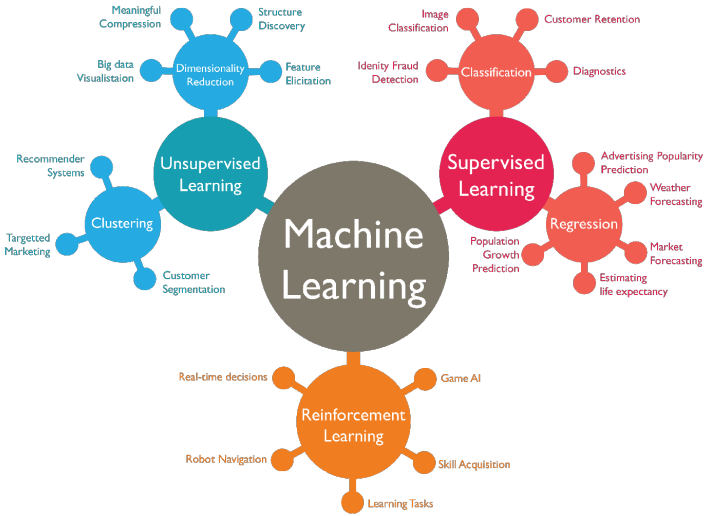
Three Main Types of Learning:

- **Supervised Learning**
  - **Feedback:** Explicit labels or correct answers provided for each input.
  - **Mechanism:** The agent learns a mapping from inputs to outputs by comparing its predictions with the "ground truth."
  - e.g. A dataset of emails labeled as "Spam" or "Not Spam."
- **Unsupervised Learning**
  - **Feedback: None**. The agent receives no labels and no explicit rewards.
  - **Mechanism:** The agent identifies hidden patterns, structures, or clusters within the raw data itself.
  - e.g. Grouping news articles into similar topics automatically.

Main Types of Learning

- **Reinforcement Learning**
  - **Feedback:** A series of **rewards** or **penalties** (reinforcements).
  - **Mechanism:** The agent learns through "trial and error." It doesn't get the "right answer" immediately, but a signal indicating how well it is performing over time.
  - e.g. A robot receiving a $(+1)$ reward for reaching a goal or a (-1) penalty for hitting an obstacle.

Main Types of Learning



PAC Learning (Probably Approximately Correct)

- The Concept: PAC Learning provides a mathematical framework to determine if a learning algorithm is effective.
- Two Criteria for Success:
  1. **Approximately Correct:** The error rate of the hypothesis h is bounded by a small constant ϵ (epsilon).
     - $Error(h) \leq \epsilon$
  2. **Probably:** The probability that the agent fails to find such a hypothesis is bounded by δ (delta).
     - $P(Error(h) > \epsilon) \leq \delta$
- **The PAC Goal:** An agent is a PAC learner if it can find a hypothesis that is "close enough" ϵ to the true function with high confidence $(1 - \delta)$.

Sample Complexity & VC Dimension

How Much Data is Enough?

- **Sample Complexity (n):** The number of training examples required to guarantee a PAC result.
  - **The Rule:** As the Hypothesis Space $\mathcal{H}$ grows larger, we need more examples to "prune" incorrect hypotheses.
  - **Formula:** $n \geq \frac{1}{\epsilon}(\ln \frac{1}{\delta} + \ln |\mathcal{H}|)$
- **Vapnik-Chervonenkis (VC) Dimension:**
  - A measure of the **capacity** (flexibility) of a hypothesis space.
  - It quantifies the complexity of the model even if the hypothesis space is infinite (e.g., linear classifiers in a continuous plane).
- **Key Insight:** Theory tells us the limits of learning. If the VC dimension is too high relative to our data, the risk of Overfitting becomes inevitable.

Why Preprocess Data?

Garbage In, Garbage Out (GIGO)

- **The Problem:** Real-world data is often:
  - **Incomplete:** Lacking attribute values or certain interests (e.g., a student profile without a Mock score).
  - **Noisy:** Containing errors or outliers (e.g., a student studying 500 hours/day).
  - **Inconsistent:** Different formats (e.g., Date as "Jan 1st" vs "01/01").
- **The Goal:** Transform raw data into a clean, consistent format that the learning algorithm can understand.
- **Key Stages:**
  1. Data Cleaning
  2. Data Transformation (Scaling)
  3. Feature Engineering

Data Cleaning

Handling Missing Values and Noise

- **Dealing with Missing Data:**
  - **Ignore the record:** Simple but loses information.
  - **Imputation:** Fill in the gap using the Mean (trung bình), Median (trung vị), or Mode (Yếu vị / Số yếu vị)
- **Handling Outliers:** Identify points that are too far from others (e.g., using a boxplot).
  - Decide whether to remove them or cap them to a maximum value.
- **Smooth Noisy Data:** Using "Binning" or Regression to smooth out random fluctuations.

Data Transformation (Feature Scaling)

Making Attributes Comparable

- **The Problem:** Attributes have different scales.
  - e.g.: Hours Studied (0–20) vs. Mock Score (0–100).
  - In k-NN, the Mock Score will dominate the distance calculation because its numbers are larger.
- **Solution 1:** Min-Max Normalization
  - Scales data to a fixed range, usually $[0, 1]$.
  - $x_{new} = \frac{x - min}{max - min}$

Data Transformation (Feature Scaling)

- **Solution 2:** Standardization (Z-score)
  - Scales data to have a mean of 0 and a standard deviation of 1.
  - $x_{new} = \frac{x-\mu}{\sigma}$
    - Mean μ represents the center of the dataset.

      $\mu = \frac{1}{N}\sum_{i=1}^{N} x_i$

      N : Total number of samples.
      x_i : Value of the i-th sample.
    - Standard Deviation σ measures the degree of dispersion of data relative to the mean.

      $\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$

      Low σ (Small): Data is concentrated near the average.
      High σ (Large): Data is widely distributed (significant fluctuation).

Data Preprocessing with Scikit-learn

1. **Handling Missing Values**
   Instead of manual loops, use **SimpleImputer**.
   - Function: **SimpleImputer(strategy='mean'|'median'|'most_frequent')**
   - e.g. Fill missing Practice Hours with the average value.
2. **Feature Scaling**
   Two main scalers used in almost every Machine Learning project:
   - **MinMaxScaler** Transforms features to a specific range (usually $[0, 1]$).
     - Best for algorithms that don't assume a distribution (e.g., k-NN, Neural Networks).
   - **StandardScaler** Scales data to zero mean and unit variance (z-score).
     - Best for algorithms that assume Gaussian distribution (e.g., SVM, Linear Regression).

Data Preprocessing with Scikit-learn

3. **Encoding Categorical Data**
   Computers need numbers, not strings.
   - **LabelEncoder** Converts labels to integers (e.g., Tank, Mage $\rightarrow$ 0, 1). Use for Target variables.
   - **OneHotEncoder** Creates dummy variables (binary vectors). Use for Input features to avoid unintended "ranking" between categories.
4. The **fit** and **transform** Pattern Scikit-learn uses a consistent API for all preprocessing tools:
   4.1 **.fit()** Calculates parameters (e.g., calculates Mean and Std Dev).
   4.2 **.transform()** Applies the transformation to the data.
   4.3 **.fit_transform()** Does both in one efficient step.

Data Preprocessing with Scikit-learn

From Raw Data

Student ID	Practice Hours	Mock Score	Role	Result
S01	10.0	80.0	Assassin	Pass
S02	**NaN**	30.0	Tank	Fail
S03	15.5	**999.0**	Mage	Pass
S04	5.0	85.0	**assasin**	Pass
S05	4.0	50.0	Tank	Fail
S06	5.0	85.0	Assassin	Pass
S07	1.0	20.0	Support	Fail

- Assassin (Sát thủ)
- Tank (Đỡ đòn)
- Mage (Pháp sư)
- Support (Hỗ trợ)

- S02: Missing Hours.
- S03: Outlier Mock Score = 999.
- S04: Typo: "assasin".

Step 1 - Data Cleaning

Refining the Raw Dataset

- **Handling Missing Values:**
  - Identify "**NaN**" (Not a Number) or empty cells.
  - Action: Fill with the **Mean** (Average) or **Median**.
- **Handling Outliers:**
  - Remove or "cap" values that are mathematically impossible or extreme (e.g., Mock Score $= 999$).
- **String Standardization:**
  - Fixing typos and casing (e.g., converting "assasin" $\rightarrow$ "Assassin").

Step 2 - Feature Encoding

Translating Categories into Numbers

- **The Problem:** Computers cannot calculate distance or gradients using strings like "Tank" or "Mage".
- **Label Encoding:**
  - Used for the Target (Result).
  - e.g. Pass, Fail $\rightarrow$ 1, 0.
- **One-Hot Encoding:**
  - Used for Categorical Features (Role).
  - Creates binary columns (Dummy Variables) for each category to avoid artificial ranking.
  - e.g. Assassin $\rightarrow$ [1, 0, 0], Tank $\rightarrow$ [0, 1, 0].

Step 3 - Feature Scaling

Normalization and Standardization

- **Why Scale?** To prevent features with large magnitudes (e.g., Rank Score 0-2500) from dominating features with small magnitudes (e.g., Hours 0-20).
- **Standardization (Z-score):**
  - Formula: $z = \frac{x-\mu}{\sigma}$
  - Result: Data has a mean of 0 and a standard deviation of 1.
- **Min-Max Normalization:**
  - Formula: $x_{new} = \frac{x-x_{min}}{x_{max}-x_{min}}$
  - Result: All values are squeezed into the range $[0, 1]$.

The Preprocessing Result

Final Dataset Ready for Training

ID	Hours(z)	Score(z)	R_Assasin	R_Tank	R_Mage	R_Support	Result
S01	$+0.68$	$+0.78$	1	0	0	0	1
S02	0.00	-1.18	0	1	0	0	0
S03	$+1.87$	$+0.20$	0	0	1	0	1
S04	$+0.20$	$+1.18$	1	0	0	0	1
S05	-0.76	-0.39	0	1	0	0	0
S06	-0.52	$+0.98$	1	0	0	0	1
S07	-1.47	-1.57	0	0	0	1	0

The Machine Learning Pipeline

From Raw Data to Model Deployment

1. **Data Collection & Understanding**
   - **Input:** Gathering raw data (Database, CSV, Sensors, APIs).
   - **Action:** Explore the data to see distributions and identify "noise".
2. **Data Preprocessing (The 80% Work)**
   - **Cleaning:** Fix missing values, outliers, and typos (S02, S03, S04).
   - **Feature Engineering:** One-Hot Encoding and creating new features.
   - **Scaling:** Normalization/Standardization to bring features to the same scale.
3. **Data Splitting**
   - **Training Set (80%):** Used to "teach" the model.
   - **Test Set (20%):** Used to "evaluate" the model on unseen data.

The Machine Learning Pipeline

4. **Model Selection & Training**
   - Choosing the right algorithm (k-NN, K-Means, Linear Regression, etc.).
   - Feeding the **Training Set** into the algorithm.
5. **Evaluation & Tuning**
   - Testing the model using the **Test Set**.
   - Tuning **Hyperparameters** (e.g., choosing the optimal k via Elbow Method).
6. **Deployment & Monitoring**

Supervised Learning

Given a training set of N example input–output pairs Training set

$$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), ..., (x^{(N)}, y^{(N)})$$

each pair was generated by an unknown function $y = f(x)$,
discover a function h that approximates the true function f.

Hypothesis: function h

Hypothesis space: possible functions $\mathcal{H}$

$h \in \mathcal{H}$

With alternative vocabulary:

- h is a **model** of the data, drawn from a **model class** $\mathcal{H}$ or
- **function** drawn from a **function class**

Ground truth: output y_i - the true answer when using model to predict.

Supervised Learning

$$\mathcal{D} = \{(x^{(1)}, y^{(1)}) \dots (x^{(i)}, y^{(i)}) \dots (x^{(n)}, y^{(n)})\}$$

$$x^{(i)} \in \mathbb{R}^d \ , \ y^{(i)} \in \mathbb{R} : \text{label.}$$

sample $x^{(1)} \rightarrow$	$(x_1^{(1)} \dots x_j^{(1)} \dots x_d^{(1)})$	$y^{(1)} \leftarrow$ label
sample $x^{(i)} \rightarrow$	$(x_1^{(i)} \dots x_j^{(i)} \dots x_d^{(i)})$	$y^{(i)} \leftarrow$ label
...	...	...
sample $x^{(n)} \rightarrow$	$(x_1^{(n)} \dots x_j^{(n)} \dots x_d^{(n)})$	$y^{(n)} \leftarrow$ label

fruit	length	width	weight	label
fruit 1	165	38	172	Banana
fruit 2	218	39	230	Banana
fruit 3	76	80	145	Orange
fruit 4	145	35	150	Banana
fruit 5	90	88	160	Orange
...				
fruit n	...	...	...	...

Linear Regression

$$\mathcal{D} = \{(x^{(1)}, y^{(1)}) \dots (x^{(i)}, y^{(i)}) \dots (x^{(n)}, y^{(n)})\}$$

$$x^{(i)} \in \mathbb{R}^d \ , \ y^{(i)} \in \mathbb{R} : \text{label.}$$

sample $x^{(1)} \rightarrow$	$(x_1^{(1)} \dots x_j^{(1)} \dots x_d^{(1)})$	$y^{(1)} \leftarrow$ label
sample $x^{(i)} \rightarrow$	$(x_1^{(i)} \dots x_j^{(i)} \dots x_d^{(i)})$	$y^{(i)} \leftarrow$ label
...	...	...
sample $x^{(n)} \rightarrow$	$(x_1^{(n)} \dots x_j^{(n)} \dots x_d^{(n)})$	$y^{(n)} \leftarrow$ label

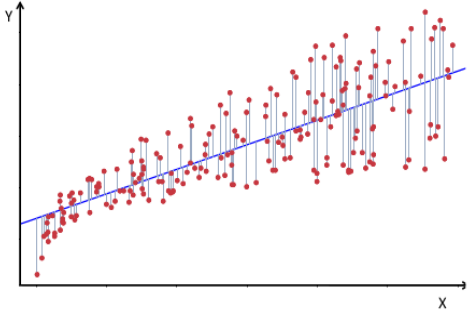
Task Learn a regression function

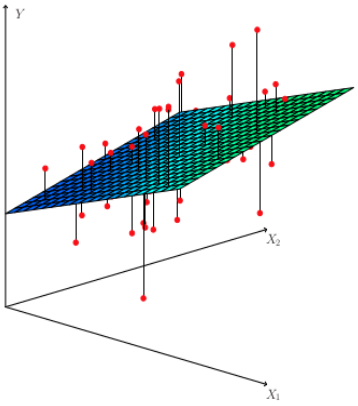
$$f : \mathbb{R}^d \rightarrow \mathbb{R}$$
$$f(x) = y$$

Linear Regression

A regression model is said to be linear if it is represented by a linear function.

Linear Regression





$d = 1$ **Line in** $\mathbb{R}^2$ $d = 2$ **Hyperplane in** $\mathbb{R}^3$

Linear Regression Model

$$\hat{y} = f(x) = \theta_0 + \sum_{j=1}^{d} \theta_j x_j \qquad \textbf{with } \theta_j \in \mathbb{R},\ j \in \{1, 2, ..., d\}$$

θ: parameters or coefficients or weights.

Learning the linear model $\rightarrow$ learning the θ

Estimation with Least squares

Least square loss

$$loss(y^{(i)}, f(x^{(i)})) = (y^{(i)} - f(x^{(i)}))^2 = (y^{(i)} - \hat{y}^{(i)})^2$$

Minimize the loss over all samples

$$\mathcal{L} = \frac{1}{2n} \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})^2$$

Univariate Linear Regression

Model with one feature $d = 1$

$$\hat{y} = f(x) = \theta_0 + \theta_1 x$$

Loss function

$$\mathcal{L} = \frac{1}{2n} \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})^2 \quad , \qquad \mathcal{L}(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^{n} (y^{(i)} - \theta_0 - \theta_1 x^{(i)})^2$$

Minimize $\mathcal{L}(\theta_0, \theta_1) \rightarrow$ **find** θ_0 , θ_1

$$\underset{\{\theta_0, \theta_1\}}{argmin}\ \mathcal{L}(\theta_0, \theta_1) = \underset{\{\theta_0, \theta_1\}}{argmin}\ \frac{1}{2n} \sum_{i=1}^{n} (y^{(i)} - \theta_0 - \theta_1 x^{(i)})^2$$

Find θ_0 , θ_1 $\qquad \frac{\partial \mathcal{L}}{\partial \theta_0} = 0 \qquad \frac{\partial \mathcal{L}}{\partial \theta_1} = 0$

$$\frac{\partial \mathcal{L}}{\partial \theta_0} = 2 \times \frac{1}{2n}\sum_{i=1}^{n}(y^{(i)} - \theta_0 - \theta_1 x^{(i)}) \ \times \ \frac{\partial}{\partial \theta_0}(y^{(i)} - \theta_0 - \theta_1 x^{(i)})$$

$$\frac{\partial \mathcal{L}}{\partial \theta_0} = \frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - \theta_0 - \theta_1 x^{(i)}) \ \times \ (-1) = 0$$

$$\theta_0 = \frac{1}{n}\sum_{i=1}^{n} y^{(i)} - \theta_1 \frac{1}{n}\sum_{i=1}^{n} x^{(i)}$$

$$\underset{\{\theta_0,\theta_1\}}{argmin}\mathcal{L}(\theta_0,\theta_1)$$

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = 2 \times \frac{1}{2n}\sum_{i=1}^{n}(y^{(i)} - \theta_0 - \theta_1 x^{(i)}) \times \frac{\partial}{\partial \theta_1}(y^{(i)} - \theta_0 - \theta_1 x^{(i)})$$

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - \theta_0 - \theta_1 x^{(i)}) \times (-x^{(i)}) = 0$$

$$\theta_1 \sum_{i=1}^{n} {x^{(i)}}^2 = \sum_{i=1}^{n} x^{(i)} y^{(i)} - \sum_{i=1}^{n} \theta_0 x^{(i)}$$

Plugging θ_0 in θ_1

$$\theta_1 = \frac{\sum_{i=1}^{n} x^{(i)} y^{(i)} - \frac{1}{n}\sum_{i=1}^{n} x^{(i)} \sum_{i=1}^{n} y^{(i)}}{\sum_{i=1}^{n} {x^{(i)}}^2 - \frac{1}{n}\sum_{i=1}^{n} x^{(i)} \sum_{i=1}^{n} x^{(i)}}$$

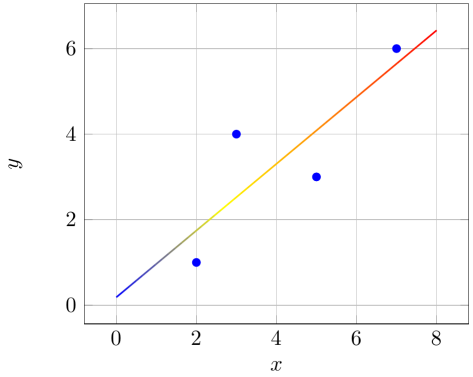
e.g.

– **Training set**

$$\mathcal{D} = \{(2,1), (3,4), (5,3), (7,6)\}$$

– **Linear Regression Model**

$$f(\theta, x) = \theta_0 + \theta_1 x$$



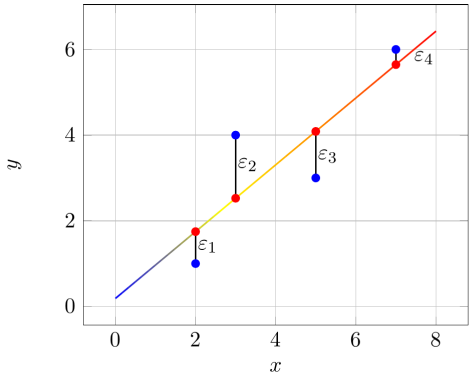
e.g.

$$\epsilon_i = (y_i, f(\theta, x_i))$$

$$\mathcal{L}_i(\theta_0, \theta_1) = {\epsilon_i}^2$$

$$= (y^{(i)} - f(\theta, x^{(i)}))^2$$

$$= (y^{(i)} - (\theta_0 + \theta_1 x^{(i)}))^2$$



Multivariate Linear Regression

Model with many feature $d > 1$

$$f(x) = \theta_0 + \sum_{j=1}^{d} \theta_j x_j$$

Minimize

$$\mathcal{L} = \frac{1}{2n} \sum_{i=1}^{n} (y^{(i)} - \theta_0 - \sum_{j=1}^{d} \theta_j x_j^{(i)})^2$$

Matrix with n sample, d feature

X Matrix $n \times (d+1)$

y Label vector

Θ Weights vector

$$X = \begin{pmatrix} 1 & x_1^{(1)} & \cdots & x_j^{(1)} & \cdots & x_d^{(1)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(i)} & \cdots & x_j^{(i)} & \cdots & x_d^{(i)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(n)} & \cdots & x_j^{(n)} & \cdots & x_d^{(n)} \end{pmatrix}$$

$$y = \begin{pmatrix} y^{(1)} \\ \vdots \\ y^{(i)} \\ \vdots \\ y^{(n)} \end{pmatrix} \qquad \Theta = \begin{pmatrix} \theta_0 \\ \vdots \\ \theta_j \\ \vdots \\ \theta_d \end{pmatrix}$$

Normal Equation

$$\mathcal{L}(\Theta) = \frac{1}{2n}\left\|(y - X\Theta)\right\|^2$$

$$\mathcal{L}(\Theta) = \frac{1}{2n}(y - X\Theta)^T(y - X\Theta)$$

$$\frac{\partial \mathcal{L}}{\partial \Theta} = -\frac{1}{n}X^T(y - X\Theta)$$

We have

$$\frac{\partial^2 \mathcal{L}}{\partial \Theta} = \frac{1}{n}X^T X \ : \ \textit{positive}$$

So that is a minimum when

$$X^T(y - X\Theta) = 0$$

The unique solution

$$\Theta = (X^T X)^{-1} X^T y$$

e.g.

- $X = \{(1, 0), (2, 3.50), (2.50, 4), (3.50, 4), (3, 1), (5, 1.50)\}$;
- $y = \{1.50, 2, 3.50, 2.50, 1, 1.50\}$

1. **Initial Matrix**

   Add column $x_0 = 1$ (bias):

$$X = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 3.5 \\ 1 & 2.5 & 4 \\ 1 & 3.5 & 4 \\ 1 & 3 & 1 \\ 1 & 5 & 1.5 \end{bmatrix}, \quad y = \begin{bmatrix} 1.5 \\ 2 \\ 3.5 \\ 2.5 \\ 1 \\ 1.5 \end{bmatrix}$$

1. Gram Matrix $X^T X$ and vector $X^T y$:

i	x_{i1}	x_{i2}	y_i	x_{i1}^2	x_{i2}^2	$x_{i1}x_{i2}$
1	1.0	0.0	1.5	1.0	0.0	0.0
2	2.0	3.5	2.0	4.0	12.25	7.0
3	2.5	4.0	3.5	6.25	16.0	10.0
4	3.5	4.0	2.5	12.25	16.0	14.0
5	3.0	1.0	1.0	9.0	1.0	3.0
6	5.0	1.5	1.5	25.0	2.25	7.5
$\sum$	**17.0**	**14.0**	**12.0**	**57.5**	**48.5**	**45.75**

2. $(X^T X)\Theta = X^T y$

$$X^T X = \begin{bmatrix} 6 & 17 & 14 \\ 17 & 57.5 & 45.75 \\ 14 & 45.75 & 48.5 \end{bmatrix}, \quad X^T y = \begin{bmatrix} 12 \\ 33.5 \\ 34.25 \end{bmatrix}$$

$$\begin{cases} 6\theta_0 + 17\theta_1 + 14\theta_2 = 12 \\ 17\theta_0 + 57.5\theta_1 + 45.75\theta_2 = 33.5 \\ 14\theta_0 + 45.75\theta_1 + 48.5\theta_2 = 34.25 \end{cases}$$

3. Gauss, get vector:

$$\Theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix} \approx \begin{bmatrix} 0.7251 \\ 0.0544 \\ 0.4805 \end{bmatrix}$$

$$\hat{y} = 0.7251 + 0.0544x_1 + 0.4805x_2$$

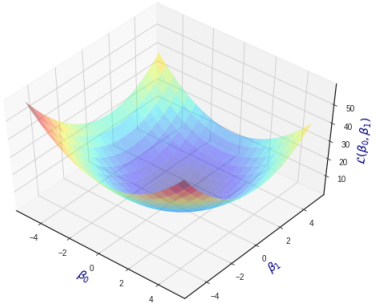
Gradient descent algo.

The Philosophy:

$$\mathcal{D} \;=\; \{\; (x^{(1)}, y^{(1)}) \dots (x^{(i)}, y^{(i)}) \dots (x^{(n)}, y^{(n)}) \;\}$$
$$x^{(i)} \in \mathbb{R}^d \;,\; y^{(i)} \in \mathbb{R}.$$

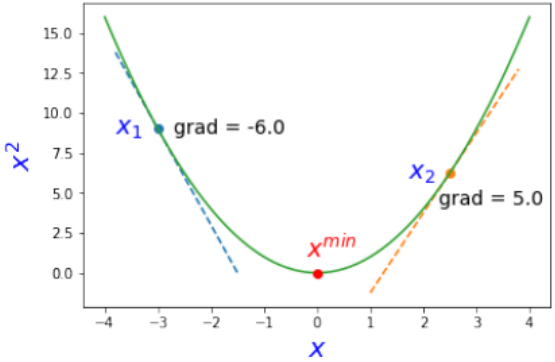
Loss Function $\mathcal{L}(\cdot)$

- n
- $x^{(i)} \;\in\; \mathbb{R}^d$
- $\mathcal{L}$: k order



Gradient Descent

$$f(x) = x^2 \quad , \quad f'(x) = 2x$$



Taylor Series Expansion

1. $f(x+\epsilon) = f(x) + \epsilon f^{'}(x) + \frac{\epsilon^2}{2!} f^{''}(x) + \ldots$
2. $f(x+\epsilon) \approx f(x) + \epsilon f^{'}(x)$
3. $\eta > 0\ ,\ \epsilon = -\eta f^{'}(x)$
4. $f(x - \eta f^{'}(x)) \approx f(x) - \eta f^{'}(x)^2$

$$f(\ x\ -\ \eta f^{'}(x)\ ) \leq f(x)$$

Gradient descent algo.

1. x_0
2. $x \leftarrow x_0$
3. η : learning rate
4. ϵ : error convergence
5. while $|f'(x)| \geq \epsilon$ do

   $x \leftarrow x - \eta f'(x)$

Batch Gradient Descent

The Philosophy

1. θ_0
2. $\theta \leftarrow \theta_0$
3. η : learning rate
4. ϵ : error convergence
5. while $\|\nabla \mathcal{L}(\theta)\|_2 \geq \epsilon$ do

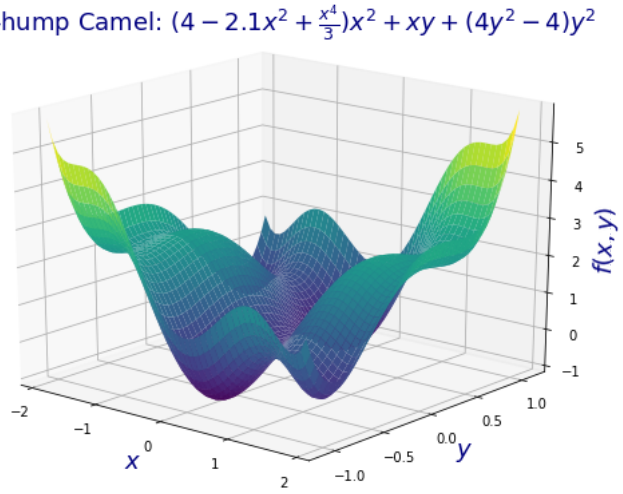
$$\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$$

Stochastic Gradient Descent

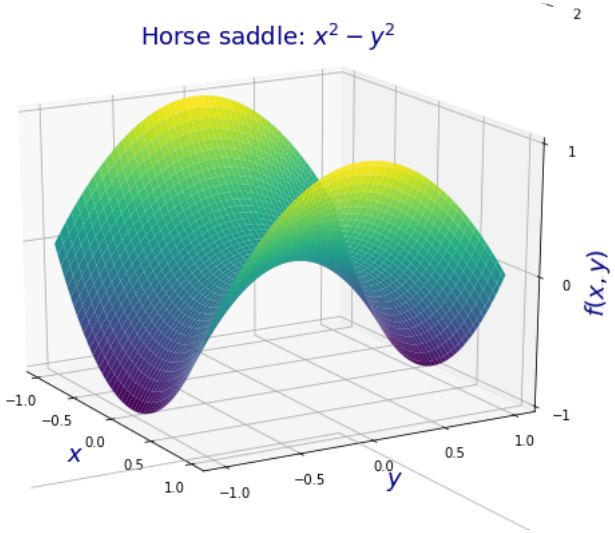
$$\mathcal{L}(\theta) = \frac{1}{2n}\sum_{i=1}^{n}(\theta^T X^{(i)} - y^{(i)})^2$$

$$\mathcal{L}(\theta, X^{(k)}) = \tfrac{1}{2}\ (\theta^T X^{(k)} - y^{(k)})^2$$

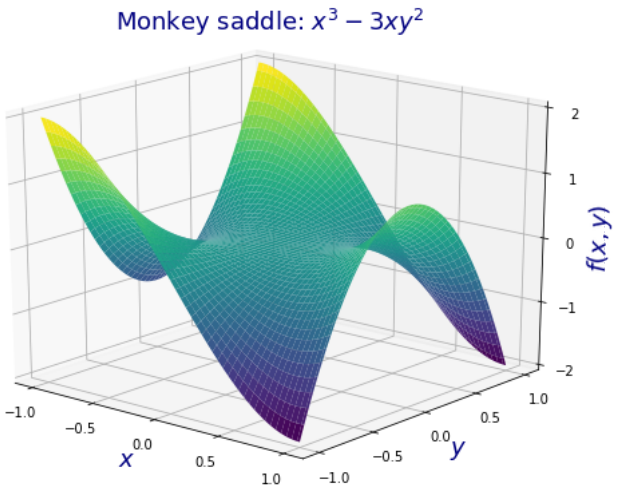
Non-Convex Function



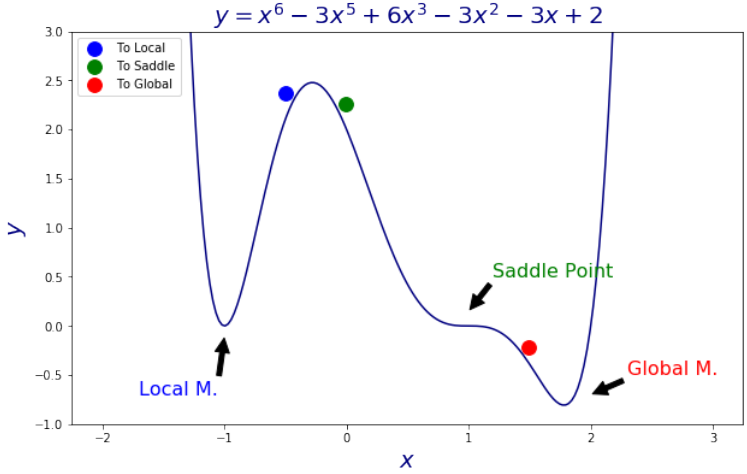
Non-Convex Function



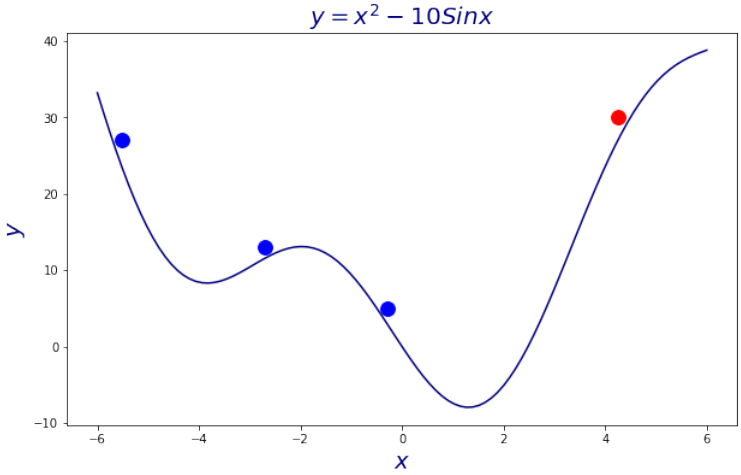
Non-Convex Function



Non-Convex Function



Momentum



Momentum Algorithm

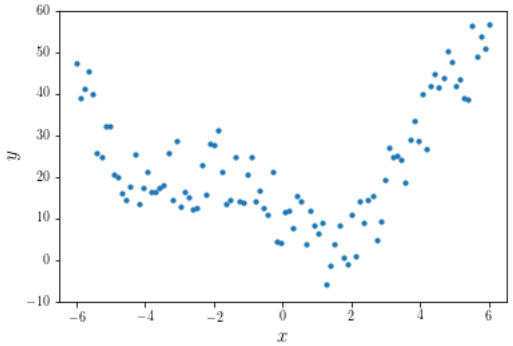
1. θ_0 ; $\theta \leftarrow \theta_0$
2. η ; ϵ
3. v_0 ; $v \leftarrow v_0$
4. γ : momentum
5. while $\|\nabla \mathcal{L}(\theta)\|_2 \geq \epsilon$ do

$$v \leftarrow \gamma v + \eta \nabla \mathcal{L}(\theta)$$
$$\theta \leftarrow \theta - v$$

Polynomial Regression

Sample with $d = 1$

$$\mathcal{D} = \{(x^{(1)}, y^{(1)}) \dots (x^{(i)}, y^{(i)}) \dots (x^{(n)}, y^{(n)})\}$$



Polynomial Regression

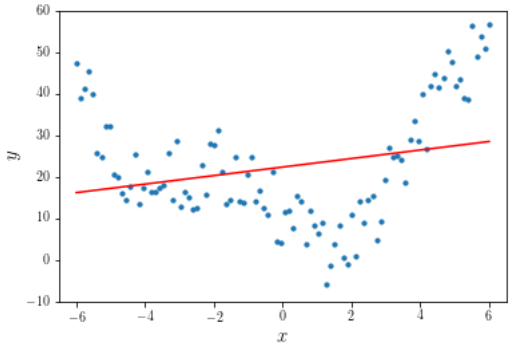
Samples

$$\mathcal{D} = \{(x^{(1)}, y^{(1)}) \dots (x^{(i)}, y^{(i)}) \dots (x^{(n)}, y^{(n)})\}$$

$$x^{(i)} \in \mathbb{R} \ , \ y^{(i)} \in \mathbb{R}$$

Simple Linear Regression

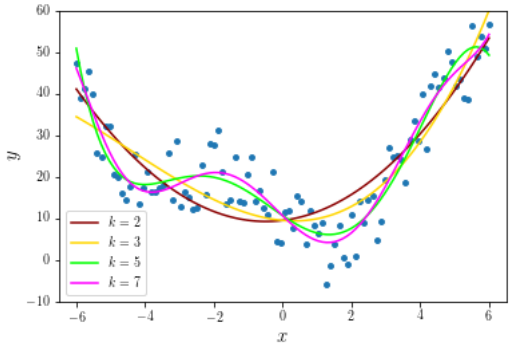
$$f(x) = \theta_0 + \theta_1 x$$



Polynomial Regression

Polynominal k-order

$$f(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \ldots + \theta_k x^k$$



Polynomial Regression

Solving the problem Polynomial Regression is a model used when the response variable is non-linear.

$$f(x) = \theta_0 + \sum_{i=1}^{k} \theta_i x^i$$

Convert to Multivariate Linear Regression

with $d = k$ feature.

$$x_i = x^i$$

$$f(x) = \theta_0 + \sum_{i=1}^{k} \theta_i x_i$$

Multivariate Polynomial Regression

So far we have

- Simple Linear Regression $f(x) = \theta_0 + \theta_1 x$
- Multivariate Linear Regression $f(x) = \theta_0 + \sum_{i=1}^{d} \theta_i x_i$
- Polynomial Regression $f(x) = \theta_0 + \sum_{i=1}^{k} \theta_i x_i^i$

Multivariate Polynomial Regression

- **Second Order**

$$f(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_{11} x_1^2 + \theta_{22} x_2^2 + \theta_{12} x_1 x_2$$

- **General Order** Big issue.

To sole this issue, it can mapped to a higher order space of independent variables called as the feature space (eg. Kernel method ...)

Logistic Regression for Classification

The Philosophy

Logistic model (Logit model): Probability of a class label in dataset.

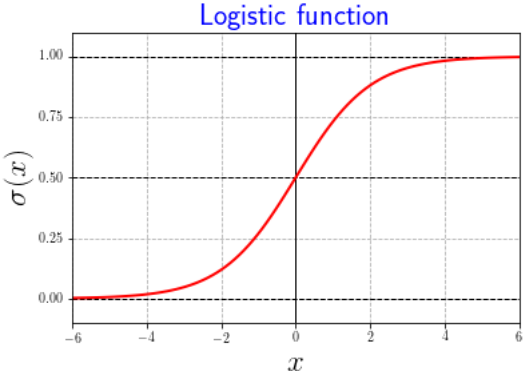
Logistic function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$x \to +\infty \ , \ \sigma(x) \to 1$

$x \to -\infty \ , \ \sigma(x) \to 0$

– *Continuous, has a first derivative.*



Probability return

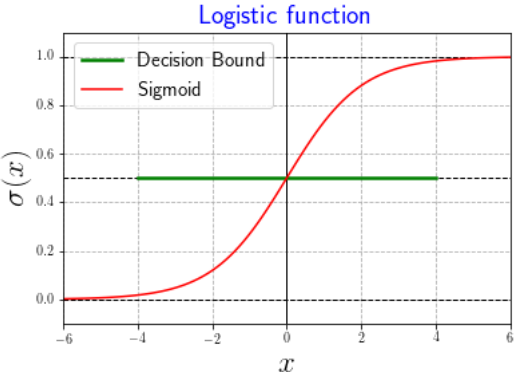
Logistic return probability score between 0 and 1

$$Prob = \sigma(x) = \frac{1}{1 + e^{-x}}$$

$Prob \in (0, 1)$

$Prob \geq 0.5\ ,\ class = 1$

$Prob < 0.5\ ,\ class = 0$



Logistic Regression Model

$$\mathcal{D} = \left\{ (x^{(i)}, y^{(i)}) \right\}_1^n, \quad x^{(i)} \in \mathbb{R}^d, \quad y^{(i)} \in \{0, 1\}$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} \in \mathbb{R}^{d+1} \qquad x = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \end{bmatrix} \in \mathbb{R}^{d+1}$$

Model

$$f(x) = \sigma(\theta.x) = \frac{1}{1 + e^{-\theta.x}}$$

dot product $\theta.x \equiv \sum_j \theta_j x_j$

Logistic Regression Model

e.g.

$x = (x_1, x_2)$, $y = \{0, 1\}$. imagine we know θ.

$z = \theta.x = \theta_0 + \theta_1 x_1 + \theta_2 x_2$

$$Prob(class = 1) = \frac{1}{1 + e^{-z}}$$

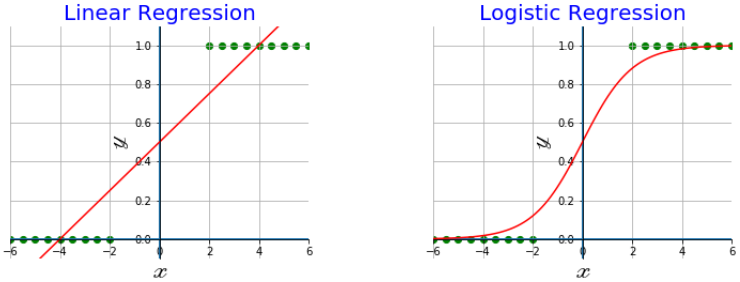
Decision boundary = .5

e.g. $Prob(class = 1)$ return .4, only have 40% chance "class 1" or this observation as "class 0".

Linear Regression Vs. Logistic Regression

- Linear Regression: *Continuous output.*
- Logistic Regression: *Constant output.*
- Linear Regression: *Using Ordinary Least Squares (OLS).*
- Logistic Regression: *Using Maximum Likelihood Estimation (MLE).*

Consider dataset which two class $\{0, 1\}$



Logistic Regression Model

Model

$$f(x) = \sigma(\theta.x) = \frac{1}{1 + e^{-\theta.x}}$$

$f(x) \in (0, 1)$

$f(x)$: Probability.

Probability return by Model

$Prob(y = 1 \mid x; \theta) = f(x)$

$Prob(y = 0 \mid x; \theta) = 1 - f(x)$

Model base on probability

$$Prob(y \mid x; \theta) = f(x)^y \ (1 - f(x))^{1-y}$$

The Likelihood

The Philosophy

One sample i

$$Prob(y^{(i)} \mid x^{(i)}; \theta) = f(x^{(i)})^{y^{(i)}} \ (1 - f(x^{(i)}))^{1-y^{(i)}}$$

Loss function $Loss(\theta)$ for all sample.

n training samples were generated independently.

Likelihood

$$Loss(\theta) = \prod_{i=1}^{n} Prob(y^{(i)} \mid x^{(i)}; \theta)$$

$$Loss(\theta) = \prod_{i=1}^{n} f(x^{(i)})^{y^{(i)}} \ (1 - f(x^{(i)}))^{1-y^{(i)}}$$

Logarithm of Likelihood

- Logarithm turns a product into a sum.
- It avoid the issue of small number(typically for probability).

$$
\begin{aligned}
\mathcal{L}(\theta) &= log\ Loss(\theta) \\
&= log \prod_{i=1}^{n} f(x^{(i)})^{y^{(i)}}\ (1 - f(x^{(i)}))^{1-y^{(i)}} \\
&= \sum_{i=1}^{n} log \left\{ f(x^{(i)})^{y^{(i)}}\ (1 - f(x^{(i)}))^{1-y^{(i)}} \right\} \\
&= \sum_{i=1}^{n} \left\{ log f(x^{(i)})^{y^{(i)}}\ +\ log(1 - f(x^{(i)}))^{1-y^{(i)}} \right\} \\
&= \sum_{i=1}^{n} \left\{ y^{(i)}\ log f(x^{(i)})\ +\ (1 - y^{(i)})\ log(1 - f(x^{(i)})) \right\}
\end{aligned}
$$

Maximization into a Minimization

Maximize the Likelihood

$$\mathcal{L}(\theta) = \sum_{i=1}^{n} \Big\{ y^{(i)} \; log f(x^{(i)}) \; + \; (1 - y^{(i)}) \; log(1 - f(x^{(i)})) \Big\}$$

– Negative log-likelihood (NLL).

– Use gradient descent algo.

So we minimize the negative $\mathcal{L}(\theta)$ with

$$\mathcal{L}(\theta) = - \sum_{i=1}^{n} \Big\{ y^{(i)} \; log f(x^{(i)}) \; + \; (1 - y^{(i)}) \; log(1 - f(x^{(i)})) \Big\}$$

Minimize the negative Likelihood

$$\mathcal{L}(\theta) = \sum_{i=1}^{n} \left\{ -y^{(i)} \; log f(x^{(i)}) \; - \; (1 - y^{(i)}) \; log(1 - f(x^{(i)})) \right\}$$

$$\frac{\partial}{\partial \theta_j} \mathcal{L}(\theta) = \sum_{i=1}^{n} \left\{ -y^{(i)} \frac{1}{\sigma(\theta.x^{(i)})} + (1 - y^{(i)}) \frac{1}{1 - \sigma(\theta.x^{(i)})} \right\} \times \frac{\partial}{\partial \theta_j} \sigma(\theta.x^{(i)})$$

since $\frac{\partial}{\partial \theta_j} \sigma(\theta.x^{(i)}) = \sigma(\theta.x^{(i)})(1 - \sigma(\theta.x^{(i)})) \frac{\partial}{\partial \theta_j} \theta.x^{(i)}$, so

$$= \sum_{i=1}^{n} \left\{ \sigma(\theta.x^{(i)}) - y^{(i)} \right\} \frac{\partial}{\partial \theta_j} \theta.x^{(i)}$$

Since $\theta.x^{(i)} = \theta_0 + \sum_{j=1}^{d} \theta_j x_j^{(i)} \implies \frac{\partial}{\partial \theta_j} \theta.x^{(i)} = x_j^{(i)}$, so

$$= \sum_{i=1}^{n} (f(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Types of Logistic Regression

- **Binary Logistic Regression** Binary class, e.g. Spam or Not Spam, Cancer or No Cancer.
- **Multinomial Logistic Regression** Many class, e.g. predicting the type of Wine.
- **Ordinal Logistic Regression** Many ordinal class, e.g. restaurant or product rating from 1 to 5.

Binary Classification

The Philosophy

$$\mathcal{D} = \left\{ \left(x^{(i)}, y^{(i)} \right) \right\}_1^n , \quad x^{(i)} \in \mathbb{R}^d \quad , \quad y^{(i)} : \textbf{discrete}.$$

Binary Classification

$$y^{(i)} \in \{0, 1\} \; or \; y^{(i)} \in \{-1, 1\}$$

e.g.

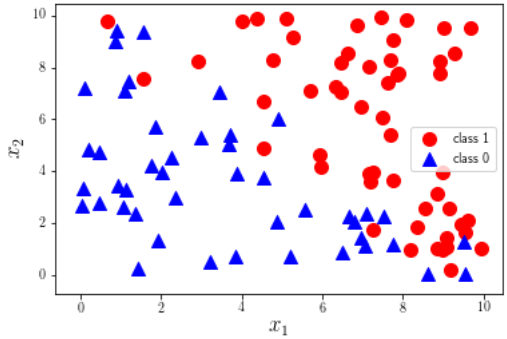
$$\left\{ \left(x^{(i)}, y^{(i)} \right) \right\}_1^n, \; x^{(i)} \in \mathbb{R}^2, \; y^{(i)} \in \{0, 1\}$$

x_1	x_2	y
8.625	0.058	0
3.828	0.723	0
7.150	3.899	1
6.477	8.198	1
1.922	1.331	0

e.g. Binary Classification

e.g. chart of dataset above

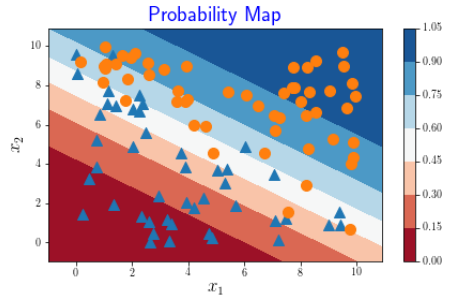
$$\left\{(x^{(i)}, y^{(i)})\right\}_1^n,\ x^{(i)} \in \mathbb{R}^2,\ \ y^{(i)} \in \{0, 1\}$$

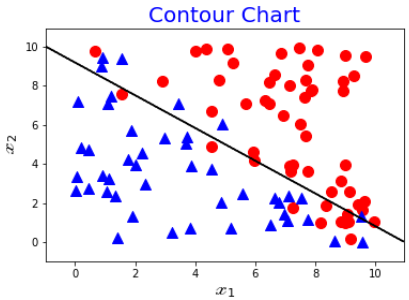


e.g. Logistic Regression

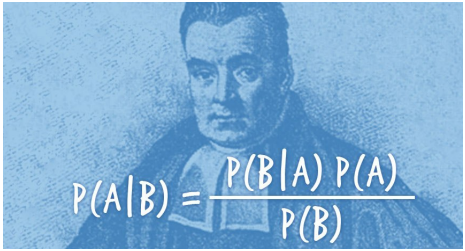
– This model predict probability of two class:

– Contour chart base on probability.





Bayes's Theorem



The Philosophy:

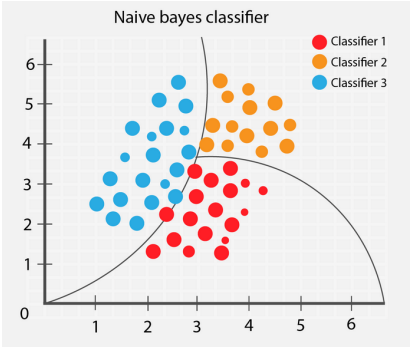
$p(A)$: *Prior*
$p(A|B)$: *Posterior*
$p(B|A)$: *Likelihood*
$p(B)$: *Evidence*

$$Posterior = \frac{Likelihood \times Prior}{Evidence}$$

$$\boxed{P(class/data) = \frac{P(data/class) \times P(class)}{P(data)}}$$

Naïve Bayes Classifier

$$P(class/data) = \frac{P(data/class) \times P(class)}{P(data)}$$



Review: Joint Probability

Tossing two coins: Independent

– A: Means the first coin lands face up
– B: Means the second coin lands face up

- $p(A) = p(B) = 0.5$
- $p(A \; and \; B) = p(A) \; p(B) = 0.25$
- $p(B|A) = p(B)$

Events are not independent

– A: Mean it rains today
– B: Means it rains tomorrow

- It rained today, it more likely rain tomorrow
- $p(B|A) > p(B)$
- $p(A \; and \; B) = p(A) \; p(B|A)$

e.g. Cookie problem

- *Suppose there are two bowls of cookies*
  - Bowl 1:
    30 vanilla
    10 chocolate
  - Bowl 2:
    20 vanilla
    20 chocolate
- *Now suppose you choose*
  - One of the bowls at random
  - Without looking, select a cookie at random

This is a conditional probability

$$p(Bowl\ 1 \mid vanilla)$$

$p(vanila \mid Bowl\ 1) = 3/4$
$\neq p(Bowl\ 1 \mid vanilla)$

Bayes's Theorem

Any events A and B

- $p(A \text{ and } B) = p(B \text{ and } A)$
- $p(A \text{ and } B) = p(A)\ p(B|A)$
- $p(B \text{ and } A) = p(B)\ p(A|B)$

$\Longrightarrow\ p(B)\ p(A|B) = p(A)\ p(B|A)$

Bayes's Theorem

$$p(A|B) = \frac{p(B|A)\ p(A)}{p(B)}$$

Cookie problem

+ $B1$: Hypothesis of cookie came from Bowl 1
+ V: Vanilla cookie

$$p(B1|V) = \frac{p(V|B1)\ p(B1)}{p(V)}$$

e.g. Cookie problem

$$p(B1|V) = \frac{p(V|B1)\ p(B1)}{p(V)}$$

$p(B1)$: Probability chose Bowl 1

$$p(B1) = 1/2$$

$p(V|B1)$: Probability vanilla cookie from Bowl 1

$$p(V|B1) = 3/4$$

$p(V)$: Probability vanilla cookie from either bowl

$$p(V) = 5/8$$

$$p(B1|V) = \frac{(3/4)\ (1/2)}{(5/8)} = 3/5$$

e.g. Elderly Fall and Death

- Elderly person is died: 10%
- Elderly people falling: 5%
- All elderly people die, they had fall: 7%

Probability that elderly people die when they fall?

$$P(Die|Fall) = \frac{P(Fall|Die) \times P(Die)}{P(Fall)}$$

$P(Die) = 0.10$
$P(Fall) = 0.05$
$P(Fall|Die) = 0.07$

$$P(Die|Fall) = \frac{0.07 \times 0.10}{0.05}$$

$$P(Die|Fall) = \mathbf{0.14}$$

- If an elderly person falls
- There is a 14% probability that they will die from the fall

e.g. Email and Spam Detection

- Email receive is spam: 2%
- Spam detector accuracy: 99%
- When an email is not spam, it will mark it as spam: 0.1%

Probability that fact spam email in spam folder?

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

$$P(B) = P(B|A) \times P(A) \ + \ P(B|not\ A) \times P(not\ A)$$

e.g. Email and Spam Detection

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

$$P(B) = P(B|A) \times P(A) \ + \ P(B|not\ A) \times P(not\ A)$$

$P(A|B)$ $= P(Spam|Detected) = ?$
$P(B|A)$ $= P(Detected|Spam) = 0.99$
$P(A)$ $= P(Spam) = 0.02$
$P(not\ A)$ $= 1 - P(Spam) = 0.98$
$P(B|not\ A)$ $= P(Detected|not\ Spam) = 0.001$

$$P(Spam|Detected) = \frac{0.99 \times 0.02}{0.99 \times 0.02 + 0.001 \times 0.98} = 0.952$$

– *Probability fact spam email in spam folder, is* **95.2%**.

e.g. Liars and Lie Detectors

- Lie Detector test persons: *if positive result* $\Longrightarrow$ *they are lying*.
- People are tested:
  - Telling the truth: 98%
  - Liars: 2%
- Liar people is tested: positive result 72%
- When the machine says they are *not lying*: this is true 97%

Probability that they are indeed lying?

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

$$P(B) = P(B|A) \times P(A) \ + \ P(B|\textit{not } A) \times P(\textit{not } A)$$

e.g. Liars and Lie Detectors

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

$$P(B) = P(B|A) \times P(A) \ + \ P(B|not\ A) \times P(not\ A)$$

$P(A|B)$ $= P(Lying|Positive) = ?$
$P(B|A)$ $= P(Positive|Lying) = 0.72$
$P(A)$ $= P(Lying) = 0.02$
$P(not\ A)$ $= 1 - P(Lying) = 0.98$
$P(not\ B|not\ A)$ $= P(not\ Positive|not\ Lying) = 0.097$
$P(B|not\ A)$ $= 1 - P(not\ B|not\ A) = 1 - 0.097 = 0.03$

$$P(Lying|Positive) = \frac{0.72 \times 0.02}{0.72 \times 0.02 + 0.03 \times 0.98} = 0.328$$

– *Probability fact lying when positive test result, is* **32.8%**.

e.g. Medical test

– People have a certain genetic defect: 1% – Testing to genetic defect (true positives): 90% – Testing have false positives: 9.6%

Probability genetic defect when get a positive test result?

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B|A) \times P(A) + P(B|not\ A) \times P(not\ A)}$$

$P(A|B)$ = $P(GeneticDefect|Positive) = ?$
$P(B|A)$ = $P(Positive|GeneticDefect) = 0.9$
$P(A)$ = $P(GeneticDefect) = 0.01$
$P(not\ A)$ = 1 - $P(GeneticDefect) = 0.99$
$P(B|not\ A)$ = $P(Positive|not\ GeneticDefect) = 0.096$

$$P(GeneticDefect|Positive) = \frac{0.9 \times 0.01}{0.9 \times 0.01 + 0.096 \times 0.99} = 0.0865$$

– *Probability faulty gene on positive result, is* **8.65%**.

e.g. Breast Cancer test

– Women over 50 have breast cancer: 1% – Women who have breast cancer, had positive result test: 90% – Women will have false positives: 8%

Probability woman has cancer if she has a positive result?

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B|A) \times P(A) + P(B|not\ A) \times P(not\ A)}$$

$P(A|B) = P(Cancer|Positive) = ?$
$P(B|A) = P(Positive|Cancer) = 0.9$
$P(A) = P(Cancer) = 0.01$
$P(not\ A) = 1 - P(Cancer) = 0.99$
$P(B|not\ A) = P(Positive|not\ Cancer) = 0.08$

$$P(Cancer|Positive) = \frac{0.9 \times 0.01}{0.9 \times 0.01 + 0.08 \times 0.99} = 0.10$$

– *Probability cancer, given a positive test result, is* **10%**.

Naïve Bayes Classifier

Given dataset $D = \{ (x^{(i)}, y^{(i)}) \}_{i=1}^{n}$, $x^{(i)} \in \mathbb{R}^d$, $y^{(i)} \in C$

given (x, y) , $x \in \mathbb{R}^d$, find $y \in C$, with maximum $p(y|x)$

$$p(y|x) = \frac{p(y)\ p(x|y)}{p(x)}$$

- $p(y)$: Prior probability of class y in dataset D
  (we have C class)
- $p(y|x)$: Posterior probability of class y given **one** evidence
  $x = (x_1, x_2, ..., x_d)$
- $p(x|y)$: Likelihood which is the probability of evidence given
  class $y \in C$
- $p(x)$: Prior probability of **one** evidence in D
  $x = (x_1, x_2, ..., x_d)$

Naïve Bayes Model

$$p(y|x_1, x_2, ..., x_d) = \frac{p(y)\ p(x_1, x_2, ..., x_d|y)}{p(x_1, x_2, ..., x_d)}$$

$(x_1, x_2, ..., x_d)$ are stochastically independent, given y:

$$p(x_1, x_2, ..., x_d|y) = p(x_1|y)\ p(x_2|y)\ ...\ p(x_d|y)$$

$$p(y|x_1, x_2, ..., x_d) = \frac{p(y) \prod_{i=1}^{d} p(x_i|y)}{p(x_1, x_2, ..., x_d)}$$

$p(x_1, x_2, ..., x_d)$ is constant given the Dataset,

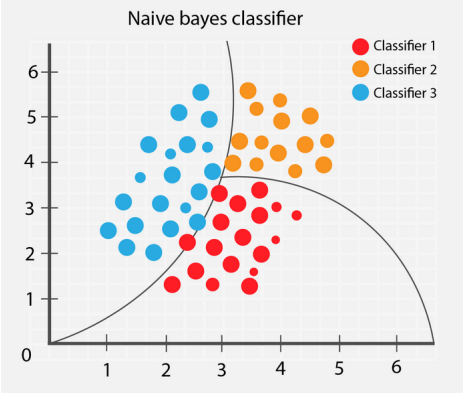
$$p(y|x_1, x_2, ..., x_d) \ \propto\ p(y) \prod_{i=1}^{d} p(x_i|y)$$

Algorithm

$$\hat{y} = \underset{y\ \in\ C}{argmax}\ p(y) \prod_{i=1}^{d} p(x_i|y)$$

Naïve Bayes Classifier algorithm

$$\hat{y} = \underset{y \in C}{argmax}\ p(y) \prod_{i=1}^{d} p(x_i|y)$$



e.g.

Outlook	Temperature	Huminity	Windy	Play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

Give a new instance x :

Outlook	Temperature	Huminity	Windy	Play
sunny	*cool*	*high*	*true*	**?**

y = yes

$p(yes) = \frac{9}{14}$
$p(sunny|yes) = \frac{2}{9}$
$p(cool|yes) = \frac{3}{9}$
$p(high|yes) = \frac{3}{9}$
$p(true|yes) = \frac{3}{9}$
$p(y = yes) = \frac{9}{14} \times \frac{2}{9} \times \frac{3}{9} \times \frac{3}{9} \times \frac{3}{9}$
$= 0.00529$

y = no

$p(no) = \frac{5}{14}$
$p(sunny|no) = \frac{3}{5}$
$p(cool|no) = \frac{1}{5}$
$p(high|no) = \frac{4}{5}$
$p(true|no) = \frac{3}{5}$
$p(y = no) = \frac{5}{14} \times \frac{3}{5} \times \frac{1}{5} \times \frac{4}{5} \times \frac{3}{5}$
$= 0.02057$

$p(y = no) > p(y = yes)$. Predict result:

Outlook	Temperature	Huminity	Windy	Play
sunny	*cool*	*high*	*true*	**no**

Probability Distribution

– A probability distribution is a statistical function that describes all the possible values that a random variable can take within a given range.

– Probability distribution depends on factors:

Mean expected value of the distribution.

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Variance describes the spread of observation from the mean.

Population: $\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x-\mu)^2$ **Sample:** $s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x-\mu)^2$

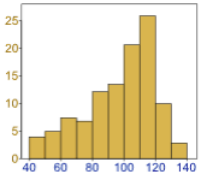
Standard deviation describes the normalized spread of observations from the mean.

σ

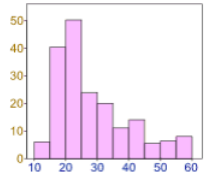
Probability Distribution

Data can be "distributed" (spread out) in different ways

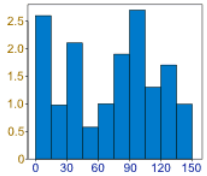
It can be spread out more on the left



Or more on the right



Or it can be all jumbled up



Normal Distribution

Many cases, data tends to be around a central value

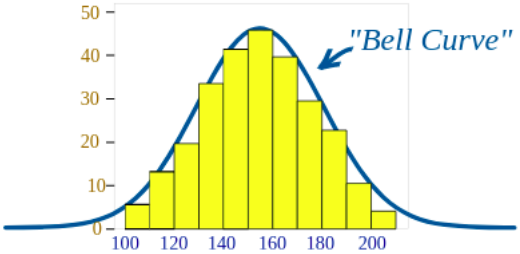
σ : Standard deviation of x

μ : Mean of x

$\pi \approx 3.14159...$

$e \approx 2.71828...$

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$



Normal Distribution

Many things closely follow a Normal Distribution:

- Heights of people
- Size of things produced by machines
- Errors in measurements
- Blood pressure
- Marks on a test

Normal Distribution

The 68 - 95 - 99.7 Rule

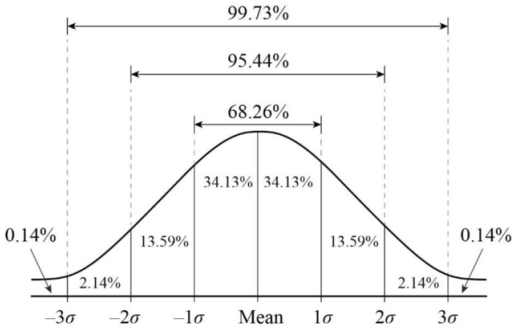
– Mean μ

– Standard deviation σ

– Approximately 68% observations fall within σ of the mean μ.

– Approximately 95% observations fall within 2σ of σ.

– Approximately 99.7% observations fall within 3σ of σ.



Gaussian Naïve Bayes

If features are continuous values, the likelihood of the features is assumed to be Gaussian:

$$P(x_i|y) = \frac{1}{\sigma_y \sqrt{2\pi}} e^{-\frac{(x_i-\mu_y)^2}{2\sigma_y^2}}$$

Algorithm

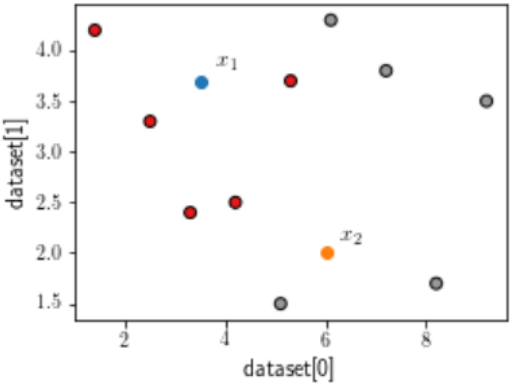
$$\hat{y} = \underset{y \in C}{argmax}\ p(y) \prod_{i=1}^{d} p(x_i|y)$$

Very easy!

e.g. Gaussian Naïve Bayes Classifier

dataset:

$$
\begin{aligned}
&(3.3\ ,\ 2.4\ ,\ 0),\\
&(2.5\ ,\ 3.3\ ,\ 0),\\
&(4.2\ ,\ 2.5\ ,\ 0),\\
&(1.4\ ,\ 4.2\ ,\ 0),\\
&(5.3\ ,\ 3.7\ ,\ 0),\\
&(6.1\ ,\ 4.3\ ,\ 1),\\
&(9.2\ ,\ 3.5\ ,\ 1),\\
&(7.2\ ,\ 3.8\ ,\ 1),\\
&(5.1\ ,\ 1.5\ ,\ 1),\\
&(8.2\ ,\ 1.7\ ,\ 1)
\end{aligned}
$$



$x_1 = (3{\cdot}5\ ,\ 3{\cdot}7)$
$x_2 = (6\ ,\ 2)$

	$y = 0$	$y = 1$
feature 1		
μ	3.34	7.16
σ	1.50	1.63
feature 2		
μ	3.22	2.96
σ	0.77	1.28

$x_1 = (3{\cdot}5\ ,\ 3{\cdot}7)$

$y = 0$

$$p(x_1|y) \quad = \quad p(y) \times p(x_{10} = 3{\cdot}5|y) \times p(x_{11} = 3{\cdot}7|y) \quad = \quad 0.5 \times 0.26 \times 0.43 = 0.056$$

$y = 1$

$$p(x_1|y) \quad = \quad p(y) \times p(x_{10} = 3{\cdot}5|y) \times p(x_{11} = 3{\cdot}7|y) \quad = \quad 0.5 \times 0.02 \times 0.26 = 0.026$$

So, predict Class of x_1 is $y = 0$

Multinomial Naïve Bayes

Used in text classification

$\{(D, y)\}$, D: Document , y Class (Category of D).

$\theta_y = (\theta_{y1}, ..., \theta_{yd})$ The distribution vector for class y
d : number of feature = size of the vocabulary.

$\theta_{yi} = P(x_i \mid y)$ The probability of feature i appearing in a sample belonging to class y.

θ_y is estimated by a smoothed version of maximum likelihood, i.e. relative frequency counting:

$$\hat{\theta}_{yi} = \frac{N_{yi} + \alpha}{N_y + \alpha d}$$

$N_{yi} = \sum_{x \in T} x_i$: number of times feature i appears in a sample of class y in the training set T.
$N_y = \sum_{i=1}^{d} N_{yi}$: total count of all features for class y.
$\alpha \geqslant 0$ prevents zero probabilities.
Setting $\alpha = 1$: Laplace smoothing, while $\alpha < 1$ Lidstone smoothing.

e.g. Text classifier

No	Doc	Class
1	All Attended, All Exercises	A+
2	Attended, Exercises	A
3	No Attended	E
4	No Exercises	E
test	Attended, No Exercises	?

e.g. Text classifier

Feature = { All, Attended, Exercises, No }

$$\hat{y} = \underset{y \in C}{argmax}\ p(y) \prod_{i=1}^{d} p(x_i|y)$$

P(A+) = 1/4 P(A) = 1/4 P(E) = 1/2

P(Attended | A+)	=	(1 + 1) / (4 + 1*4)	=	2/8
P(No | A+)	=	(0 + 1) / (4 + 1*4)	=	1/8
P(Excercises | A+)	=	(1 + 1) / (4 + 1*4)	=	2/8
P(Attended | A)	=	(1 + 1) / (2 + 1*4)	=	2/6
P(No | A)	=	(0 + 1) / (2 + 1*4)	=	1/6
P(Excercises | A)	=	(1 + 1) / (2 + 1*4)	=	2/6
P(Attended | E)	=	(1 + 1) / (4 + 1*4)	=	2/8
P(No | E)	=	(2 + 1) / (4 + 1*4)	=	3/8
P(Excercises | E)	=	(1 + 1) / (4 + 1*4)	=	2/8

e.g. Text classifier

$$\hat{y} = \underset{y \in C}{argmax}\ p(y) \prod_{i=1}^{d} p(x_i|y)$$

P(A+ | test) = 1/4 * 2/8 * 1/8 * 2/8 $\approx$ 0.002

P(A | test) = 1/4 * 2/6 * 1/6 * 2/6 $\approx$ 0.005

P(E | test) = 1/2 * 2/8 * 3/8 * 2/8 $\approx$ 0.01 $\surd$

Attended, No Exercises : E class.

Bernoulli Naïve Bayes

Multivariate Bernoulli distributions

$$D = \{ \ (x, y) \ \} \ , \ \ x \in \mathbb{R}^d \ , \ y \in C$$

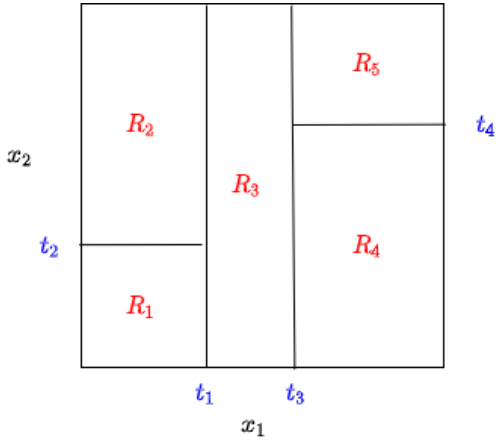
x_i binary-valued (Bernoulli, boolean) variable.

The decision rule for Bernoulli naive Bayes is based on

$$P(x_i|y) = P(i|y)x_i + (1 - P(i|y))(1 - x_i)$$

Decision Tree

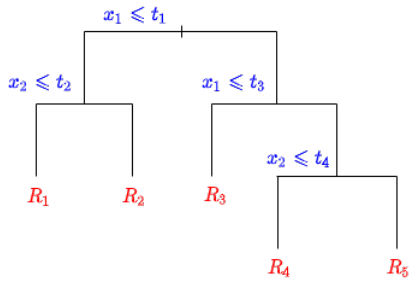
e.g. split $x(x_1, x_2)$ to 5 region

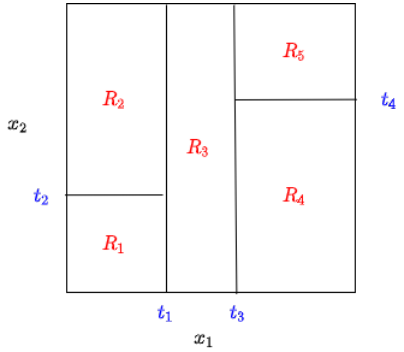


Decision Tree

The Philosophy:

$\{x(x_1, x_2), y\}$





$$\hat{f}(x) = \sum_{m=1}^{5} c_m I\{ (x_1, x_2) \in R_m \}$$

e.g. Regression Decision Tree

Dataset

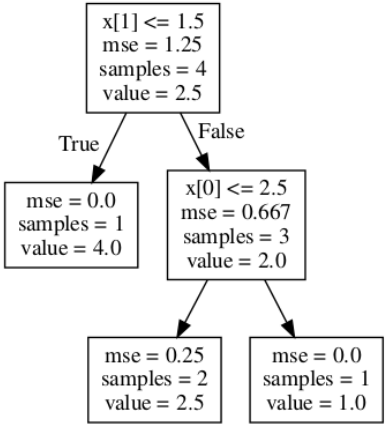
$x = \{\ (2,4), (3,3), (1,2), (4,1)\ \}$

$y = \{\ 3, 1, 2, 4\ \}$

Predict

$(2.3, 2) \Rightarrow \hat{y} = 2.5$

$(2.5, 1) \Rightarrow \hat{y} = 4.0$



e.g. Classification Decision Tree

Dataset

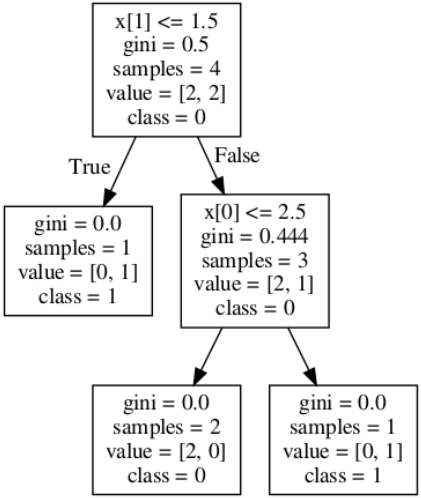
$x = \{\ (2,4), (3,3), (1,2), (0,1)\ \}$

$y = \{\ 0, 1, 0, 1\ \}$

Predict

$(3.2, 1.4) \Rightarrow \hat{y} = 1$

$(2.3, 2.4) \Rightarrow \hat{y} = 0$



Decision Tree Model

- Dataset: $\{(x_i, y_i)\}_1^n$, $x_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}$ $(or\ y_i \in\ Category)$
- Partition: M regions $R_1, R_2, ..., R_M$
- Model response as a constant c_m in each region:
- I: Indicator function

$$f(x) = \sum_{m=1}^{M} c_m I\{\ x \in R_m\ \}$$

- Data at node R with n samples
- Candidate split (j, s) : feature j , threshold s

$$R(j,s) \rightarrow R_1(j,s)\ +\ R_2(j,s)$$

Left $\quad R_1(j,s) = \{\ (x,y) \mid x_j \leqslant s\ \}$

Right $\quad R_2(j,s) = \{\ (x,y) \mid x_j > s\ \}$

Decision Tree algorithm

Loss Function

– $H()$: **Loss Function**

– $G()$: **Information Gain**

$$G(R(j,s)) = \frac{n_1}{n} H(R_1(j,s)) + \frac{n_2}{n} H(R_2(j,s))$$

$$(j,s)^* = \underset{(j,s)}{argmin}\, G(R(j,s))$$

Recurse

$R_1(j,s)$, $R_2(j,s)$

Until maximum allowable depth is reached

Decision Tree use for both classification and regression tasks.

Regression Decision Tree algo.

CART algorithm

Loss function base on Mean Squared Error (MSE or $\mathscr{L}_2$ error)

$$MSE(.) = \frac{1}{n}\sum_{i=1}^{n}(y_i - f(x_i))^2$$

The best $f(x_i)$ is average of y_i on region R

$$c_k = ave(y_i \mid x_i \in R_k)$$

$$H(R_k) = MSE(R_k) = \frac{1}{n}\sum_{y \in R_k}(y - c_k)^2$$

$$G(R(j,s)) = \left\{ \frac{n_1}{n}\frac{1}{n_1}\sum_{y \in R_1}(y - c_1)^2 \; + \; \frac{n_2}{n}\frac{1}{n_2}\sum_{y \in R_2}(y - c_2)^2 \right\}$$

Regression Decision Tree algo.

CART algorithm

$$G(R(j,s)) = \frac{1}{n}\Big\{ \sum_{y \in R_1} (y - c_1)^2 \; + \; \sum_{y \in R_2} (y - c_2)^2 \Big\}$$

$$(j,s)^* = \underset{(j,s)}{argmin} \Big\{ \sum_{y \in R_1(j,s)} (y - c_1)^2 + \sum_{y \in R_2(j,s)} (y - c_2)^2 \Big\}$$

Regression Decision Tree algo.

CART(R, $stop!$**)**

1. $list\ W = \{\ \}$
2. $for\ all\ j:\ feature\ x_j$
   - $sort\ Domain\ \{x_j\}$
   - $for\ all\ t_k \in Domain\ \{x_j\}$
     - $choose\ s:\ s = \frac{(t_k\ +\ t_{k+1})}{2}$
     - $w(j,s) = \sum_{y \in R_1(j,s)} (y - c_1)^2 + \sum_{y \in R_2(j,s)} (y - c_2)^2$
     - $add\ w(j,s)\ to\ list\ W$
3. $w(j,s)\ =\ min\{W\}$
4. **CART(**$R_1(j,s)$, $stop!$**)** , **CART(**$R_2(j,s)$, $stop!$**)**

Regression Tree Algorithm

Problem

– How large should we grow the tree?

– Very large tree might overfit the data.

– Small tree might not show the important structure.

– Optimal tree size, we can choose from the dataset.

– Real domain x !. We have to partition it.

Classification Decision Tree algo.

Loss function base on proportion p_{mk} of class k in region R_m with N_m observations:

$$p_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I(y_i = k) = \frac{n_k}{N_m}$$

– **Gini**

$$H(R_m) = \sum_{k=1}^{K} p_{mk}(1 - p_{mk}) = 1 - \sum_{k=1}^{K} p_{mk}^2$$

– **Entropy**

$$H(R_m) = -\sum_{k=1}^{K} p_{mk} \; log \; p_{mk}$$

– **Misclassification**

$$H(R_m) = 1 - max(p_{mk})$$

CART algorithm

gini

$$(j,s)^* = \underset{(j,\ s)}{argmin} \left\{ \frac{n_1}{n} H(R_1) \ + \ \frac{n_2}{n} H(R_2) \right\}$$

$$(j,s)^* = \underset{(j,\ s)}{argmin} \left\{ \frac{n_1}{n} (1 - \sum_{k=1}^{K} p_{1k}^2) \ + \ \frac{n_2}{n} (1 - \sum_{k=1}^{K} p_{2k}^2) \right\}$$

$$(j,s)^* = \underset{(j,\ s)}{argmin} \left\{ 1 - (\frac{n_1}{n} \sum_{k=1}^{K} p_{1k}^2 \ + \ \frac{n_2}{n} \sum_{k=1}^{K} p_{2k}^2) \right\}$$

$$(j,s)^* = \underset{(j,\ s)}{argmax} \left\{ n_1 \sum_{k=1}^{K} p_{1k}^2 \ + \ n_2 \sum_{k=1}^{K} p_{2k}^2 \right\}$$

CART algorithm (gini)

CART($R,\ stop!$**)**

1. $list\ W = \{\ \}$
2. $for\ all\ j:\ feature\ x_j$
   - $sort\ Domain\ \{x_j\}$
   - $for\ all\ t_k \in Domain\ \{x_j\}$
     - $choose\ s:\ s = \frac{(t_k\ +\ t_{k+1})}{2}$
     - $w(j,s) = n_1 \sum_{k=1}^{K} p_{1k}^2\ +\ n_2 \sum_{k=1}^{K} p_{2k}^2$
     - $add\ w(j,s)\ to\ list\ W$
3. $w(j,s)\ =\ max\{W\}$
4. **CART(**$R_1(j,s),\ stop!$**)** , **CART(**$R_2(j,s),\ stop!$**)**

e.g. Classification Decision Tree

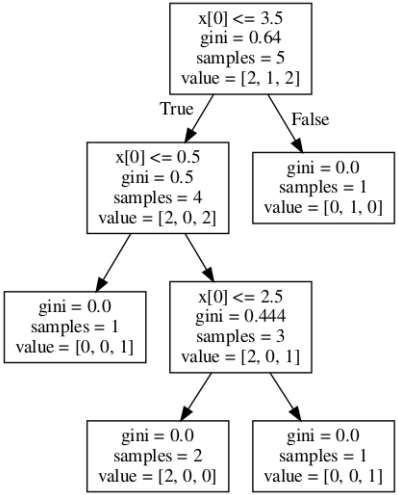
Dataset

$x = \{\ 3,\ 1,\ 0,\ 4,\ 2\ \}$
$y = \{\ 2,\ 0,\ 2,\ 1, 0\ \}$

Predict

$value = [n_0, n_1, n_2]$
$number\ of\ class$

$(x = 4.5) \Rightarrow \hat{y} = 1$
$(x = 1.3) \Rightarrow \hat{y} = 0$



Nonparametric Models

- **Parametric Models:**
  - Assume the target function has a fixed form a linear equation $y = ax + b$.
  - Once trained, we can throw away the data and just keep the parameters a, b.
- **Nonparametric Models:**
  - Do not assume a fixed structure. The complexity of the model grows with the amount of data.
  - Key Idea: The data is the model. We keep the training examples to make future predictions.
  - Also call **instance-based learning** or **memory-based learning**.
  - Simplest method is **Table lookup:**
    - All the training examples, put them in a lookup table.
    - When asked for Table lookup $h(x)$, see if x is in the table; if it is, return the corresponding y

The Core Concept of Nearest-Neighbor

Learning by Remembering (Instance-Based Learning)

- **The Philosophy:** Instead of forming a general hypothesis h, the agent simply **remembers** all training examples (x, y).
- **The Prediction:** For a new input x, find the example (x_j, y_j) in memory that is closest to x and return y_j.
- **Key Characteristic:** It is a **Nonparametric Model** – the complexity of the model grows with the size of the dataset.
- **Lazy Learning:** No work is done during training; all computation happens at the time of prediction (query time).

Measuring Similarity (Minkowski Distance)

How do we define "Closest"?

To find the nearest neighbor, we use a distance function $D(x_i, x_j)$,
Russell & Norvig highlight the **Minkowski Distance L^p norm**:

$$D(x_i, x_j) = \left(\sum_k |x_{i,k} - x_{j,k}|^p\right)^{1/p}$$

- $p = 1$ (**Manhattan Distance**): Calculates distance along axis-aligned paths (like city blocks).
- $p = 2$ (**Euclidean Distance**): The standard "straight-line" distance.
- k-**Nearest Neighbors** (k-**NN**): To reduce sensitivity to noise, we look at the k closest points:
  - **Classification:** Plurality vote among k neighbors.
  - **Regression:** Average value of k neighbors.

Examples: k-NN Classification

1. **Training Dataset (Memory)**

We want to predict if a student will **Pass** or **Fail** an AI exam based on two features:

1. **Hours Studied** x_1
2. **Mock Test Score** x_2

Student	**Hours** x_1	**Mock Score** x_2	**Result** y
A	10	80	Pass
B	2	30	Fail
C	8	70	Pass
D	4	50	Fail

Examples: k-NN Classification

2. **The Query (New Input)**
   A new student X has: **Hours** $= 5$, **Mock Score** $= 60$.

3. **Calculation Using Euclidean Distance** $p = 2$

$$D(X, x) = \sqrt{(X_1 - x_1)^2 + (X_2 - x_2)^2}$$

$$D(X, A) = \sqrt{(X_1 - A_1)^2 + (X_2 - A_2)^2} = \sqrt{(5 - 10)^2 + (60 - 80)^2} \approx 20.61$$
$$D(X, B) = \sqrt{(X_1 - B_1)^2 + (X_2 - B_2)^2} = \sqrt{(5 - 2)^2 + (60 - 30)^2} \approx 9.68$$
$$D(X, C) = \sqrt{(X_1 - C_1)^2 + (X_2 - C_2)^2} = \sqrt{(5 - 8)^2 + (60 - 70)^2} \approx 4.77$$
$$D(X, D) = \sqrt{(X_1 - D_1)^2 + (X_2 - D_2)^2} = \sqrt{(5 - 4)^2 + (60 - 50)^2} \approx 10.04$$

Examples: k-NN Classification

4. **Decision** $k = 3$

The 3 nearest neighbors are: $C(4.77), B(9.68), D(10.04)$

- **Results:** {Pass, Fail, Fail}
- **Final Prediction** for X: **Fail** (by plurality vote 2:1)

Examples: k-NN Regression

1. **Training Dataset (Memory)**

Predict the Final Exam Score (0-100) for student

1. **Hours Studied** x_1
2. **Mock Test Score** x_2

Student	Hours x_1	Mock Score x_2	Final Score y
A	10	80	85
B	2	30	40
C	8	70	92
D	4	50	55

Examples: k-NN Regression

2. **The Query (New Input)**
   A new student X has: **Hours** $= 5$, **Mock Score** $= 60$.

3. **Calculation Using Euclidean Distance** $p = 2$

$$D(X, x) = \sqrt{(X_1 - x_1)^2 + (X_2 - x_2)^2}$$

$$D(X, A) = \sqrt{(X_1 - A_1)^2 + (X_2 - A_2)^2} = \sqrt{(5 - 10)^2 + (60 - 80)^2} \approx 20.61$$
$$D(X, B) = \sqrt{(X_1 - B_1)^2 + (X_2 - B_2)^2} = \sqrt{(5 - 2)^2 + (60 - 30)^2} \approx 9.68$$
$$D(X, C) = \sqrt{(X_1 - C_1)^2 + (X_2 - C_2)^2} = \sqrt{(5 - 8)^2 + (60 - 70)^2} \approx 4.77$$
$$D(X, D) = \sqrt{(X_1 - D_1)^2 + (X_2 - D_2)^2} = \sqrt{(5 - 4)^2 + (60 - 50)^2} \approx 10.04$$

Examples: k-NN Regression

2. **The Query (New Input)**
   A new student X has: **Hours** $= 5$, **Mock Score** $= 60$.

3. **Calculation Using Euclidean Distance** $p = 2$

$$D(X, x) = \sqrt{(X_1 - x_1)^2 + (X_2 - x_2)^2}$$

$$D(X, A) = \sqrt{(X_1 - A_1)^2 + (X_2 - A_2)^2} = \sqrt{(5 - 10)^2 + (60 - 80)^2} \approx 20.61$$
$$D(X, B) = \sqrt{(X_1 - B_1)^2 + (X_2 - B_2)^2} = \sqrt{(5 - 2)^2 + (60 - 30)^2} \approx 9.68$$
$$D(X, C) = \sqrt{(X_1 - C_1)^2 + (X_2 - C_2)^2} = \sqrt{(5 - 8)^2 + (60 - 70)^2} \approx 4.77$$
$$D(X, D) = \sqrt{(X_1 - D_1)^2 + (X_2 - D_2)^2} = \sqrt{(5 - 4)^2 + (60 - 50)^2} \approx 10.04$$

Examples: k-NN Regression

4. **Decision $k = 3$**

The 3 nearest neighbors are: $C(4.77), B(9.68), D(10.04)$

- Target values y: 92, 40, 55
- Calculation (Average): Score $= \frac{92+40+55}{3} = \mathbf{62.33}$

Final Result: Predicted Final Score for X is 62.33

Equation of a Line

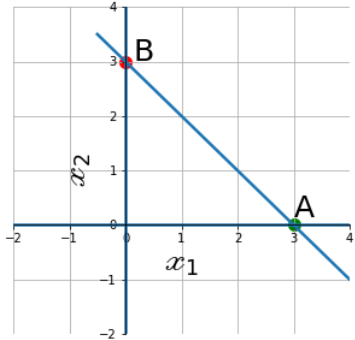
in $\mathbb{R}^2$ Line from 2 points.

$$\frac{x_1 - x_1^A}{x_1^A - x_1^B} = \frac{x_2 - x_2^A}{x_2^A - x_2^B}$$

$$\iff \frac{x_1 - 3}{3 - 0} = \frac{x_2 - 0}{0 - 3}$$

$$\iff x_2 = -x_1 + 3$$

$$\iff x_1 + x_2 - 3 = 0$$



$A(x_1^A, x_2^A)$, $B(x_1^B, x_2^B)$

General Line *(dot product)*

$$\overline{w} \cdot \overline{x} + b = 0$$

with $\overline{w} = (w_1, w_2)$, $\overline{x} = (x_1, x_2)$

Normal and Direction vector

in $\mathbb{R}^2$

$$\overline{w} \cdot \overline{x} + b = 0$$

$\overline{w} = (w_1, w_2)$: *Normal vector.*
$\overline{s} = (w_2, -w_1)$: *Direction vector.*

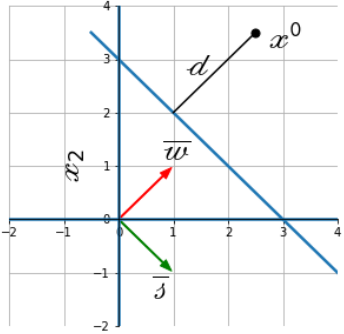
Norm of $\overline{w}$ $\quad \|\overline{w}\| = \sqrt{w_1^2 + w_2^2}$

d : *distance from* x^0 *to line.*

$$d = \frac{|\overline{w} \cdot \overline{x^0} + b|}{\|\overline{w}\|}$$

$$\Rightarrow \textit{distance from } (0,0) \textit{ to line } = \frac{|b|}{\|\overline{w}\|}$$

$\overline{u} \cdot \overline{v} = u_1 v_1 + u_2 v_2 + ... + u_n v_n$ $\quad (algebraic)$
$\overline{u} \cdot \overline{v} = \|\overline{u}\| \|\overline{v}\| cos(\alpha)$ $\quad (geometric)$



Distance between Parallel Lines

in $\mathbb{R}^2$

$$\ell_1 : \overline{w} \cdot \overline{x} + b_1 = 0$$
$$\ell_2 : \overline{w} \cdot \overline{x} + b_2 = 0$$

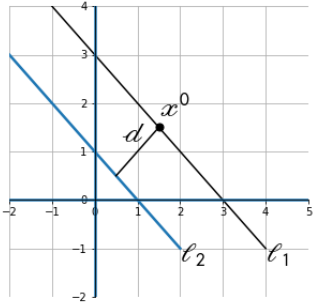
ℓ_1 *parallel with* ℓ_2

$x^0 \in \ell_1 \Rightarrow \overline{w} \cdot \overline{x^0} + b_1 = 0$

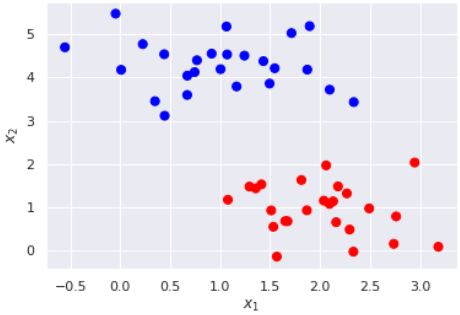
d : *distance from* x^0 *to* ℓ_2

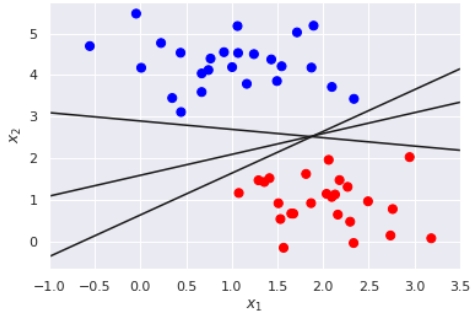
$$d = \frac{|\overline{w} \cdot \overline{x^0} + b_2|}{\|\overline{w}\|} = \frac{|b_2 - b_1|}{\|\overline{w}\|}$$

$$d = \frac{|b_2 - b_1|}{\|\overline{w}\|} \quad , \quad \textit{unit vector } \hat{w}, \quad \hat{w} = \frac{\overline{w}}{\|w\|}$$



SVM Overview

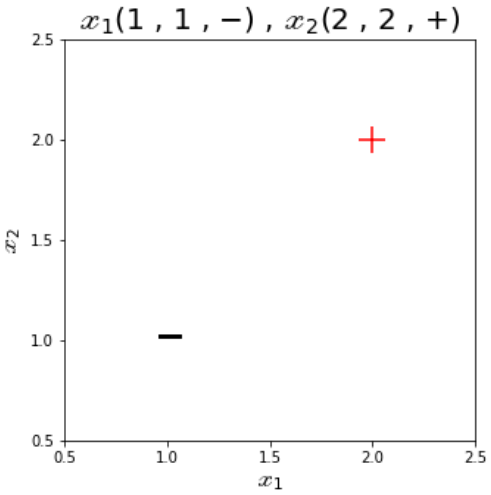


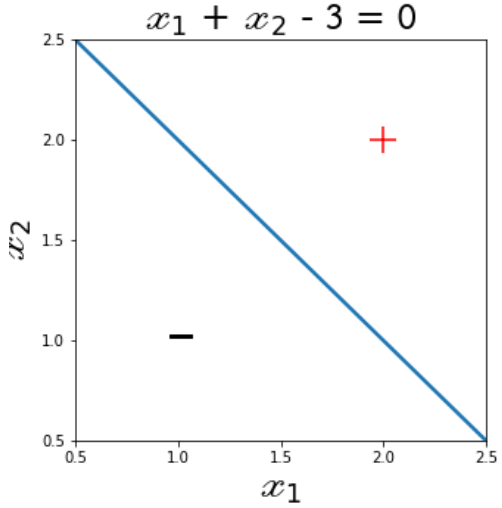


– dataset from sklearn. 2 class.

– many lines can separate 2 class.

SVM Overview





SVM, Linear, Binary class

$$\mathcal{D} = \left\{(x_i, y_i)\right\}_1^n \ , \ x_i \in \mathbb{R}^d \ , \ y_i \in \{-1, 1\}$$

Key idea: find widest separating "street"

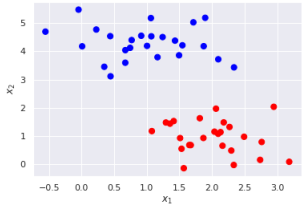
$h^{\text{o}} \ : \quad w \cdot x + b = 0$

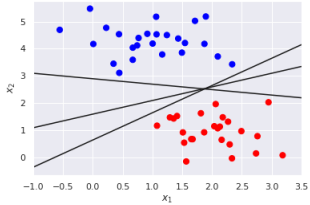
$h^{+} : \quad w \cdot x + b = +1$

$h^{-} : \quad w \cdot x + b = -1$

Model

$$f(x) = sign(w \cdot x + b)$$





SVM, Linear, binary classifiers

$\forall(x_i, y_i)$

$h^+ : \quad w \cdot x_i + b - 1 \geqslant 0 \quad , \quad y_i = +1$

$h^- : \quad w \cdot x_i + b + 1 \leqslant 0 \quad , \quad y_i = -1$

$\Rightarrow$

$w \cdot x_i + b \geqslant +1 \quad , \quad y_i = +1$

$w \cdot x_i + b \leqslant -1 \quad , \quad y_i = -1$

$\Rightarrow$

1st *constrain*

$$y_i(w \cdot x_i + b) \geqslant 1$$

Distance from h^+ to h^-

$$d = \frac{|(b-1)-(b+1)|}{\|w\|} = \frac{2}{\|w\|}$$

2nd *constrain*

$$\text{Maximize } \frac{2}{\|w\|} \Rightarrow \text{minimize } \frac{1}{2}\|w\|^2$$

SVM, Optimization with constraints

$$\underset{w,b}{argmin} \quad \tfrac{1}{2}\|w\|^2$$

$$s.t. \qquad y_i(w \cdot x_i + b) - 1 \geqslant 0 \quad , \quad \forall (x_i, y_i) \in \mathcal{D}$$

Lagrange Multipliers

$$\mathcal{L}(w, b, \alpha) = \tfrac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i [y_i(w \cdot x_i + b) - 1]$$

$$= \tfrac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i \; y_i \; w \cdot x_i - \sum_{i=1}^{n} \alpha_i \; y_i \; b + \sum_{i=1}^{n} \alpha_i \quad (1)$$

- Lagrange multiplier α_i
- Satisfies the Karush–Kuhn–Tucker (KKT)

$$\alpha_i [y_i(w \cdot x_i + b) - 1] = 0$$
$$\alpha_i \geqslant 0$$

- Minimize $\mathcal{L}$ respect to w , b
- and maximize $\mathcal{L}$ respect to α

SVM, Optimization with constraints

Minimize $\mathcal{L}$ respect to w , b

$$\frac{\partial \mathcal{L}}{\partial w} = 0 \quad \Rightarrow \quad w = \sum_{i=1}^{n} \alpha_i y_i x_i \quad (2) \,, \qquad \frac{\partial \mathcal{L}}{\partial b} = 0 \quad \Rightarrow \quad \sum_{i=1}^{n} \alpha_i y_i = 0 \quad (3)$$

and maximize $\mathcal{L}$ respect to α use (1) and substitute by (2), (3):

$$\mathcal{L} = \tfrac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i \; y_i \; w \cdot x_i - \sum_{i=1}^{n} \alpha_i \; y_i \; b + \sum_{i=1}^{n} \alpha_i$$

$$\begin{aligned} \mathcal{L}_{dual} &= \tfrac{1}{2} w \cdot w - w \cdot \left(\sum_{i=1}^{n} \alpha_i \; y_i \; x_i \right) - b \sum_{i=1}^{n} \alpha_i \; y_i + \sum_{i=1}^{n} \alpha_i \\ &= -\tfrac{1}{2} w \cdot w + \sum_{i=1}^{n} \alpha_i \\ &= \sum_{i=1}^{n} \alpha_i - \tfrac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \; \alpha_j \; y_i \; y_j \; x_i \cdot x_j \end{aligned}$$

Dual problem

$$\underset{\alpha}{argmax} \quad \mathcal{L}_{dual} = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \ \alpha_j \ y_i \ y_j \ x_i \cdot x_j \qquad (4)$$

$$s.t. \qquad \sum_{i=1}^{n} \alpha_i \ y_i = 0 \ , \qquad \alpha_i \geqslant 0$$

$w, \ b$

KKT: $\alpha_i[y_i(w \cdot x_i + b) - 1] = 0$

Two case:

1. $\alpha_i = 0$, or
2. $y_i(w \cdot x_i + b) - 1 = 0 \quad \Rightarrow \quad y_i(w \cdot x_i + b) = 1$

$\Rightarrow \alpha > 0 \ \ then \ \ y_i(w \cdot x_i + b) = 1$

and x_i must be a support vector

if a point is not a support vector, then $\alpha_i = 0$

SVM Linear Classifier

$$w = \sum_{\alpha_i > 0} \alpha_i y_i x_i$$

b_i per support vector:

$\alpha_i(y_i(w \cdot x_i + b_i) - 1) = 0$

$y_i(w \cdot x_i + b_i) = 1$

$$b_i = \frac{1}{y_i} - w \cdot x_i$$

$$b_i = y_i - w \cdot x_i$$

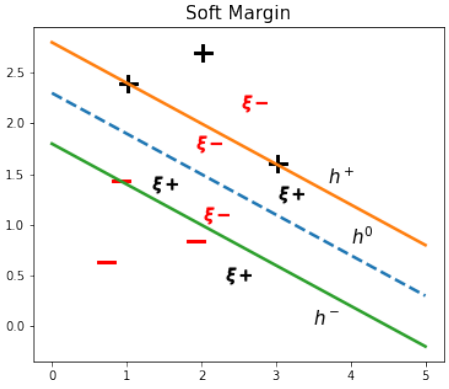
$$b = \underset{\alpha_i > 0}{avg}\{b_i\}$$

Linear Classifier

$$\hat{y} = sign(w \cdot x + b)$$

Soft Margin

- x_i away from hyperlane
- slack variables ξ_i for x_i
- $\xi_i \geqslant 0$
- $\xi_i = 0$, x_i at least $\frac{1}{\|w\|}$ away from hyperlane
- $0 < \xi_i < 1$ x_i within the margin
- $\xi_i \geqslant 1$ x_i appears on the wrong side



$$y_i(w \cdot x_i + b) \geqslant 1 - \xi_i$$

Soft Margin

$$\underset{w,b,\xi_i}{argmin} \quad \tfrac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$$

$$s.t. \qquad y_i(w \cdot x_i + b) \geqslant 1 - \xi_i$$

$$\xi_i \geqslant 0 \ , \ \forall x_i \in \mathcal{D}$$

$C > 0$

– large C $\sim$ small margin,

– small C $\sim$ large margin.

Soft Margin, Lagrange Multipliers

$$\underset{\alpha}{argmax} \;\; \mathcal{L}_{dual} \;=\; \sum_{i=1}^{n} \alpha_i - \tfrac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \; \alpha_j \; y_i \; y_j \; x_i \cdot x_j \qquad (4)$$

$$s.t. \qquad \sum_{i=1}^{n} \alpha_i \; y_i = 0$$

$$0 \leqslant \alpha_i \leqslant C$$

w vector and b

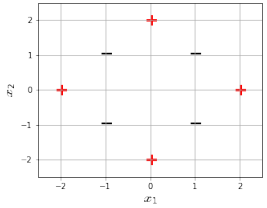
$$w = \sum_{\alpha_i > 0} \alpha_i \; y_i \; x_i$$

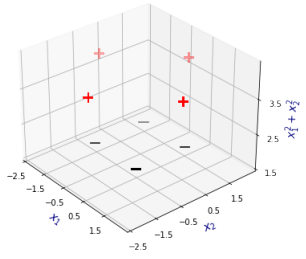
$$b = \underset{\alpha_i > 0}{avg} \{ b_i \}$$

Linear Classifier

$$\hat{y} = \; sign(w \cdot x + b)$$

Kernel Trick





e.g. Simple Kernel Trick: $\mathcal{R}^2 \rightarrow \mathcal{R}^3$

$$x(x_1, x_2)$$

$$\phi(x) = (x_1, x_2, x_1^2 + x_2^2)$$

Kernel Trick, Lagrange Multipliers

$$\underset{\alpha}{argmax} \;\; \mathcal{L}_{dual} \;=\; \sum_{i=1}^{n} \alpha_i - \tfrac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \; \alpha_j \; y_i \; y_j \; K(x_i, x_j) \qquad (5)$$

$$s.t. \qquad \sum_{i=1}^{n} \alpha_i \; y_i = 0$$

$$0 \leqslant \alpha_i \leqslant C$$

w vector and b

$$w = \sum_{\alpha_i > 0} \alpha_i \; y_i \; \phi(x_i)$$

$$b = \underset{\alpha_i > 0}{avg} \{b_i\}$$

Kernel SVM Classifier

$$\hat{y} = \; sign(w \cdot \phi(x) + b)$$

Dual Solution: Gradient Ascent

$(5) \Rightarrow$

$$\mathcal{L}(\alpha_k) = \alpha_k - \tfrac{1}{2}\alpha_k^2\ y_k^2\ K(x_k \cdot x_k) - \alpha_k\ y_k \sum_{\substack{i=1 \\ i \neq k}}^{n} \alpha_i\ y_i\ K(x_i \cdot x_k)$$

$$\nabla \mathcal{L}(\alpha) = \Big(\frac{\partial \mathcal{J}(\alpha)}{\partial \alpha_1}, \frac{\partial \mathcal{J}(\alpha)}{\partial \alpha_2}, ..., \frac{\partial \mathcal{J}(\alpha)}{\partial \alpha_n} \Big)$$

$$\frac{\partial \mathcal{L}}{\partial \alpha_k} \quad = \quad 1 - y_k \Big(\sum_{i=1}^{n} \alpha_i\ y_i\ \widetilde{K}(x_i \cdot x_k) \Big)$$

Gradient ascent

$$\alpha_{t+1} = \alpha_t + \eta \nabla \mathcal{J}(\alpha_t)$$

This algorithm can demo by scikit-learn.

Wisdom of the Crowd

Weight-guessing contest the ox.

Sir Francis Galton (1822-1911)

book, 2004, "The Wisdom of Crowds: Why the Many Are Smarter Than the Few and How Collective Wisdom Shapes Business, Economies, Societies and Nations".

James Michael Surowiecki.

Naturally, not all crowds are wise (for example, greedy investors of a stock market bubble)

Lior Rokach, 2010, PATTERN CLASSIFICATION USING ENSEMBLE METHODS

Wisdom of the Crowd

In order to become wise, the crowd should comply with the following criteria:

- **Diversity of opinion** Each member should have private information.
- **Independence** Members' opinions are not determined by the opinions of those around them.
- **Decentralization** Members are able to specialize and draw conclusions based on local knowledge.
- **Aggregation** Some mechanism exists for turning private judgments into a collective decision.

Lior Rokach, 2010, PATTERN CLASSIFICATION USING ENSEMBLE METHODS

Majority Voting

e.g. Ensemble of ten classifiers (using ten learning method)

Classifier	A score	B score	C score	Selected Label
1	0.2	0.7	0.1	B
2	0.1	0.1	0.8	C
3	0.2	0.3	0.5	C
4	0.1	0.8	0.1	B
5	0.2	0.6	0.2	B
6	0.6	0.3	0.1	A
7	0.25	0.65	0.1	B
8	0.2	0.7	0.1	B
9	0.2	0.2	0.8	C
10	0.4	0.3	0.3	A

Voted result

	Class A	Class B	Class C
Votes	2	5	3

Majority Voting

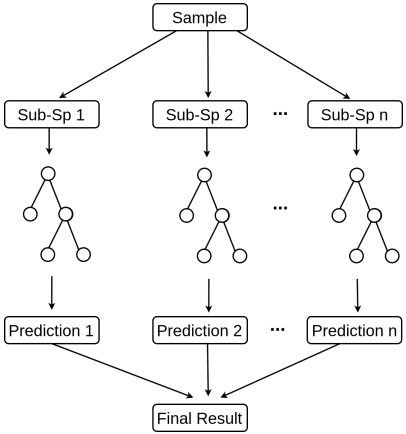
x : sample.

$y_k(x)$: classification of the k'th classifier.

$I\{y = c\}$: Indicator function.

$$class(x) = \underset{c_i \in dom(y)}{argmax} \left(\sum_k I \{ y_k(x) = c_i \} \right)$$

Random Forest



Introduction to Unsupervised Learning

Learning Without Feedback

- **The Core Philosophy:** The agent receives inputs but no explicit feedback or labels. It must discover hidden patterns and structures in the data on its own.
- **Objective:** To learn the underlying distribution of the data.
- **Key Tasks:**
  - **Clustering:** Grouping similar objects together.
  - **Dimensionality Reduction:** Simplifying data while preserving essential information.
- **Why it matters:** Most data in the real world is unlabeled.

K-Means Algorithm

$$\mathcal{D} = \left\{ x^{(i)} \right\}_1^n \ , \ x^{(i)} \in \mathbb{R}^d$$

Euclidian distance

$$e(x^{(i)}, x^{(j)}) = \sum_{m=1}^{d} (x_m^{(i)} - x_m^{(j)})^2 = \| \ x^{(i)} - x^{(j)} \ \|^2$$

e.g.

$$x^{(1)} = (1, 2, 3); \ x^{(2)} = (4, 5, 6), d = 3$$

$$e(x^{(1)}, x^{(2)}) = \sum_{m=1}^{3} (x_m^{(1)} - x_m^{(2)})^2 = (1-4)^2 + (2-5)^2 + (3-6)^2 = 27$$

K-Means Algorithm

Defining clusters

Partitioning dataset into k clusters $C_1, C_2, ..., C_k; k < n$

- C_i : Set of data points that belong together
- From C_i, compute μ^i a centroid that represents the cluster's center

$$\mu^i = \frac{1}{|C_i|} \sum_{x \in C_i} x \quad , \quad \mu^{(i)} \in \mathbb{R}^d$$

$|C_i|$: Number of points in cluster i

$\sum_{x \in C_i} x$: Sum of all data points in cluster i

$\frac{1}{|C_i|}$: Divide by the number of points to get the average.

e.g.

$$C_1 = \{x_1(1,2,3), x_2(3,4,5), x_3(6,7,8)\}$$
$$\mu^1 = (1+3+6, 2+4+7, 3+5+7)/3 = (3{\cdot}3, 4{\cdot}3, 7)$$

K-Means Algorithm

The Philosophy

- Dataset: $\mathcal{D} = \left\{ x^{(i)} \right\}_1^n \ , \ x^{(i)} \in \mathbb{R}^d$
- Cluster: $C = \{C_1, C_2, ..., C_k\} \quad , \quad k < n$
- Centroids: $\mu^{(i)} = \frac{1}{|C_i|} \sum_{x \in C_i} x \quad , \quad \mu^{(i)} \in \mathbb{R}^d$
- Objective: $\underset{C}{argmin} \left(\sum_{i=1}^{k} \sum_{x \in C_i} \| x - \mu^{(i)} \|^2 \right)$

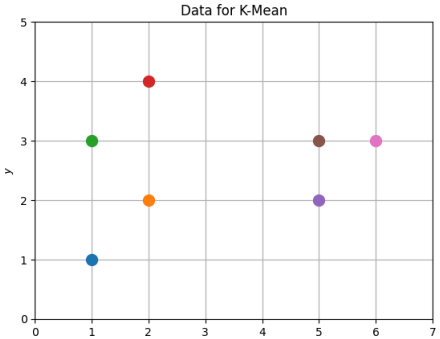
K-Means Algorithm

The Iterative Algorithm (Lloyd's Algorithm)

1. **Initialization:** Select initial centroids:
   $t = 0\ ,\ 1 \leqslant i \leqslant k, \mu^{(i)}(t) = random$
2. **Assignment Step:** Assign each observation to the cluster:
   $1 \leqslant i \leqslant k$
   $C_i(t) = \left\{ x : \parallel x - \mu^{(i)}(t) \parallel^2 \leqslant \parallel x - \mu^{(j)}(t) \parallel^2 \ \forall j, 1 \leqslant j \leqslant k \right\}$
3. **Update Step:** Calculate the new centroids:
   $1 \leqslant i \leqslant k$
   $\mu^{(i)}(t+1) = \frac{1}{|C_i(t)|} \sum_{x \in C_i(t)} x$
   $t = t + 1$
4. **Convergence:** Repeat steps 2 and 3 until the assignments no longer change or a maximum number of iterations is reached.

K-Means Algorithm

e.g.



$$x^{(1)}(1,1), x^{(2)}(2,2), x^{(3)}(1,3), x^{(4)}(2,4), x^{(5)}(5,2), x^{(6)}(5,3), x^{(7)}(6,3)$$

K-Means Algorithm

K-mean with $K = 3$

Step 1. We can choose random or using k-mean++ for initial centroids.

With k=3 we choose the highest, lowest, and rightmost points in Dataset

$t = 0$; $\mu^{(1)}(0) = (1, 1), \mu^{(2)}(0) = (2, 4), \mu^{(3)}(0) = (6, 3)$

Step 2. Squared L^2 Norm: $\| x^{(n)} - \mu^{(i)}(t) \|^2 = (x_1^{(n)} - \mu_1^{(i)})^2 + (x_2^{(n)} - \mu_2^{(i)})^2$

$x^{(1)}$

$\| x^{(1)} - \mu^{(1)}(0) \|^2 = 0$

$\| x^{(1)} - \mu^{(2)}(0) \|^2 = 10$

$\| x^{(1)} - \mu^{(3)}(0) \|^2 = 29$

$x^{(2)}$

$\| x^{(2)} - \mu^{(1)}(0) \|^2 = 2$

$\| x^{(2)} - \mu^{(2)}(0) \|^2 = 4$

$\| x^{(2)} - \mu^{(3)}(0) \|^2 = 17$

$x^{(3)}$

$\| x^{(3)} - \mu^{(1)}(0) \|^2 = 4$

$\| x^{(3)} - \mu^{(2)}(0) \|^2 = 2$

$\| x^{(3)} - \mu^{(3)}(0) \|^2 = 25$

$x^{(4)}$

$\| x^{(4)} - \mu^{(1)}(0) \|^2 = 10$

$\| x^{(4)} - \mu^{(2)}(0) \|^2 = 0$

$\| x^{(4)} - \mu^{(3)}(0) \|^2 = 17$

$x^{(5)}$

$\| x^{(5)} - \mu^{(1)}(0) \|^2 = 17$

$\| x^{(5)} - \mu^{(2)}(0) \|^2 = 13$

$\| x^{(5)} - \mu^{(3)}(0) \|^2 = 2$

$x^{(6)}$

$\| x^{(6)} - \mu^{(1)}(0) \|^2 = 20$

$\| x^{(6)} - \mu^{(2)}(0) \|^2 = 10$

$\| x^{(6)} - \mu^{(3)}(0) \|^2 = 1$

K-Means Algorithm

$x^{(7)}$

$\| x^{(7)} - \mu^{(1)}(0) \|^2 = 29$

$\| x^{(7)} - \mu^{(2)}(0) \|^2 = 17$

$\| x^{(7)} - \mu^{(3)}(0) \|^2 = 0$

$C_1 = \{x^{(1)}, x^{(2)}\}$, $C_2 = \{x^{(3)}, x^{(4)}\}$, $C_3 = \{x^{(5)}, x^{(6)}, x^{(7)}\}$

$t = 1$

Step 3.

$\mu^{(1)}(1) = (x^{(1)} + x^{(2)})/2 = (1.5, 1.5)$

$\mu^{(2)}(1) = (x^{(3)} + x^{(4)})/2 = (1.5, 3.5)$

$\mu^{(3)}(1) = (x^{(5)} + x^{(6)} + x^{(7)})/3 = (5.3, 2.7)$

Step 4.

Next to **Step 2.**

K-Means Algorithm

Step 2. (rep 2)

$x^{(1)}$

$$\| x^{(1)} - \mu^{(1)}(1) \|^2 = 0.5$$
$$\| x^{(1)} - \mu^{(2)}(1) \|^2 = 6.5$$
$$\| x^{(1)} - \mu^{(3)}(1) \|^2 = 21.38$$

$x^{(2)}$

$$\| x^{(2)} - \mu^{(1)}(1) \|^2 = 0.5$$
$$\| x^{(2)} - \mu^{(2)}(1) \|^2 = 2.5$$
$$\| x^{(2)} - \mu^{(3)}(1) \|^2 = 11.38$$

$x^{(3)}$

$$\| x^{(3)} - \mu^{(1)}(1) \|^2 = 2.5$$
$$\| x^{(3)} - \mu^{(2)}(1) \|^2 = 0.5$$
$$\| x^{(3)} - \mu^{(3)}(1) \|^2 = 18.58$$

$x^{(4)}$

$$\| x^{(4)} - \mu^{(1)}(1) \|^2 = 6.5$$
$$\| x^{(4)} - \mu^{(2)}(1) \|^2 = 0.5$$
$$\| x^{(4)} - \mu^{(3)}(1) \|^2 = 12.58$$

$x^{(5)}$

$$\| x^{(5)} - \mu^{(1)}(1) \|^2 = 12.5$$
$$\| x^{(5)} - \mu^{(2)}(1) \|^2 = 14.5$$
$$\| x^{(5)} - \mu^{(3)}(1) \|^2 = 0.58$$

$x^{(6)}$

$$\| x^{(6)} - \mu^{(1)}(1) \|^2 = 14.5$$
$$\| x^{(6)} - \mu^{(2)}(1) \|^2 = 12.5$$
$$\| x^{(6)} - \mu^{(3)}(1) \|^2 = 0.18$$

$x^{(7)}$

$$\| x^{(7)} - \mu^{(1)}(1) \|^2 = 22.5$$
$$\| x^{(7)} - \mu^{(2)}(1) \|^2 = 20.5$$
$$\| x^{(7)} - \mu^{(3)}(1) \|^2 = 0.58$$

K-Means Algorithm

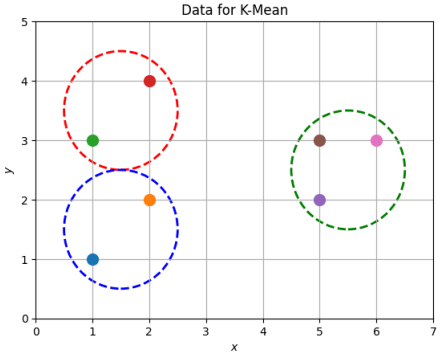
$C_1 = \{x^{(1)}, x^{(2)}\}$, $C_2 = \{x^{(3)}, x^{(4)}\}$, $C_3 = \{x^{(5)}, x^{(6)}, x^{(7)}\}$
$t = 2$

Step 3. (rep 2)

$$\mu^{(1)}(2) = (x^{(1)} + x^{(2)})/2 = (1.5, 1.5) = \mu^{(1)}(1)$$
$$\mu^{(2)}(2) = (x^{(3)} + x^{(4)})/2 = (1.5, 3.5) = \mu^{(2)}(1)$$
$$\mu^{(3)}(2) = (x^{(5)} + x^{(6)} + x^{(7)})/3 = (5.3, 2.7) = \mu^{(3)}(1)$$

Step 4.

No change $\Rightarrow$ **Stop.**



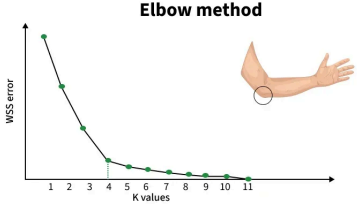
Optimizing the value of k

The Philosophy: Elbow Method

1. Repeat for $k = 1, 2, 3, 4, ...$
   - For each k, runing K-Means $\Rightarrow \mu^{(i)}$
   - Calculate WCSS (Within-Cluster Sum of Squares)

     $$WCSS = \sum_{i=1}^{k} \sum_{x \in C_i} \| x - \mu^{(i)} \|^2$$

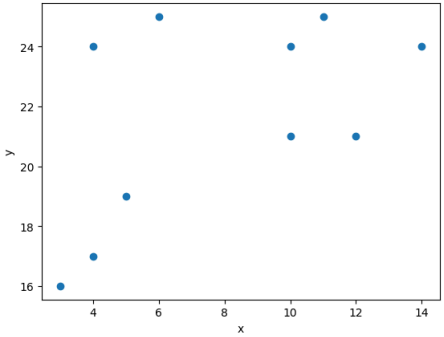
   - Plot $WCSS$ point
2. Connect all $WCSS$ point, we have an Elbow, that is an optimal k



Optimizing the value of k

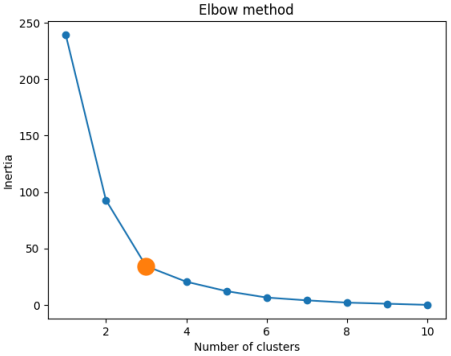
e.g.

Data for K-Meanes



Optimizing the value of k

$k = 1..10$



Optimal $k = 3$

Mathematical Foundations of PCA

From Variance to Eigenvectors

PCA is based on transforming the coordinate system to maximize the amount of information. To achieve this, we need the following three mathematical pillars:

1. Variance σ^2: Measures the spread of data in a single dimension.
   - $\sigma^2 = \frac{\sum(x_i - \mu_x)^2}{n-1}$, $\mu_x : mean$
2. Covariance $Cov(x, y)$: Measures the relationship between two dimensions.
   - $Cov(x, y) = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{n-1}$

     Note: PCA seeks the directions that maximize Variance while ensuring Covariance equals zero.

3. The Covariance Matrix Σ
   Consolidating all relationships between pairs of attributes into a symmetric square matrix:

   e.g. Covariance Matrix for 2 Variables x_1, x_2

$$\Sigma = \begin{bmatrix} Var(x_1) & Cov(x_1, x_2) \\ Cov(x_2, x_1) & Var(x_2) \end{bmatrix}$$

   e.g. Covariance Matrix for 3 Variables x_1, x_2, x_3

$$\Sigma = \begin{bmatrix} Var(x_1) & Cov(x_1, x_2) & Cov(x_1, x_3) \\ Cov(x_2, x_1) & Var(x_2) & Cov(x_2, x_3) \\ Cov(x_3, x_1) & Cov(x_3, x_2) & Var(x_3) \end{bmatrix}$$

Mathematical Foundations of PCA

4. Eigenvalues & Eigenvectors (Trị riêng & Vector đặc trưng)
   - To identify the new coordinate system, we solve the characteristic equation for the covariance matrix Σ:

     $$\Sigma v = \lambda v$$

   - Eigenvectors v: Determine the **directions** of the new axes (Principal Components).
   - Eigenvalues λ: Scalars determine the magnitude (amount of variance) captured along each corresponding eigenvector.
5. Final Projection (Dimensionality Reduction)

$$FinalData = CenteredData \times V_k$$

Calculate v , λ

$$\Sigma v = \lambda v$$

• When the matrix Σ (our data's "shape") acts on vector v, the vector does not change its direction. It only gets scaled by a factor of λ.

$$\Sigma v - \lambda v = 0$$

$$(\Sigma - \lambda I)v = 0$$

I: Identity matrix

• v is a non-zero vector, so the matrix $(\Sigma - \lambda I)$ must be Singular (non-invertible)

• In linear algebra, a matrix is singular if and only if its Determinant is zero:

$$det(\Sigma - \lambda I) = 0$$

Calculate v , λ

• e.g. Suppose after preprocessing two features (e.g., Hours Studied and Mock Score), we obtain the following Covariance Matrix Σ:

$\Sigma = \begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix}$ We solve the characteristic equation: $det(\Sigma - \lambda I) = 0$

$$det \begin{bmatrix} 4-\lambda & 2 \\ 2 & 1-\lambda \end{bmatrix} = 0$$

$$(4-\lambda)(1-\lambda) - (2 \times 2) = 0$$

$$\lambda^2 - 5\lambda + 4 - 4 = 0 \Rightarrow \lambda^2 - 5\lambda = 0$$

• Result: $\lambda_1 = 5$ and $\lambda_2 = 0$.

• **Observation:** PC1 captures 100% of the variance $\lambda = 5$, while PC2 captures 0%. This dataset is effectively 1D residing in a 2D space.

Calculate v , λ

Find the Eigenvector v for $\lambda_1 = 5$

Substitute $\lambda_1 = 5$ into the equation $(\Sigma - \lambda I)v = 0$:

$$\begin{bmatrix} 4-5 & 2 \\ 2 & 1-5 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Rightarrow \begin{bmatrix} -1 & 2 \\ 2 & -4 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

From the first row: $-v_1 + 2v_2 = 0 \Rightarrow v_1 = 2v_2$.

If we choose $v_2 = 1$, then $v_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$.

e.g. Hand Calculation

Raw Dataset (4 Dimensions)

Sample	A	B	C	D
S1	1	2	5	1
S2	2	4	4	0
S3	3	6	3	1
S4	4	8	2	0
S5	5	10	1	1

Step 1. Variance σ^2

Mean Centering $x' = x - \mu$

$\mu_A = 3,\ \mu_B = 6,\ \mu_C = 3,\ \mu_D = 0.6$

$A' = A - \mu_A,\ B' = B - \mu_B,\ C' = C - \mu_C,\ D' = D - \mu_D$

Center Data

Sample	A'	B'	C'	D'
S1	-2	-4	2	0.4
S2	-1	-2	1	-0.6
S3	0	0	0	0.4
S4	1	2	-1	-0.6
S5	2	4	-2	0.4

Step 1. Variance σ^2

Variance σ^2

$$\sigma^2 = \frac{\sum(x_i - \mu_x)^2}{n-1}$$

$$Var(A) = \frac{(-2)^2+(-1)^2+0^2+1^2+2^2}{5-1} = \frac{10}{4} = \mathbf{2.5}$$

$$Var(B) = \frac{(-4)^2+(-2)^2+0^2+2^2+4^2}{5-1} = \frac{40}{4} = \mathbf{10}$$

$$Var(C) = \frac{(2)^2+(1)^2+0^2+1^2+2^2}{5-1} = \frac{10}{4} = \mathbf{2.5}$$

$$Var(D) = \frac{(0.4)^2+(0.6)^2+(0.4)^2+(0.6)^2+(0.4)^2}{5-1} = \frac{1.2}{4} = \mathbf{0.3}$$

Step 2. Covariance $Cov(x, y)$

$$Cov(x, y) = \frac{\sum(x_i' \times y_i')}{n-1}$$

$$Cov(A, B) = \frac{(-2\times-4)+(-1\times-2)+(0\times0)+(1\times2)+(2\times4)}{5-1} = \frac{20}{4} = \mathbf{5.0}$$

$$Cov(A, C) = \frac{(-2\times-2)+(-1\times1)+(0\times0)+(1\times-1)+(2\times-2)}{5-1} = \frac{-10}{4} = \mathbf{-2.5}$$

$$Cov(A, D) = \frac{(-2\times0.4)+(-1\times-0.6)+(0\times0.4)+(1\times-0.6)+(2\times0.4)}{5-1} = \frac{0}{4} = \mathbf{0.0}$$

$$Cov(B, C) = \frac{(-4\times2)+(-2\times1)+(0\times0)+(2\times-1)+(4\times-2)}{5-1} = \frac{-20}{4} = \mathbf{-5.0}$$

$$Cov(B, D) = \frac{(-4\times0.4)+(-2\times-0.6)+(0\times0.4)+(2\times-0.6)+(4\times0.4)}{5-1} = \frac{0}{4} = \mathbf{0.0}$$

$$Cov(C, D) = \frac{(2\times0.4)+(1\times-0.6)+(0\times0.4)+(-1\times-0.6)+(-2\times0.4)}{5-1} = \frac{0}{4} = \mathbf{0.0}$$

Step3. Covariance Matrix Σ

$$\Sigma = \begin{bmatrix} Var(A) & Cov(A,B) & Cov(A,C) & Cov(A,D) \\ Cov(B,A) & Var(B) & Cov(B,C) & Cov(B,D) \\ Cov(C,A) & Cov(C,B) & Var(C) & Cov(C,D) \\ Cov(D,A) & Cov(D,B) & Cov(D,C) & Var(D) \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} 2.5 & 5.0 & -2.5 & 0.0 \\ 5.0 & 10.0 & -5.0 & 0.0 \\ -2.5 & -5.0 & 2.5 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.3 \end{bmatrix}$$

Step 4. Eigenvalues & Eigenvectors

Finding the Eigenvalues λ

- To determine the amount of information (Variance) each new axis holds, we solve the characteristic equation:

$$\det(\Sigma - \lambda I) = 0$$

- Using the Covariance Matrix Σ calculated in Step 3:

$$\det \begin{bmatrix} 2.5 - \lambda & 5.0 & -2.5 & 0 \\ 5.0 & 10.0 - \lambda & -5.0 & 0 \\ -2.5 & -5.0 & 2.5 - \lambda & 0 \\ 0 & 0 & 0 & 0.3 - \lambda \end{bmatrix} = 0$$

- The Results: $\lambda_1 = 15.0, \lambda_2 = 0.3, \lambda_3 = 0, \lambda_4 = 0$

Step 4. Eigenvalues & Eigenvectors

1. **Principal Component 1 (PC1)**

   For $\lambda_1 = 15.0$, the normalized eigenvector is:

   $$v_1 = [0.41, 0.82, -0.41, 0]^T$$

2. **Principal Component 2 (PC2)**

   For $\lambda_2 = 0.3$, the normalized eigenvector is:

   $$v_2 = [0, 0, 0, 1]^T$$

Feature Vector Matrix:

$$V_k = \begin{bmatrix} 0.41 & 0 \\ 0.82 & 0 \\ -0.41 & 0 \\ 0 & 1 \end{bmatrix}$$

Step 5. Final Projection (Dimensionality Reduction)

$$FinalData = CenteredData \times V_k$$

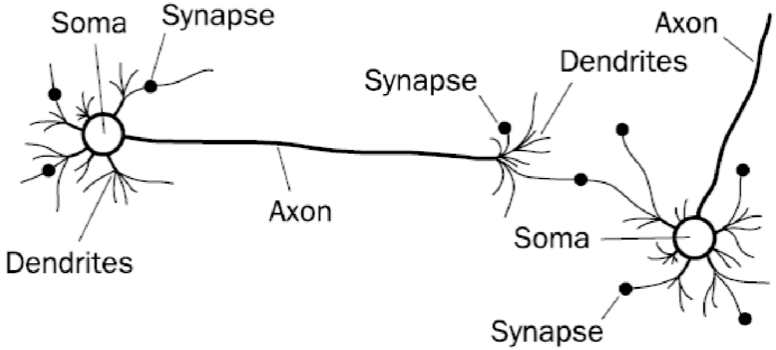
Where:

- **CenteredData:** The 5×4 matrix (Raw data minus the mean).
- V_k: The 4×2 matrix containing the top 2 eigenvectors v_1 and v_2.

Final Reduced Dataset (2D Result)

Sample	**PC1** (General Trend)	**PC2** (Feature D)
S1	-4.90	0.4
S2	-2.45	-0.6
S3	0.00	0.4
S4	2.45	-0.6
S5	4.90	0.4

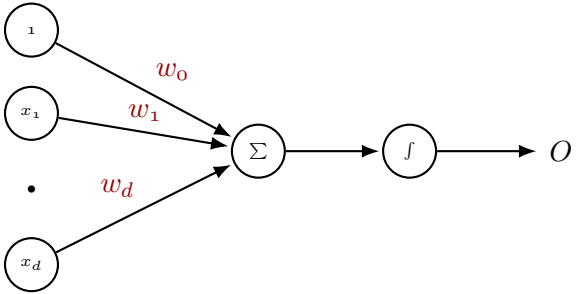
Biological Neuron



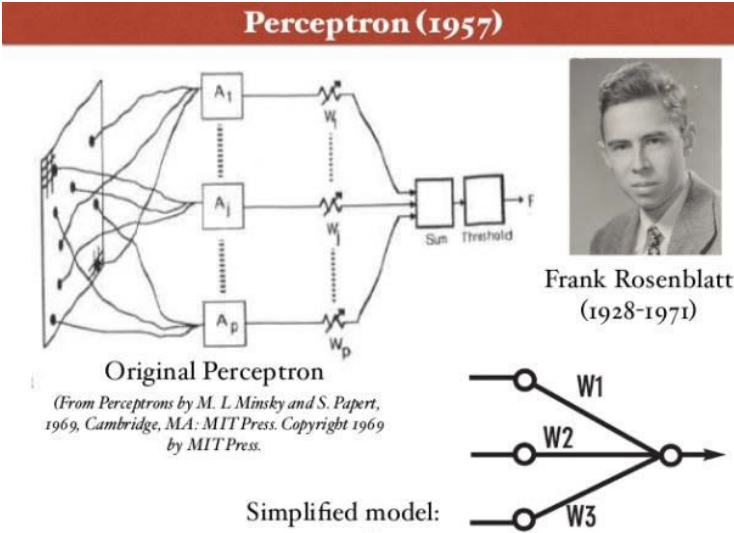
Biological Neuron vs. Artificial Neuron

Biological Neuron	**Artificial Neuron**
Cell Nucleus (Soma)	Node
Dendrites	Input
Synapse	Weights or interconnections
Axon	Output

Artificial Neuron

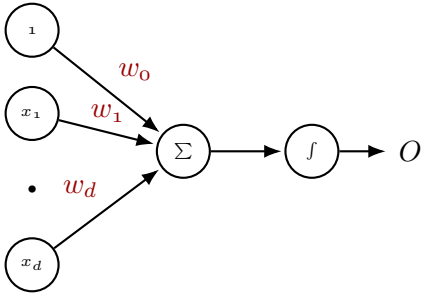


Rosenblatt's Perceptron



Rosenblatt's Perceptron

$$\mathcal{D} = \left\{ (x^{(i)}, y^{(i)}) \right\}_1^n \ , \ x^{(i)} \in \mathbb{R}^d \quad , \quad y^{(i)} \in \{\mathscr{C}_1, \mathscr{C}_2\} \equiv \{0, 1\}$$



Input: $x = [+1, x_1, x_2, ..., x_d]^T$ **Weight**: $w = [w_0, w_1, w_2, ..., w_d]^T$

$$\hat{y}^{(i)} = f(x^{(i)}) = \begin{cases} 1 & \textit{if } \sum_{j=0}^{d} w_j x_j^{(i)} > 0 \\ 0 & \textit{otherwise} \end{cases}$$

Perceptron Training Rule

Initialization: Initial weight vector w and learning rate η, set to some small value (e.g., 0.1)

$$w = random \quad , \quad \eta \in (0, 1]$$

Loop for *n epoch* and *not Convergence*

for each training $(x^{(i)}, y^{(i)})$ in dataset $\mathcal{D}$

Activation function:

$$sign(\sum_{j=0}^{d} w_j x_j^{(i)}) = \begin{cases} 1 & if \ \sum_{j=0}^{d} w_j x_j^{(i)} > 0 \\ 0 & otherwise \end{cases}$$

Class response: $\hat{y}^{(i)} = sign(\sum_{j=0}^{d} w_j x_j^{(i)})$

Modify weight vector: $w = w + \eta(y^{(i)} - \hat{y}^{(i)})x^{(i)}$

End

Perceptron Training Rule

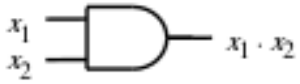
$$w = w + \eta(y^{(i)} - \hat{y}^{(i)})x^{(i)}$$

– This learning procedure can be proven to convergen to a weight vector that correctly classifies all training samples (Minsky and Papert 1969).

– If the data are not linearly separable, convergence is not assured.

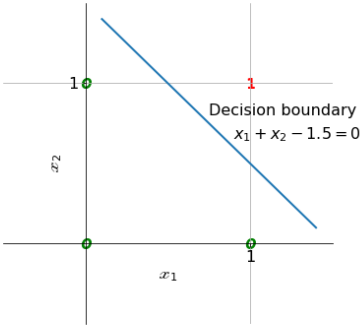
Perceptron Training Rule

AND gate



How to learn?

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1



Perceptron learning AND gate

Initial Weight vector: $w = (-0.5, 0.4, 1)$, Learning rate: $\eta = 0.1$

$(1, x)$	w	$w^T x$	$\hat{y}$	y	$w + \eta(y - \hat{y})x$
$(1, 0, 0)$	$(-0.5, 0.4, 1)$	-0.5	0	0	*not update* w
$(1, 0, 1)$	$(-0.5, 0.4, 1)$	0.5	1	0	$(-0.5, 0.4, 1) + 0.1(0 - 1)(1, 0, 1)$
$(1, 1, 0)$	$(-0.6, 0.4, 0.9)$	-0.2	0	0	*not update* w
$(1, 1, 1)$	$(-0.6, 0.4, 0.9)$	0.7	1	1	*not update* w
$(1, 0, 0)$	$(-0.6, 0.4, 0.9)$	-0.6	0	0	*not update* w
$(1, 0, 1)$	$(-0.6, 0.4, 0.9)$	0.3	1	0	$(-0.6, 0.4, 0.9) + 0.1(0 - 1)(1, 0, 1)$
$(1, 1, 0)$	$(-0.7, 0.4, 0.8)$	-0.3	0	0	*not update* w
$(1, 1, 1)$	$(-0.7, 0.4, 0.8)$	0.5	1	1	*not update* w
$(1, 0, 0)$	$(-0.7, 0.4, 0.8)$	-0.7	0	0	*not update* w
$(1, 0, 1)$	$(-0.7, 0.4, 0.8)$	0.1	1	0	$(-0.7, 0.4, 0.8) + 0.1(0 - 1)(1, 0, 1)$
$(1, 1, 0)$	$(-0.8, 0.4, 0.7)$	-0.4	0	0	*not update* w
$(1, 1, 1)$	$(-0.8, 0.4, 0.7)$	0.3	1	1	*not update* w

Perceptron learning AND gate

$(1, x)$	w	$w^T x$	$\hat{y}$	y	$w + \eta(y - \hat{y})x$
$(1, 0, 0)$	$(-0.8, 0.4, 0.7)$	-0.8	0	0	*not update* w
$(1, 0, 1)$	$(-0.8, 0.4, 0.7)$	-0.1	0	0	*not update* w
$(1, 1, 0)$	$(-0.8, 0.4, 0.7)$	-0.4	0	0	*not update* w
$(1, 1, 1)$	$(-0.8, 0.4, 0.7)$	0.3	1	1	*not update* w

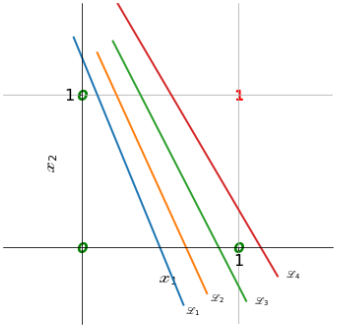
Learning result $\mathscr{L}_4$

$\mathscr{L}_1 : 0.4x_1 + x_2 - 0.5 = 0$

$\mathscr{L}_2 : 0.4x_1 + 0.9x_2 - 0.6 = 0$

$\mathscr{L}_3 : 0.4x_1 + 0.8x_2 - 0.7 = 0$

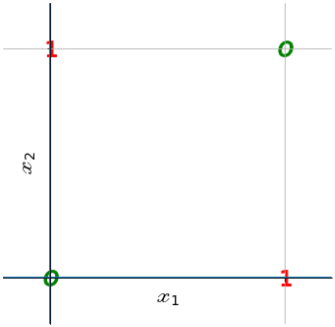
$\mathscr{L}_4 : 0.4x_1 + 0.7x_2 - 0.8 = 0$



Perceptron XOR problem



x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



Two class $\{0, 1\}$ cannot be separated by a single linear line.
XOR gate cannot be solved by Rosenblatt's perceptron.

Perceptron XOR problem

Solution 1

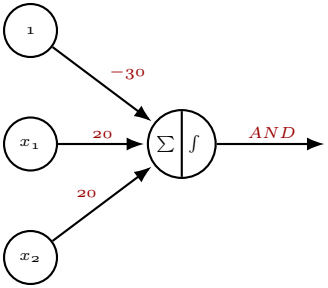
x_1	x_2	y	$\overline{x}_1 x_2$	$x_1 \overline{x}_2$	$\overline{x}_1 x_2 + x_1 \overline{x}_2$
0	0	0	0	0	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	0	0	0

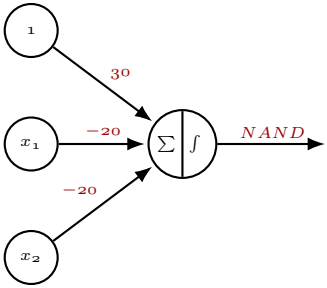
Solution 2

x_1	x_2	y	$\overline{x_1 x_2}$	$x_1 + x_2$	$\overline{x_1 x_2}(x_1 + x_2)$
0	0	0	1	0	0
0	1	1	1	1	1
1	0	1	1	1	1
1	1	0	0	1	0

Perceptron XOR problem

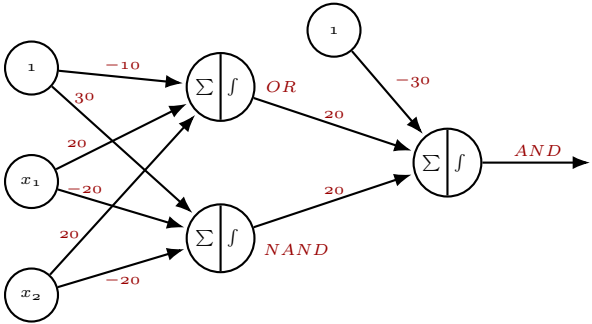
Perceptron for Solution 2





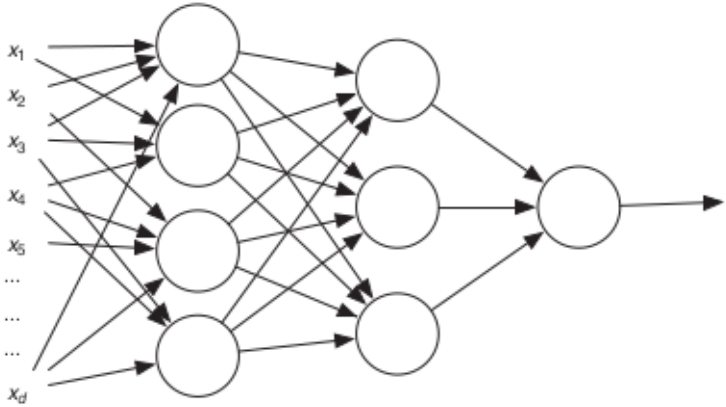
Perceptron XOR problem

Connect three gate

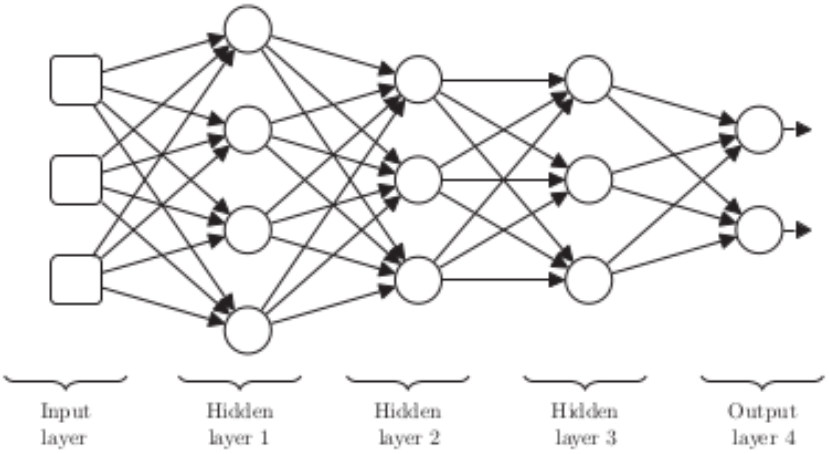


XOR as a combination of 3 basic perceptrons

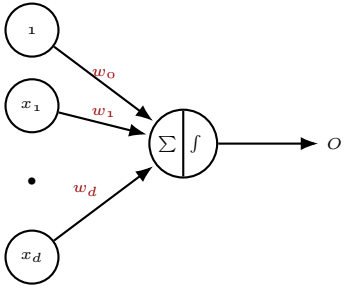
Multilayer Networks



Multilayer Networks



Activation function

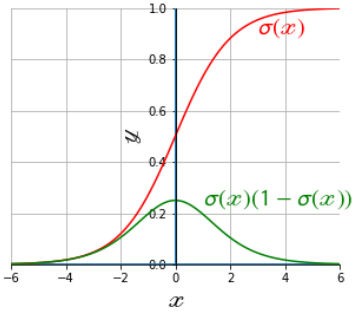


$$f(x^{(i)}) = g\left(\sum_{j=0}^{d} w_j x_j^{(i)}\right)$$

Activate function g

step (sign), sigmoid, tanh, relu, softmax, ...

Sigmoid activation function



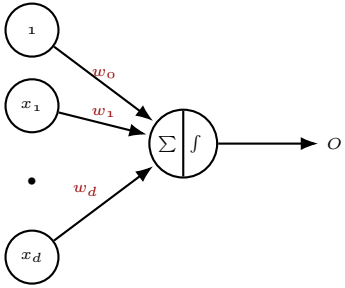
– $e = 2.71828...$ Continuous functions, differentiable, use for Gradient desent.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\frac{\partial \sigma}{\partial x} = \sigma(x)(1 - \sigma(x))$$

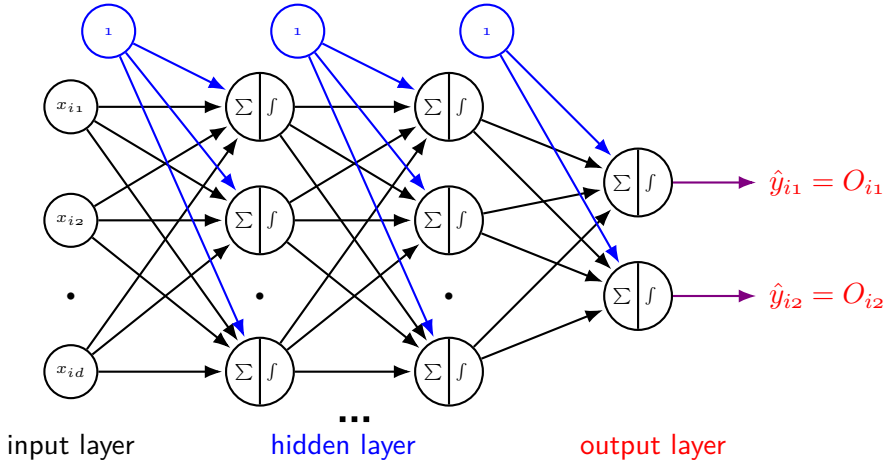
– Useful for tiny change in the weight.

Perceptron (one neuron)



$$Net = \sum_{j=0}^{d} w_j x_j \ ; \qquad O = g(Net) \ ; \qquad g\text{: Activate Function}$$

Neural Network (many neurons)



Loss function

– Network consider $\mathcal{K}$ outputs.

– $y_k \triangleq t_k$: Target $k^{th} \in \mathcal{K}$ of example $(x, y) \in \mathcal{D}$

– $\hat{y}_k \triangleq o_k$: Output $k^{th} \in \mathcal{K}$ of networks of $(x, y) \in \mathcal{D}$

Loss function:

Sum the errors over all of the network output units

$$Loss(w) \triangleq E(w) = \frac{1}{2} \sum_{\mathcal{D}} \sum_{\mathcal{K}} (y_k - \hat{y}_k)^2$$

$$= \frac{1}{2} \sum_{\mathcal{D}} \sum_{\mathcal{K}} (y_k - o_k)^2$$

$$= \frac{1}{2} \sum_{\mathcal{D}} \sum_{\mathcal{K}} (t_k - o_k)^2$$

Employs gradient descent algorithm to minimize $E(w)$.

Loss function for one example

– One example $e = (x, y)$.

– After forward network, we have k out put $o_k \triangleq \hat{y}_k$.

– Error all k out put o_k throught w denote by $E_e(w)$:

$$E_e(w) = \frac{1}{2} \sum_k (y_k - o_k)^2$$

Employs Gradient descent

The Backprop weight update rule is based on the gradient descent method w (abandon index e)

$$w_{ij} = w_{ij} - \eta \frac{\partial E(w)}{\partial w_{ij}}$$

η: learning rate.

Backpropagation algorithm

Notations:

- x_{ij}: the input from node i to unit j.
- w_{ij}: the weight associated with x_{ij}.
- $z_j = \sum w_{ij} x_{ij}$, weighted sum of inputs for unit j.
- o_j: output computed by unit j.
- σ: the sigmoid function.
- $\mathcal{K}$: the set of units in the final layer of the network.
- $succ(j)$: the set of units whose immediate inputs from the output of unit j.

Chain Rule

Chain rule to write:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial z_j} \frac{\partial z_j}{\partial w_{ij}}$$

since,

$$\frac{\partial z_j}{\partial w_{ij}} = \left(\sum w_{ij} x_{ij}\right)' = (w_{ij} x_{ij})' + \left(\sum_{k \neq i} w_{kj} x_{kj}\right)' = x_{ij} + 0$$

so,

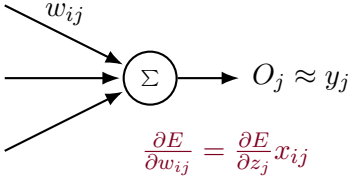
$$\boxed{\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial z_j} x_{ij}}$$

Consider two cases: the case where unit j is an output for the network, and the case where j is an internal unit.

Two cases in calculating $\frac{\partial E}{\partial z_j}$

Backpropagation algorithm

– **Case 1: Neuron j is an output neuron**



$$\frac{\partial E}{\partial z_j} = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial z_j}$$

$$\frac{\partial E}{\partial o_j} = \frac{\partial}{\partial o_j} \frac{1}{2} \sum_k (y_k - o_k)^2 \quad , \quad (k \in \mathcal{K})$$

$$\frac{\partial}{\partial o_j} (y_k - o_k)^2 = 0 \text{ for all output units } k \text{ except when } k = j.$$

$$\begin{aligned} \frac{\partial E}{\partial o_j} &= \frac{\partial}{\partial o_j} \frac{1}{2} (y_j - o_j)^2 \\ &= \frac{1}{2} 2 (y_j - o_j) \frac{\partial (y_j - o_j)}{\partial o_j} \\ &= -(y_j - o_j) \end{aligned}$$

Backpropagation algorithm

Since $o_j = \sigma(z_j)$

$\frac{\partial o_j}{\partial z_j} = \sigma(z_j)(1 - \sigma(z_j))$. therefore,

$\frac{\partial o_j}{\partial z_j} = \frac{\partial \sigma(z_j)}{\partial z_j} = o_j(1 - o_j)$

substituting, we obtain:

$$\frac{\partial E}{\partial z_j} = -(y_j - o_j)\ o_j\ (1 - o_j)$$

so

$$w_{ij} = w_{ij} + \eta\ (y_j - o_j)\ o_j\ (1 - o_j)\ x_{ij}$$

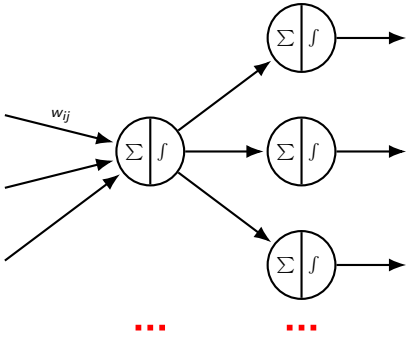
we will note:

$$\boxed{\delta_j = -\frac{\partial E}{\partial z_j} = (y_j - o_j)\ o_j\ (1 - o_j)}$$

$$\boxed{w_{ij} = w_{ij} + \eta\ \delta_j\ x_{ij}}$$

Backpropagation algorithm

– **Case 2: Neuron j is a hidden neuron**



$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial z_j} x_{ij}$$

Backpropagation algorithm

$$
\begin{aligned}
\frac{\partial E}{\partial z_j} &= \sum_{k \in succ(j)} \frac{\partial E}{z_k} \; \frac{\partial z_k}{\partial z_j} \\
&= \sum_{k \in succ(j)} -\delta_k \; \frac{\partial z_k}{\partial z_j} \\
&= \sum_{k \in succ(j)} -\delta_k \; \frac{\partial z_k}{\partial o_j} \; \frac{\partial o_j}{\partial z_j} \\
&= \sum_{k \in succ(j)} -\delta_k \; w_{jk} \; \frac{\partial o_j}{\partial z_j} \\
&= \sum_{k \in succ(j)} -\delta_k \; w_{jk} \; o_j \; (1 - o_j)
\end{aligned}
$$

Backpropagation algorithm

note

$$\boxed{\delta_j = -\frac{\partial E}{\partial z_j} = \; o_j \; (1 - o_j) \sum_{k \in succ(j)} \delta_k \; w_{jk}}$$

and

$$\boxed{w_{ij} = w_{ij} + \eta \; \delta_j \; x_{ij}}$$

Backpropagation algorithm

Input:

(x, y) training example.
η learning rate (e.g., $\eta = 0.1$).
n_i the number of network inputs.
n_h the number of units in the hidden layer.
n_o the number of output units.

The input from unit i into unit j is denoted x_{ij}, and the weight from unit i to unit j is denoted w_{ij}.

Output:

A neural network with one input layer, one hidden layer and one output layer with n_i, n_h and n_o number of neurons respectively and all its weights.

1. Create_feedforward_network (n_i, n_h, n_o)
2. Initialize all network weights to small random numbers (e.g., between -.05 and .05).

Backpropagation algorithm

3. Repeat until convergence: *small changes in the weights*
   For each training example (x, y)
   3.1 **Feed forward**: Propagate example x through the network and compute the output o_j from every neuron.
   3.2 **Propagate backward**: Propagate the errors backward.
   Case 1 For each output neuron k, calculate its error

$$\boxed{\delta_k = o_k(1 - o_k)(y_k - o_k)}$$

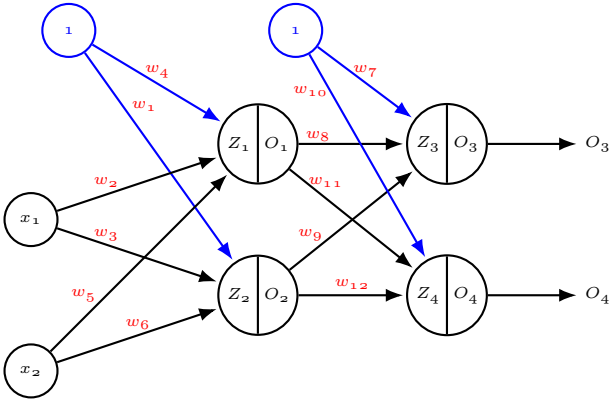
   Case 2 For each hidden neuron h, calculate its error

$$\boxed{\delta_h = o_h(1 - o_h) \sum_{k \in succ(h)} w_{hk}\delta_k}$$

   3.3. **Update each weight**:

$$\boxed{w_{ij} = w_{ij} + \eta\ \delta_j\ x_{ij}}$$

Example



$(x_1 = 0.2\ ,\ x_2 = 0.3)\ \ (y_1 = 1\ ,\ y_2 = 0)$

$w_1 = w_4 = 0.01\ ,\ w_2 = 0.02\ ,\ w_3 = 0.03\ ,\ w_5 = 0.05\ ,\ w_6 = 0.06$

$w_7 = w_{10} = 0.1\ ,\ w_8 = 0.08\ ,\ w_9 = 0.09\ w_{11} = 0.11\ w_{12} = 0.12$

Step by step

$\eta = 0.1$

Feed Forward:

$z_1 = x_1 w_2 + x_2 w_3 + w_4 = 0.023$

$o_1 = \sigma(z_1) = 0.506$

$z_2 = x_1 w_5 + x_2 w_6 + w_1 = 0.038$

$o_2 = \sigma(z_1) = 0.509$

$z_3 = o_1 w_8 + o_2 w_9 + w_7 = 0.186$

$o_3 = \sigma(z_3) = 0.546$

$z_4 = o_1 w_{11} + o_2 w_{12} + w_{10} = 0.217$

$o_4 = \sigma(z_4) = 0.554$

Propagate backward:

$\delta_3 = o_3(1 - o_3)(y_1 - o_3) = 0.112$

$\delta_4 = o_4(1 - o_4)(y_2 - o_4) = -0.137$

$\delta_1 = o_1(1 - o_1)(w_8\delta_3 + w_{11}\delta_4) = -0.002$ $\delta_2 = o_2(1 - o_2)(w_9\delta_3 + w_{12}\delta_4) = -0.002$

Step by step

Propagate backward Update weights:

$$
\begin{aligned}
w_7 &= w_7 + \eta\ \delta_3\ 1 &&= 0.112\\
w_8 &= w_8 + \eta\ \delta_3\ o_1 &&= 0.086\\
w_9 &= w_9 + \eta\ \delta_3\ o_2 &&= 0.096\\
w_{10} &= w_{10} + \eta\ \delta_4\ 1 &&= 0.085\\
w_{11} &= w_{11} + \eta\ \delta_4\ o_1 &&= 0.103\\
w_{12} &= w_{12} + \eta\ \delta_4\ o_2 &&= 0.113\\
w_1 &= w_1 + \eta\ \delta_2\ 1 &&= 0.010\\
w_2 &= w_2 + \eta\ \delta_1\ x_1 &&= 0.020\\
w_3 &= w_3 + \eta\ \delta_1\ x_2 &&= 0.030\\
w_4 &= w_4 + \eta\ \delta_1\ 1 &&= 0.010\\
w_5 &= w_5 + \eta\ \delta_2\ x_1 &&= 0.045\\
w_6 &= w_6 + \eta\ \delta_2\ x_2 &&= 0.060
\end{aligned}
$$

Repeat until Convergence (small changes in the weights)

References